Nexacro Module Developer Guide

24.0.0.600

NEXACRO

TOBESOFT strives to maintain the accuracy of information provided in this document and will consistently modify and supplement it. However, we cannot guarantee accuracy of information in this document. TOBESOFT assumes no responsibility for errors and omissions. No liability is assumed for damages resulting from the use of information contained herein.

Users of this document implicitly agree that they understand the contents of this disclaimer.

Each trademark herein, including the product name of each company, is a solely registered trademark of each company who developed it, and is protected by patent laws and the copyright laws, and should be used for the purpose of reference only.

TOBESOFT Co.,Ltd.
2-5F INTOPS Building, 617 Bongeunsa-Ro, Gangnam-Gu,
Seoul 06083, Korea
www.tobesoft.com

Table of Contents

Copyright and Disclaimer ..... **ii**

Table of Contents ..... **iii**

1. About Nexacro Module Developer **1**

1.1 Installation & System Requirements 1

1.2 How to Run the Program 2

2. Module Project Generation & Deployment **3**

2.1 Generating Project / Object 3

2.2 Configuring Composite Component Screen 5

2.3 Generating Module Installation File 9

2.4 Installing Module & Placing Components on Screen 11

2.5 Sharing Module Project 13

3. Adding Objects **14**

3.1 Common Input Items 15

3.2 Composite Component 15

3.3 Visible Object 16

3.4 Other Objects 17

3.4.1 Invisible Object, EventInfo Object 17

3.4.2 Device Adaptor, Protocol Adaptor 18

3.5 Entering Meta-info Details 19

4. Adding Properties, Methods & Events **20**

4.1 Adding Properties 20

4.1.1 Adding String Property 20

4.1.2 Adding Enum Property 23

4.1.3 Processing Property Value when Running Components 27

4.2 Adding Methods 28

4.2.1 Adding Parameterless Method (getDayCount) 28

4.2.2 Adding Parameterized Method (addDay) 31

4.3 Adding Events 34

4.4 Checking Added Properties, Methods & Events after Module Installation ... 37
4.4.1 Displaying Added Properties & Methods in Property Window ... 37
4.4.2 Displaying Added Properties, Methods, & Events when IntelliSense Is Activated ... 38
4.4.3 Checking Method Operation ... 38
4.4.4 Checking Event Operation ... 39
4.5 Metainfo Editing (Properties, Methods, & Events) ... 40
4.5.1 Properties ... 41
4.5.2 Methods ... 43
4.5.3 Events ... 44
5. Specifying Individual Style of Components ... 45
5.1 Editing XCSS File & Applying to Screen ... 45
5.2 Installing Module & Checking Applied Style ... 48
6. Editing Metainfo ... 49
6.1 Editing Object Metainfo ... 49
6.1.1 Items Automatically Set when Generating Object ... 50
6.1.2 Items that Can Be Set ... 50
6.2 Common Information Metainfo Editing ... 51
6.2.1 Adding Items ... 51
Enum ... 52
Unit ... 53
Refresh ... 54
Display ... 55
6.2.2 Editing by Copying Existing Item ... 57
7. Utilizing Object Interface Functions ... 60
7.1 Checking Object Interface List in Property Window ... 60
7.1.1 XCDL File (Composite Component) ... 60
7.1.2 JS File (Object Other than Composite Component) ... 61
7.2 Adding Object Interface Function ... 61
7.2.1 Adding in Property Window ... 61
7.2.2 Adding from Menu ... 62
7.3 Checking Object Interface Function Help ... 63
8. Contents Editor ... 65
8.1 Setting with Metainfo Contents Property ... 65
8.1.1 Setting Contents Properties ... 65
Setting in Component Wizard ... 65
Setting in Metainfo ... 66
8.1.2 Setting Contents Format & Contents Type Properties ... 67
Contents Format ... 68
Contents Type ... 68

8.1.3 Setting Contents Item Properties ... 69
8.1.4 Creating Simple Sample ... 70
Generating Module Project ... 70
Configuring Screen ... 71
Setting Contents Item ... 72
Writing Script ... 73
Checking Operation in Emulator ... 76
Installing Module & Checking Operation ... 78
8.2 Setting with Object Contentseditor Property ... 80
9. Creating Help ... 82
9.1 Installing HTML Help Workshop ... 82
9.2 Generating Simple CHM File ... 83
9.3 Adding Help to Module Project ... 87
10. Setting Icons ... 89
10.1 Importing Icon File ... 89
10.2 Deployment after Setting Icon to Module Object ... 90
10.3 Creating Icon File ... 91
10.3.1 Finding Default Icon Image ... 91
10.3.2 Generating ico File ... 92
11. Option Settings ... 93
11.1 Project ... 93
11.1.1 General ... 93
11.1.2 Deploy ... 94
11.2 Environment ... 94
11.2.1 General ... 95
11.2.2 Startup ... 95
11.2.3 Auto Recover ... 96
11.2.4 Font and Color ... 96
11.2.5 Show Information ... 98
11.2.6 Script ... 98
General ... 99
Version ... 99
11.2.7 Generate ... 100
11.2.8 Advanced ... 101
11.3 Form Design ... 101
11.3.1 General ... 102
11.3.2 Guide ... 102
11.3.3 Paste Special ... 104
12. Toolbar & Menu Bar ... 105

12.1 Toolbar Feature 105
12.1.1 Standard 105
12.1.2 Align 106
12.1.3 Bookmark 107
12.1.4 Build 107
12.1.5 Component 108
Selecting and moving components 108
Basic components 108
DeviceAPI 109
Other objects 110
12.1.6 Position 110
12.2 Menu Bar 111
12.2.1 File 111
12.2.2 Edit 113
12.2.3 Assist 114
12.2.4 View 115
12.2.5 Design 116
12.2.6 Layout 118
12.2.7 Generate 118
12.2.8 Deploy 119
12.2.9 Tools 119
12.2.10 Window 120
12.2.11 Help 120

Appendix A. Composite Component Example 122
A.1 Postal Code Search 122
A.2 Toggle Button 127
Appendix B. Example of the protocol adaptor 136
B.1 JSON data processing 136
B.2 Symmetric key encryption and decryption processing (crypto-js) 142

1.

About Nexacro Module Developer

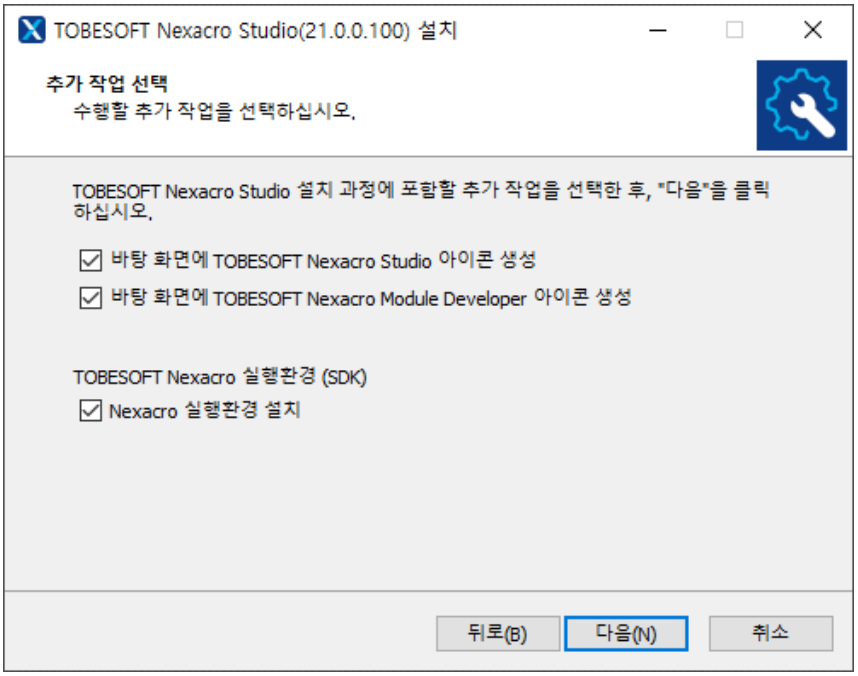
Nexacro Module Developer is a development tool that supports Nexacro based user module generation. The basic menu configuration and usage are similar to Nexacro Studio. The development of the user component that is created by inheriting specific components, as well as the composite component that provides complex function by combining existing components, are supported.

1.1 Installation & System Requirements

nexacro Module Developer is deployed with Nexacro Studio. The system requirements are the same as Nexacro Studio. Please refer to the link below.

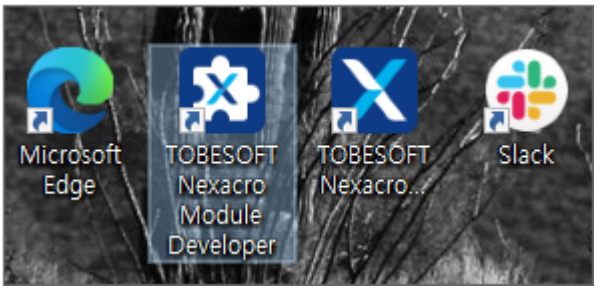
> Nexacro Studio System Requirements

You can specify whether to create a shortcut icon when installing Nexacro Studio.



1.2 How to Run the Program

If you have created a shortcut icon on the desktop, double-click the icon to run nexacro Module Developer.



You can also select the "nexacromoduledeveloper.exe" file from the Nexacro Studio installation path to run it directly.

2.

Module Project Generation & Deployment

The basic process of generating a module is simple. Generate a new project, configure a screen, and generate it as a file that can be deployed in the Deploy step. To generate a more specific module, you need to add features and adjust various settings. This chapter will take a quick look at the basic steps of generating a module.



Composite Component configures the screen in the form of placing the default components on Form, but other objects must be written in the script for all functions. In this chapter, we used Composite Component as an example to better understand the process of creating and deploying the module project.

2.1 Generating Project / Object

A project can have more than one object. For example, in the Chart module project, you can include multiple objects (components) such as Line Chart, Bar Chart, etc. As there must be at least one object in a project, you need to generate objects when generating a project for the first time.

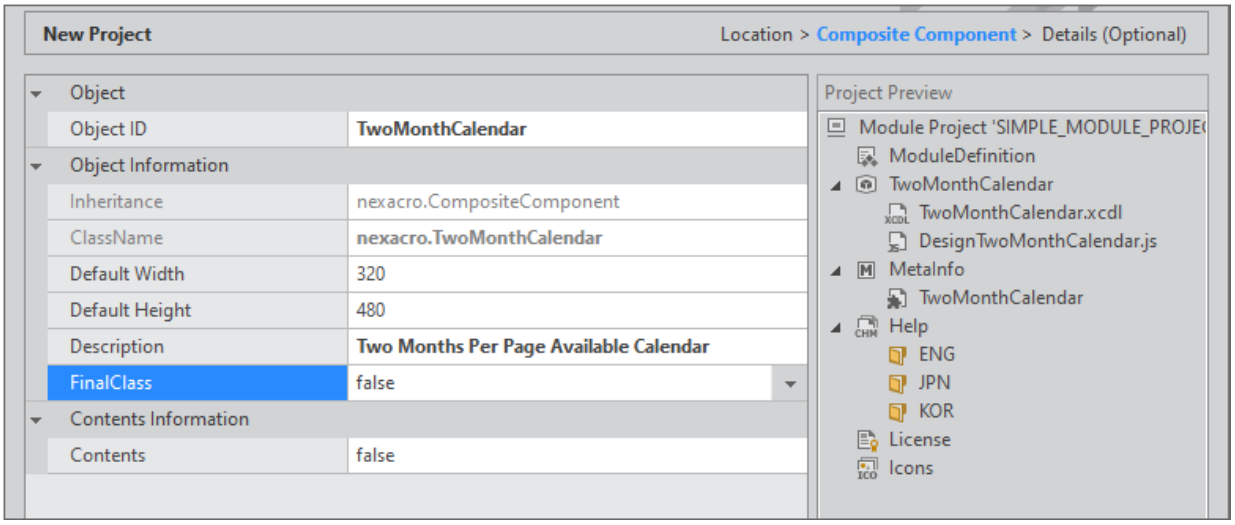
1. Run nexacro Module Developer.
2. Select [File > New > Project] from the menu to start Project Wizard.
3. Select the "Composite Component" item from the Module list.
4. Enter the Project Name and click the [Next] button.



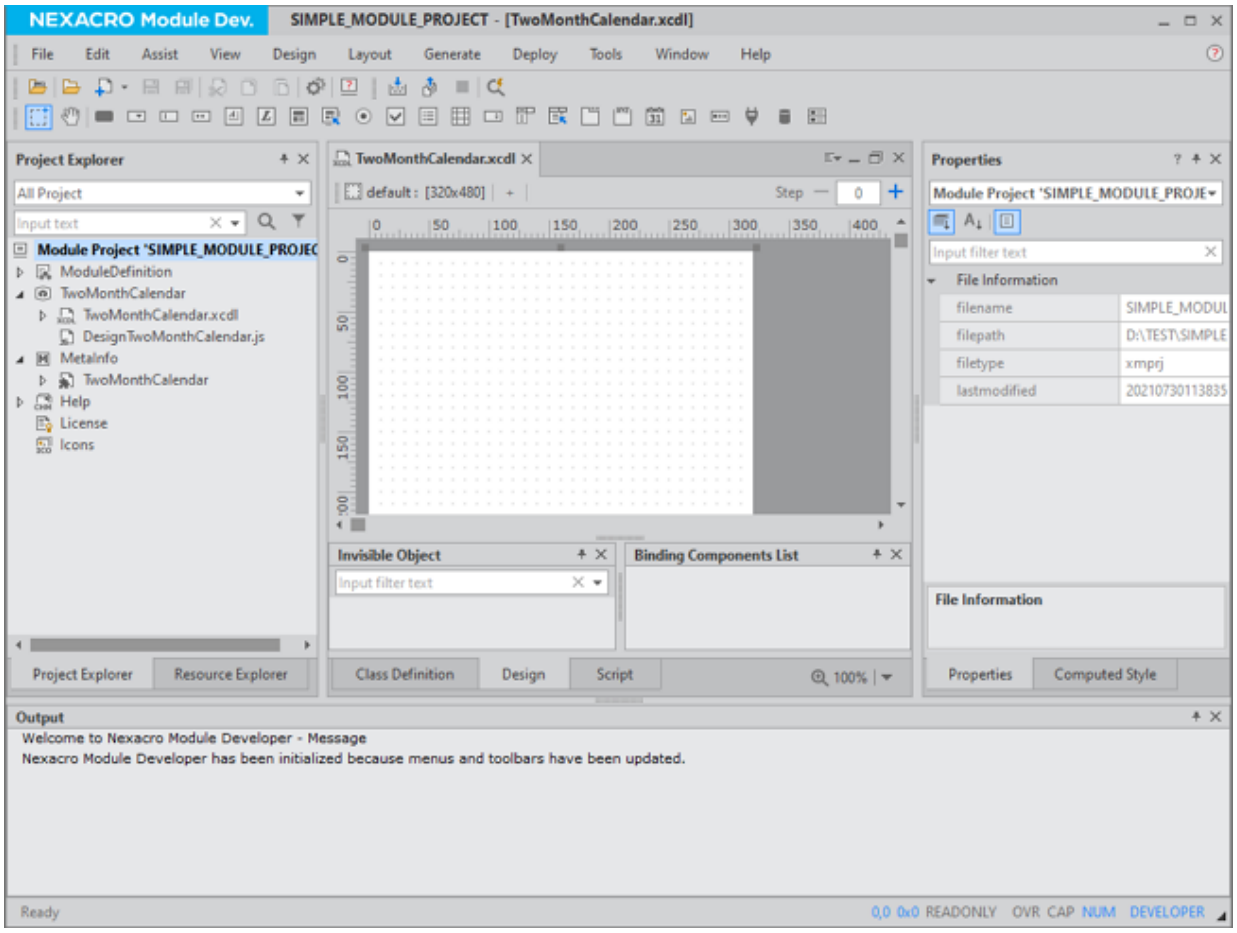
5 Enter the object ID value and click the [Finish] button.

Since the object ID is used as the component name when deploying the module, it is recommended to contain features and characteristics of the component to be generated. In the example, we will generate a component with 2 Calendar components side by side.

When the object ID is entered, the project structure to be generated is displayed in the Project Preview area.



6 The project is generated and the design editing screen where you can place the screen of the generated objects is displayed.



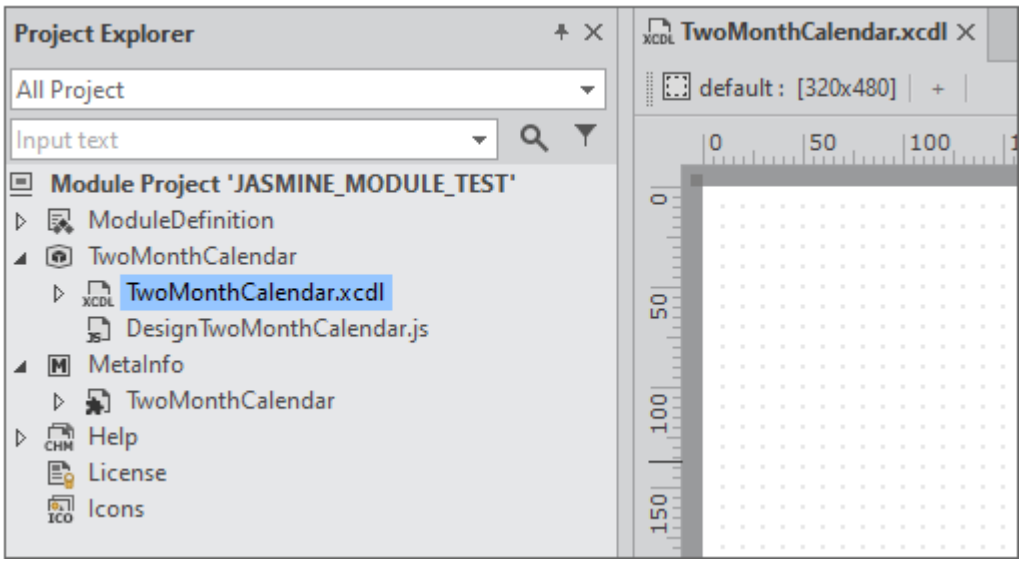
2.2 Configuring Composite Component Screen

It determines how the screen is shown when the user installs the module and places the components on the screen.

In the example, two Calendar components are placed side by side. Then, the type property value of the Calendar component is set to "monthonly" for the user to be able to immediately select the date when the components are placed.

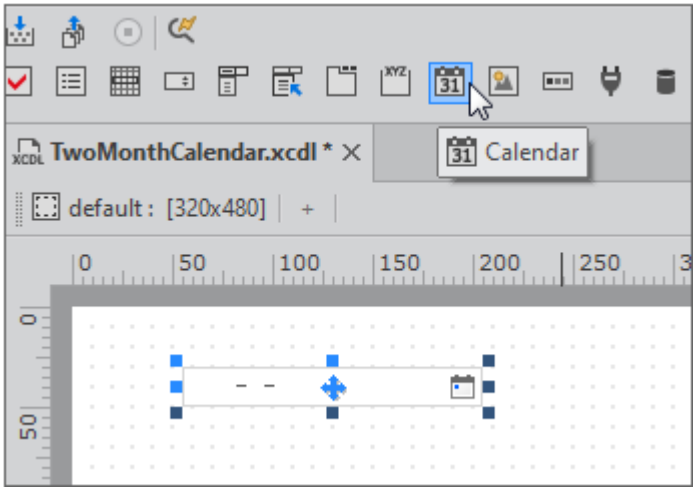
1 In Project Explorer, select the xcdl file under the corresponding object.

When the project is generated for the first time, the xcdl file is displayed as open after the project is generated.

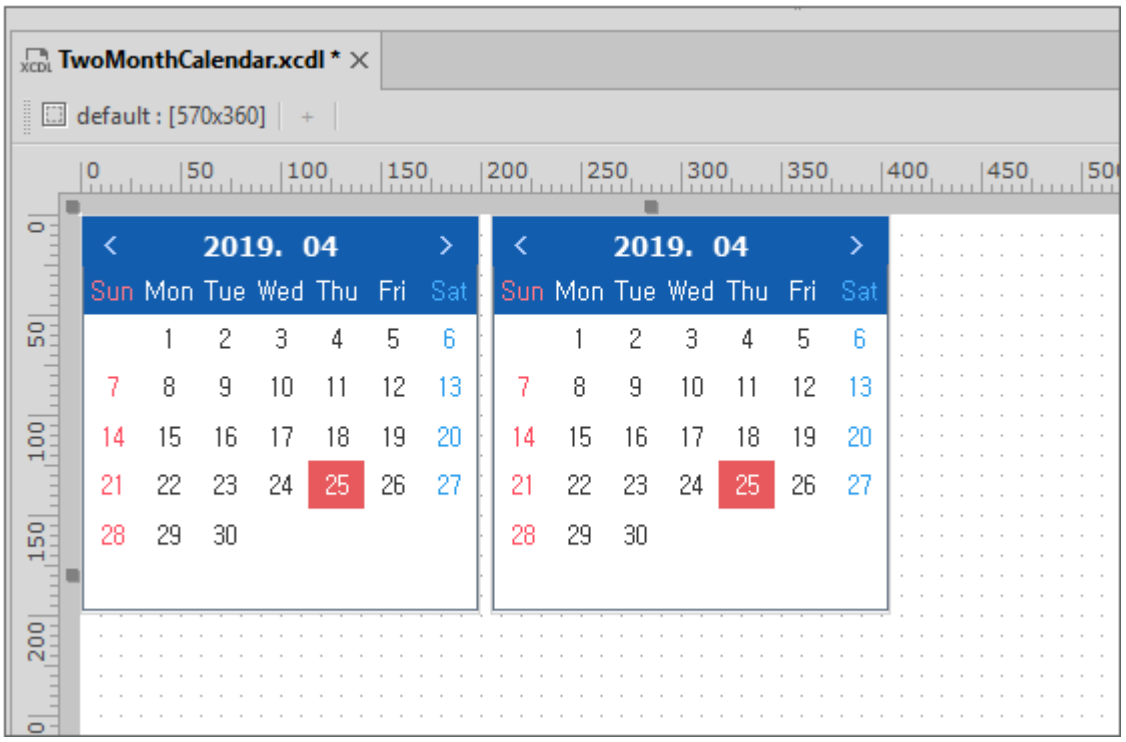


2 Select the Calendar component from the toolbar and place it on the screen.

Placing and adjusting the screen is the same as screen designing in Nexacro Studio.

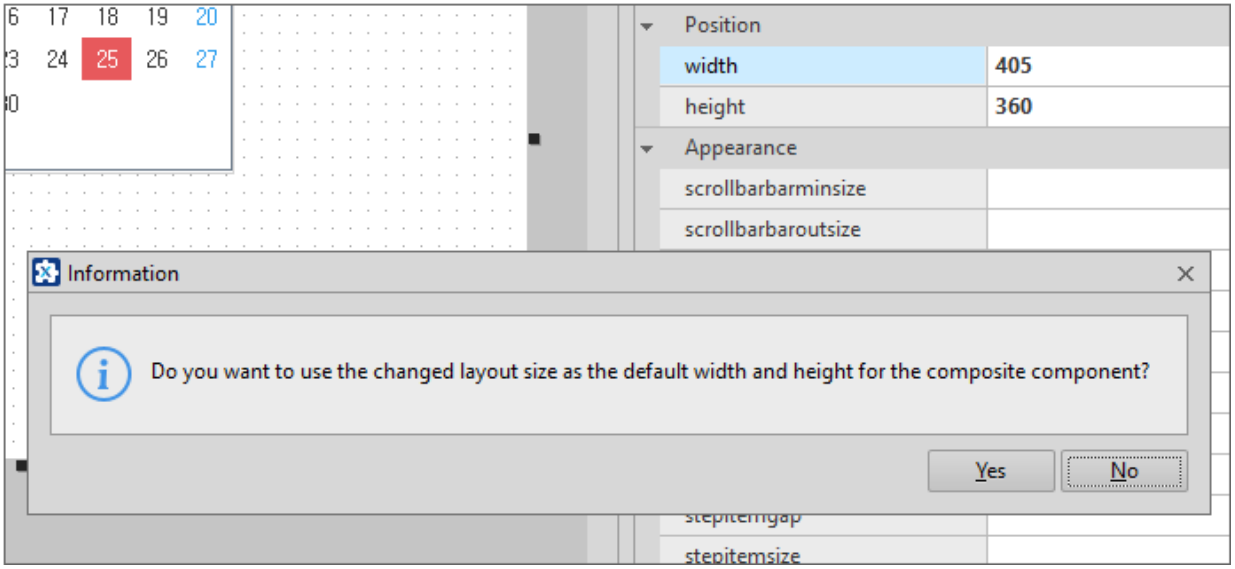


3 Place another Calendar component and set the type property value to "monthonly". Adjust the size of each component to be appropriate.

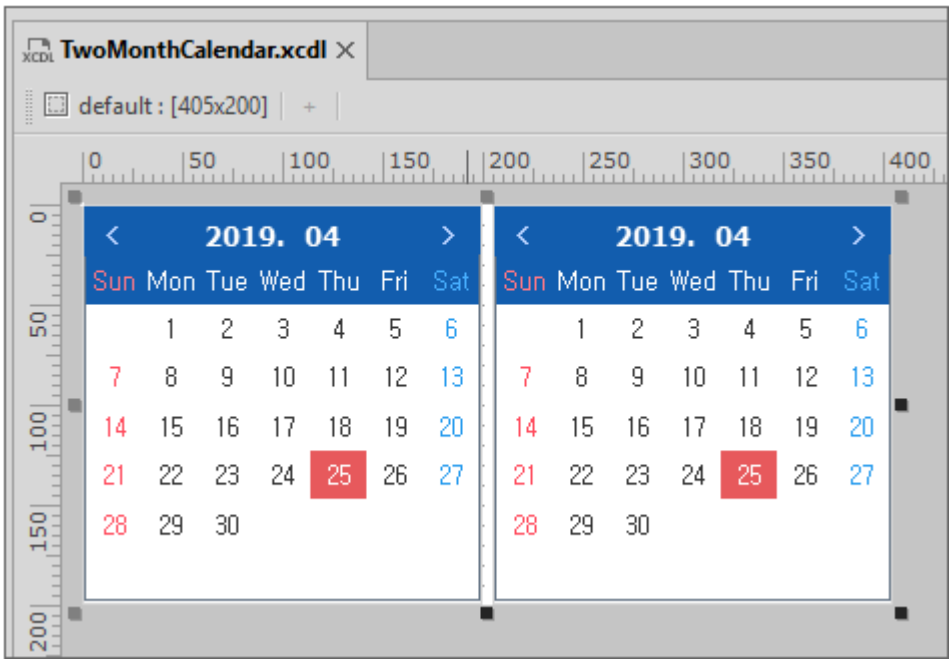


4 Adjust the Form size to the size of the placed component.

As it is modifying the Default Size specified when generating the object, a warning window is displayed when the size is changed. Default Size is the size that is set as default when the user selects and places the component on the screen.

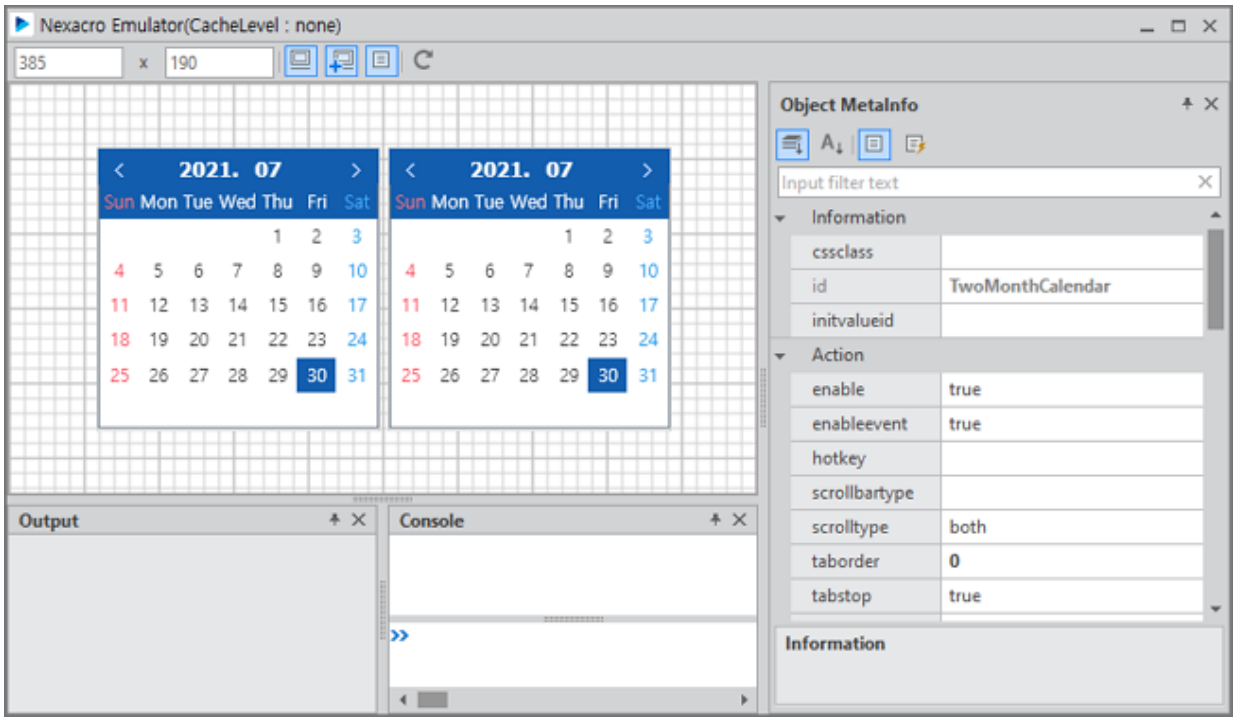


The Form size can vary depending on the use of the composite component. In the example, we set it to perfectly fit the size of two Calendar components.



5 Select the menu [Generate > QuickView] to check whether the generated composite component is displayed properly.

Although the emulator can check the added features, as we did not generate additional features, we can just check how the composite component is shown.

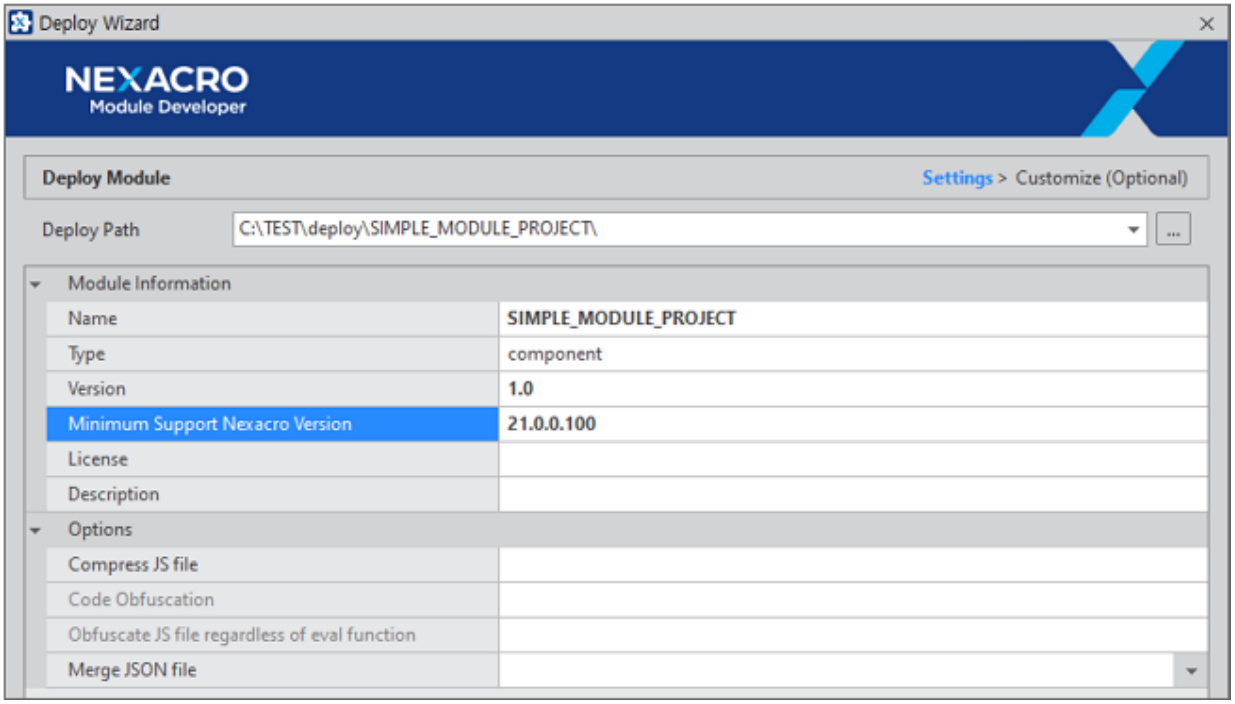


2.3 Generating Module Installation File

The generated project can be generated into a module file to deploy to the user (developer).

1. Select the menu [Deploy > Module Package] and run Deploy Wizard.

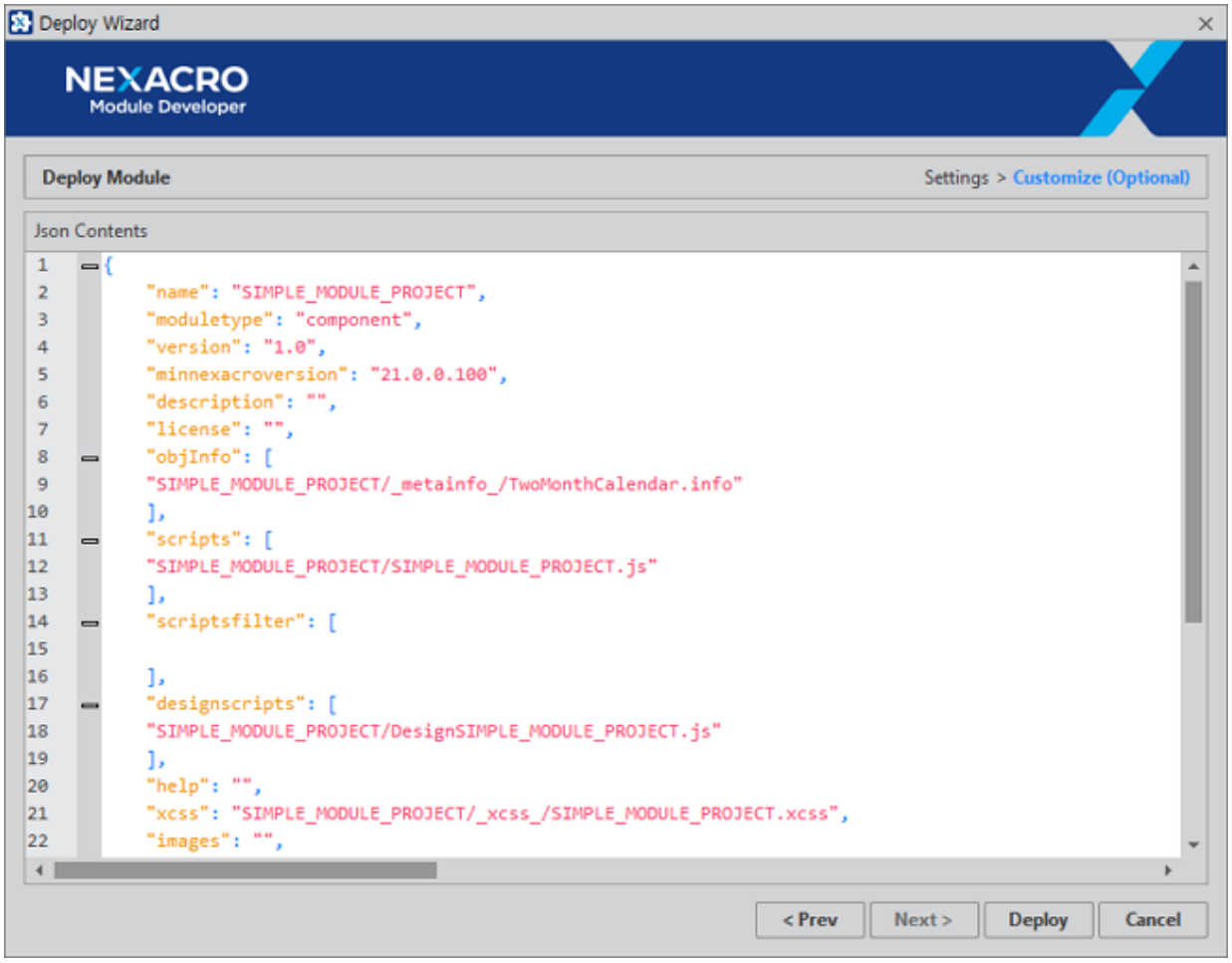
2. Modify Deploy Path to the desired path and set the Version information.



Options can be set upon deployment or in the [Tools > Options > Project > Deploy] option. For more information, see Option Settings.

3. You can edit the JSON file by clicking the [Next] button.

If you do not need the JSON editing step, then click the [Deploy] button instead of clicking the [Next] button as in the previous step.



In the scriptsfilter item, you can set whether to deploy the app according to the operating system after installing the module. For example, with the script called TEST, if you want to deploy only the files corresponding to the operating system, then set the scriptsfilter item as follows.

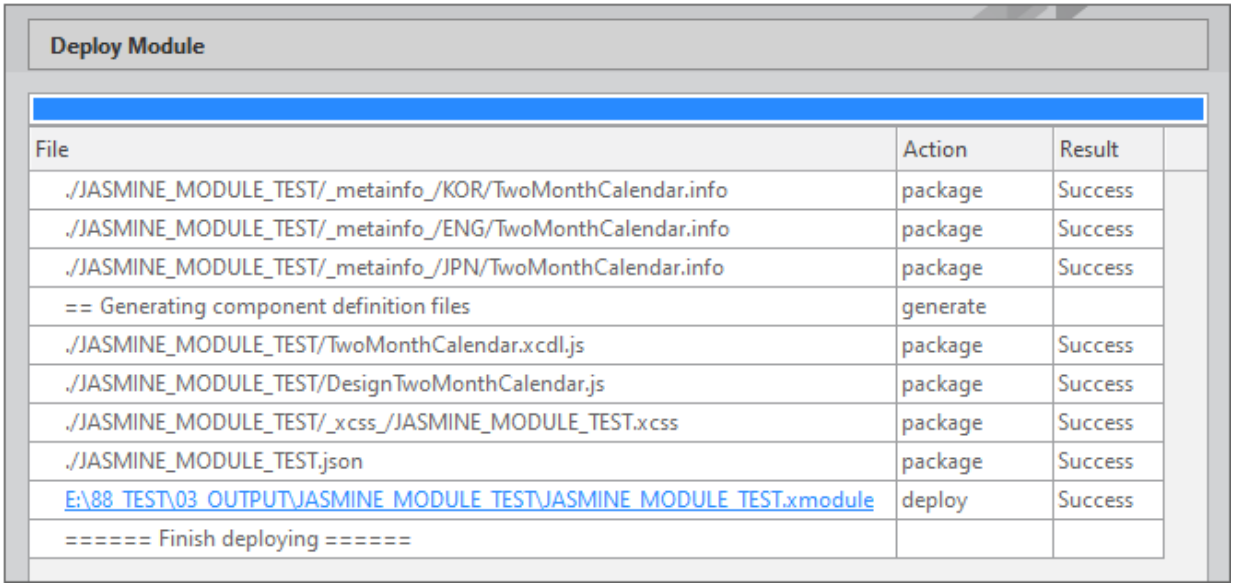
```
"scriptsfilter":[
    {name: "SIMPLE_MODULE_PROJECT/TEST_NRE.js", target:"NRE"},
    {name: "SIMPLE_MODULE_PROJECT/TEST_WRE.js", target:"WRE, iOS_NRE"}
]
```

The items that can be set as the target property value are as follows.

- NRE: Operating system NRE excluding iOS/iPadOS
- iOS_NRE: iOS/iPadOS NRE
- WRE

4 After clicking the [Deploy] button, a file is generated as the xmodule extension.

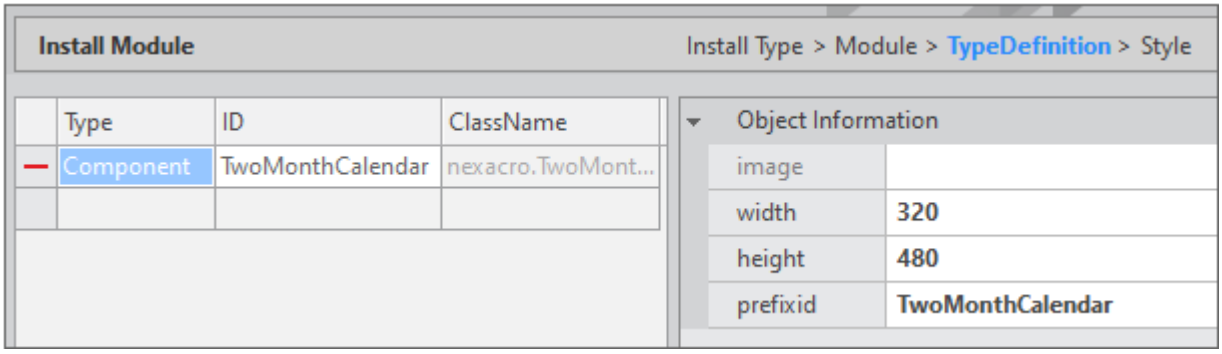
Click the link in blue to open the corresponding folder.



2.4 Installing Module & Placing Components on Screen

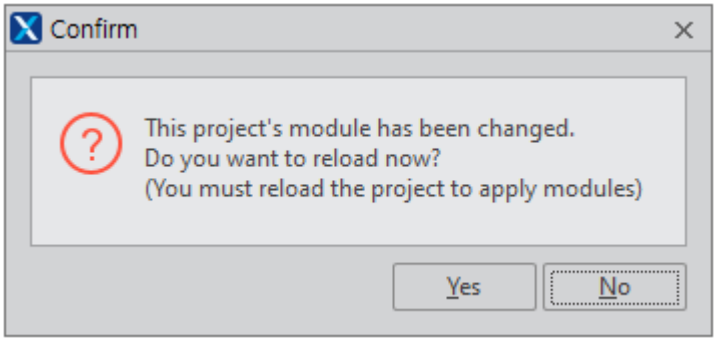
Open a project in Nexacro Studio to install the deployed module file and place the composite component on the screen.

1. Run Nexacro Studio.
2. Generate a new project or run an existing project.
3. Select [File > Install Module] and run Install Module Wizard.
4. Select "Module Package" for Install Type and click the [Next] button.
5. Select the xmodule file generated in nexacro Module Developer and click the [Next] button.
6. Check the name of the composite component to be installed and click the [Next] button.



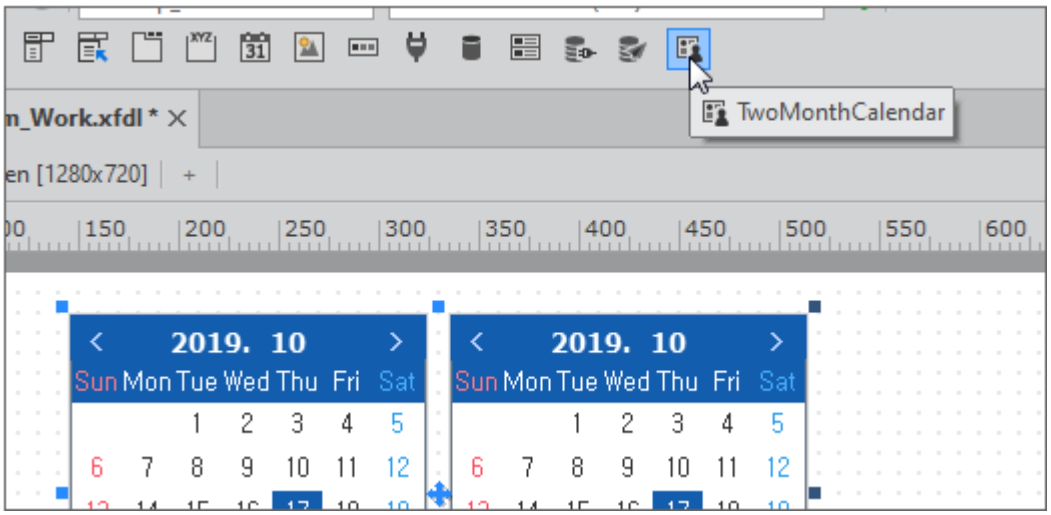
7 Do not check the xcss and image related settings and click the [Install] button.

8 Reload the current project when the installation is complete.



9 Open the Form screen to place the composite component.

10 Select the icon from the toolbar and place it on the screen.



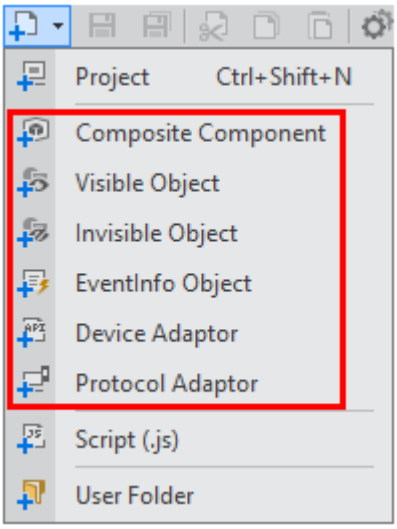
2.5 Sharing Module Project

You can compress and share the entire folder of the generated module project.

3.

Adding Objects

Let us take a look at how to create a new project and add objects. The objects that can be added are as follows.



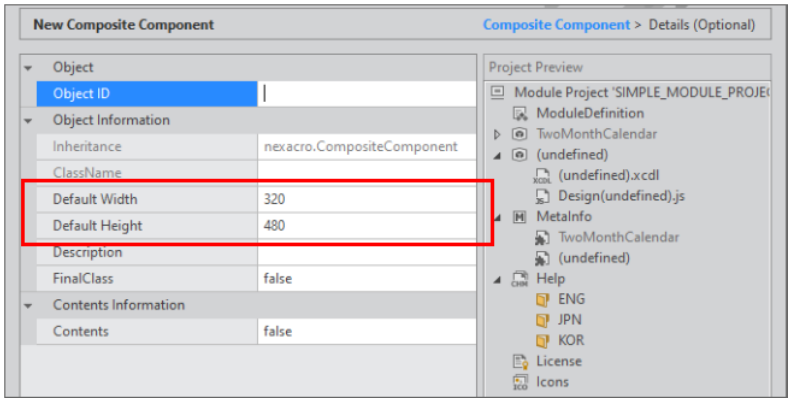
Object	Description
Composite Component	Places 1 or more components on Form. Inherits nexacro.CompositeComponent.
Visible Object	Creates the component shown on the screen. Inherits and implements components such as Button or inherits nexacro.Component to create a new form of component.
Invisible Object	Creates the object not shown on the screen. Inherits and implements objects such as Dataset or inherits nexacro.Object to create a new form of the object.
EventInfo Object	Creates the EventInfo object that is transmitted when the event occurs. Inherits and implements EventInfo like ClickEventInfo or inherits nexacro.EventInfo to create a new form of EventInfo object.
Device Adaptor	This is the object that Inherits and implements nexacro.DeviceAdaptor.
Protocol Adaptor	This is the object that Inherits and implements nexacro.ProtocolAdaptor.

3.1 Common Input Items

Item	Description
Object ID	Sets Object ID. The file name connected to the object in the project is also created based on the Object ID. (It is displayed as undefined in the Project Preview area on the right before entering the Object ID).
Object Information	
Inheritance	Sets or displays the objects to be inherited. Composite Component is fixed as nexacro.CompositeComponent, and for other objects, select the object to inherit from the list on the left.
ClassName	Automatically entered to display according to the Object ID.
Description	Sets the summary description of the object to create. When the object to inherit is selected, Description of the object is displayed.
FinalClass	Sets whether to allow inheritance. When set to false, it is displayed as a list of objects that can be inherited when creating other objects. If you do not want to allow inheritance, then change it to true.
Contents Information	
Contents	Sets whether to use Content Editor.

3.2 Composite Component

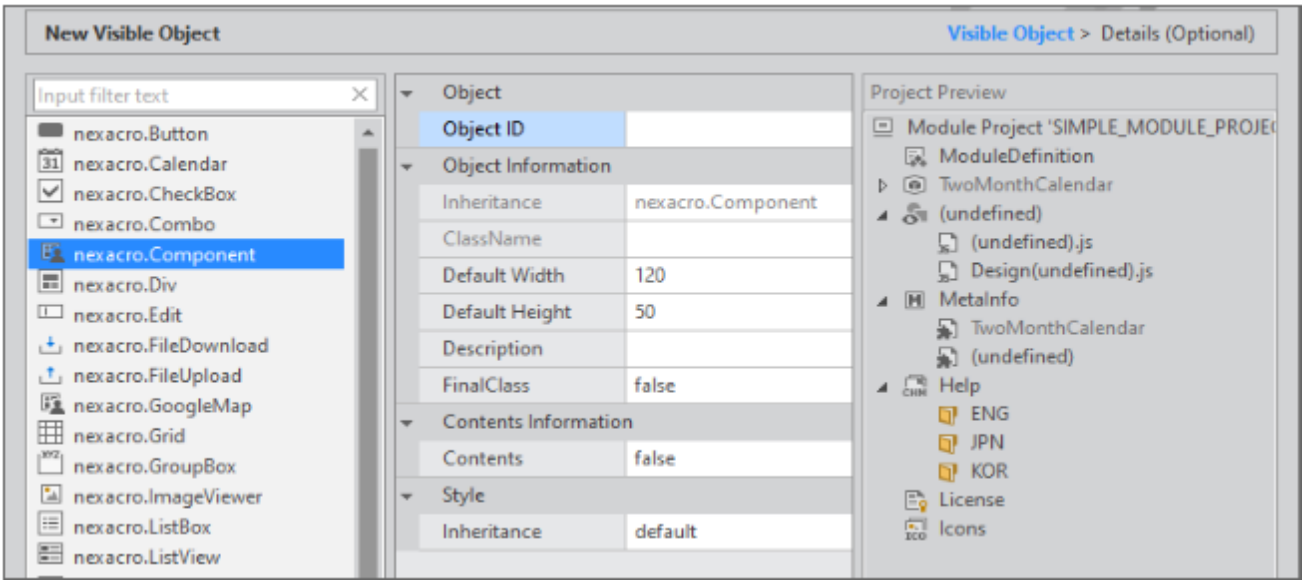
Composite Component is fixed in a structure that inherits nexacro.CompositeComponent.



Item	Description
Object Information	
Default Width	Sets the width property value of the Form object.
Default Height	Sets the height property value of the Form object.

3.3 Visible Object

Create the component that is shown on the screen. You can select the component to inherit and set the style-related property.



Item	Description
Object Information	
Default Width	Sets the default width property value of the component.
Default Height	Sets the default height property value of the component.

Item	Description
Style	
Inheritance	Sets the default style of the component. - default Specify only the minimum style property to classify the component area (-nexa-border: 1px solid red) - theme Imports the style property of the object inherited from the currently used theme.



Notes when setting the Style Inheritance property value to "theme"
- Style properties according to Status are not imported.
- If nexacro.Component is inherited, then style properties are not specified because there is no theme specified.

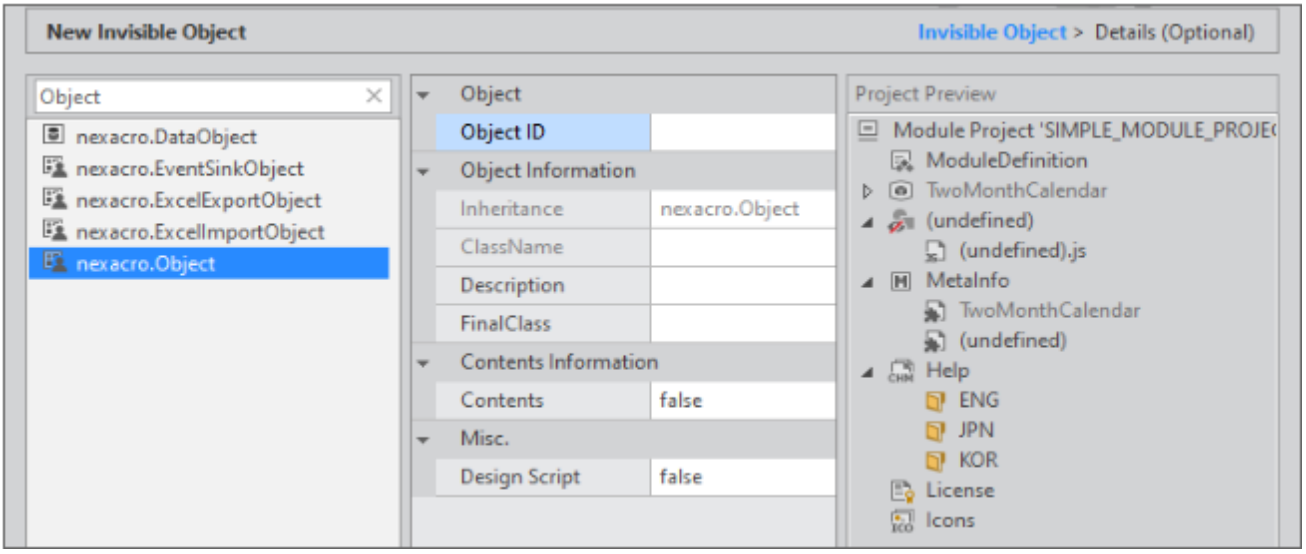
3.4 Other Objects

Create the object that is not displayed on the screen. You can set whether to create the Design Script file.

Item	Description
Misc.	
Design Script	Sets whether to create the Design Script file. It is used to define necessary actions in the Nexacro Studio edit window when installing the module.

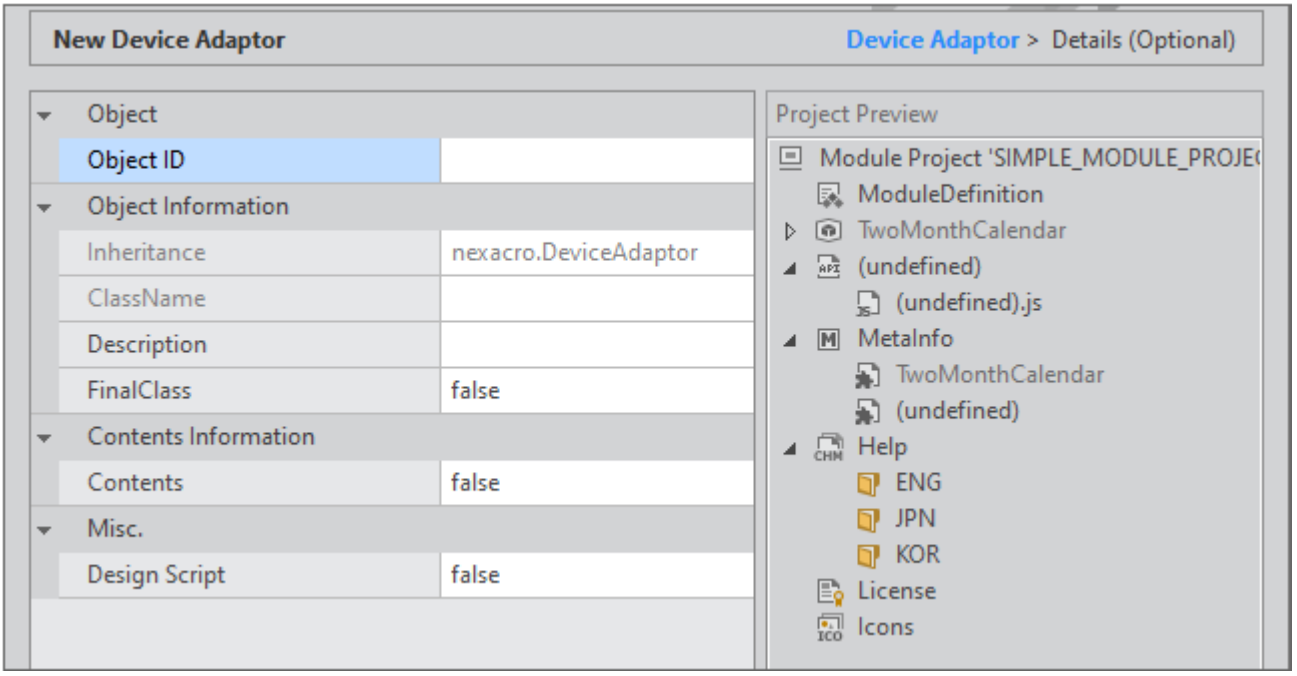
3.4.1 Invisible Object, EventInfo Object

Select the object to inherit and enter the rest of the items.



3.4.2 Device Adaptor, Protocol Adaptor

Enter the rest of the items while the object to inherit is set.

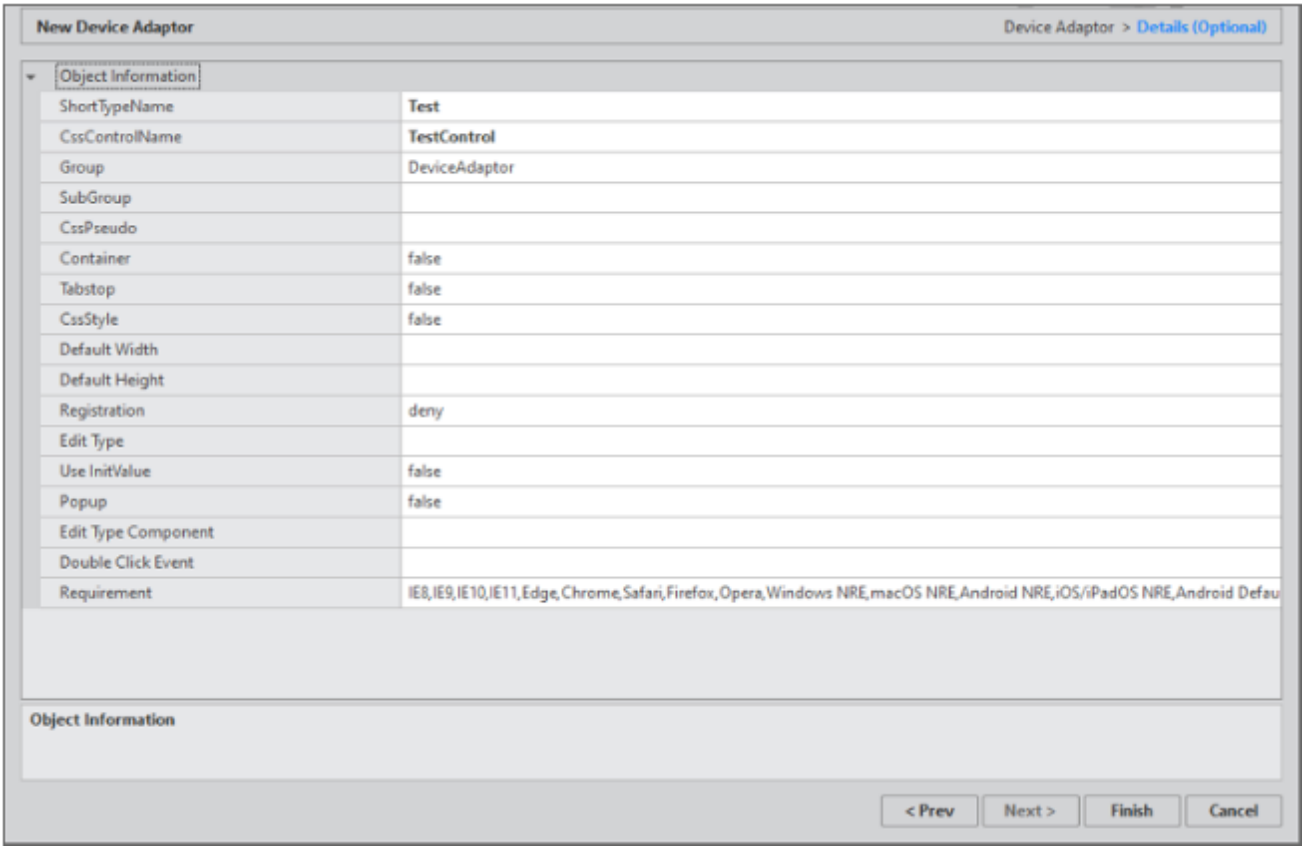


3.5 Entering Meta-info Details

Select the object type to create, enter the items, and click the [Finish] button to create the object. If you want to set the meta-info details of the object, then you can click the [Next] button to set the detailed items.



The meta-info details can be modified even after the object creation.
Please refer to the following.
Editing Metainfo



4.

Adding Properties, Methods & Events

In this chapter, we will look at how to add properties, methods and events to composite components.

4.1 Adding Properties

4.1.1 Adding String Property

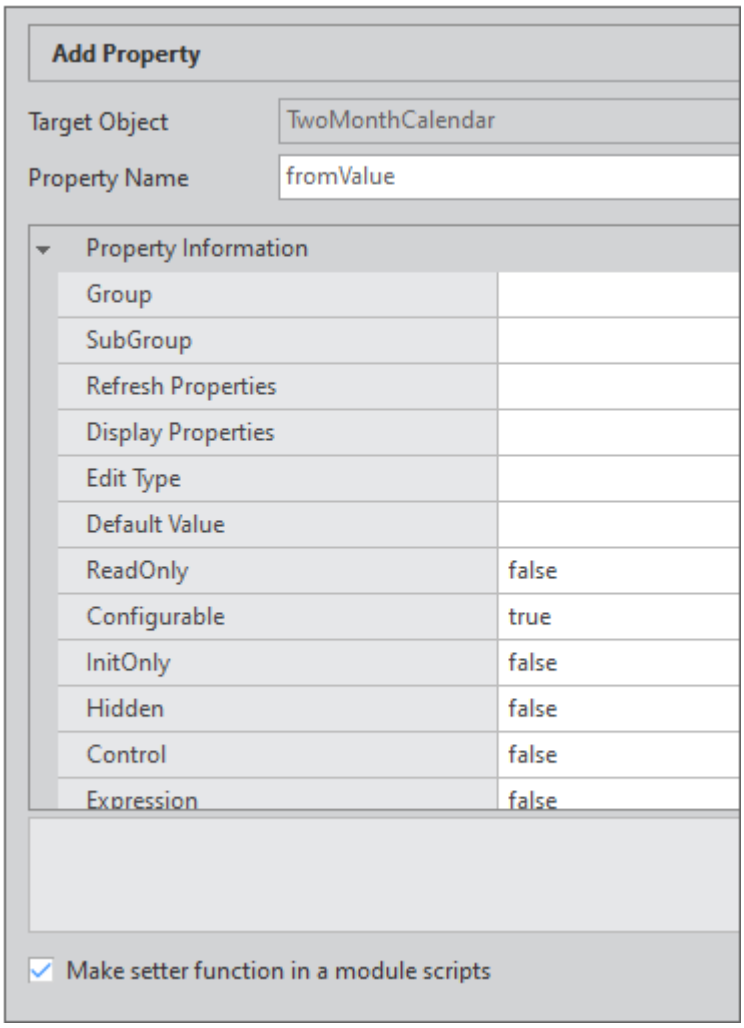
Add properties that can directly access the selected value in the Calendar component. The names of the properties are fromValue and toValue.

1. Select [Edit > Add > Property] item from the menu and run Add Property Wizard.

   You can also select an object in Project Explorer and select [Add > Property] item from the context menu.

2. Check if the object displayed in the Target Object item is the one selected and enter the property name to be added in the Property Name item.

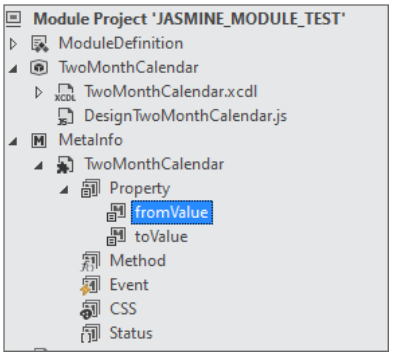
   Property related information can be set in the Property Information item. In the example, the property-related information remains at its default value.



Checking "Make setter function in a module scripts" item automatically generates the Setter Function skeleton code of the property. If any of the metainfo items, unused, readonly, or initonly, is set to true, the corresponding checkbox is disabled.

If the checkbox is disabled or unchecked, the corresponding code will not be added to the xcdl file in the next step.

3 Click the [Finish] button to add the specified property and add the property to the MetaInfo item.



4 You can check that in the xcdl file, the Class Definition tab is activated and the inserted code is available.

```
18      nexacro.TwoMonthCalendar.prototype._p_fromValue = "";
19      nexacro.TwoMonthCalendar.prototype.set_fromValue = function (v)
20      {
21          // TODO : enter your code here.
22
23      };
24
25      Object.defineProperty(nexacro.TwoMonthCalendar.prototype, "fromValue", {
26              get: new Function("return this._p_fromValue"),
27              set: nexacro.TwoMonthCalendar.prototype["set_fromValue"],
28              enumerable : true,
29              configurable : true});
30
31  }
```

5 Modify the script as follows.

When the user sets the fromValue property value, the corresponding property value is delivered as the value of the Calendar component.

```
nexacro.TwoMonthCalendar.prototype._p_fromValue = "";
nexacro.TwoMonthCalendar.prototype.set_fromValue = function (v)
{
    if(this.form.calFrom)
        this.form.calFrom.value = v;
    if(this._p_fromValue != v)
        this._p_fromValue = v;
};
```

6 Select the Calendar component in the design tab and generate the onchanged event function as below. The generated event function can be checked in the Script tab.

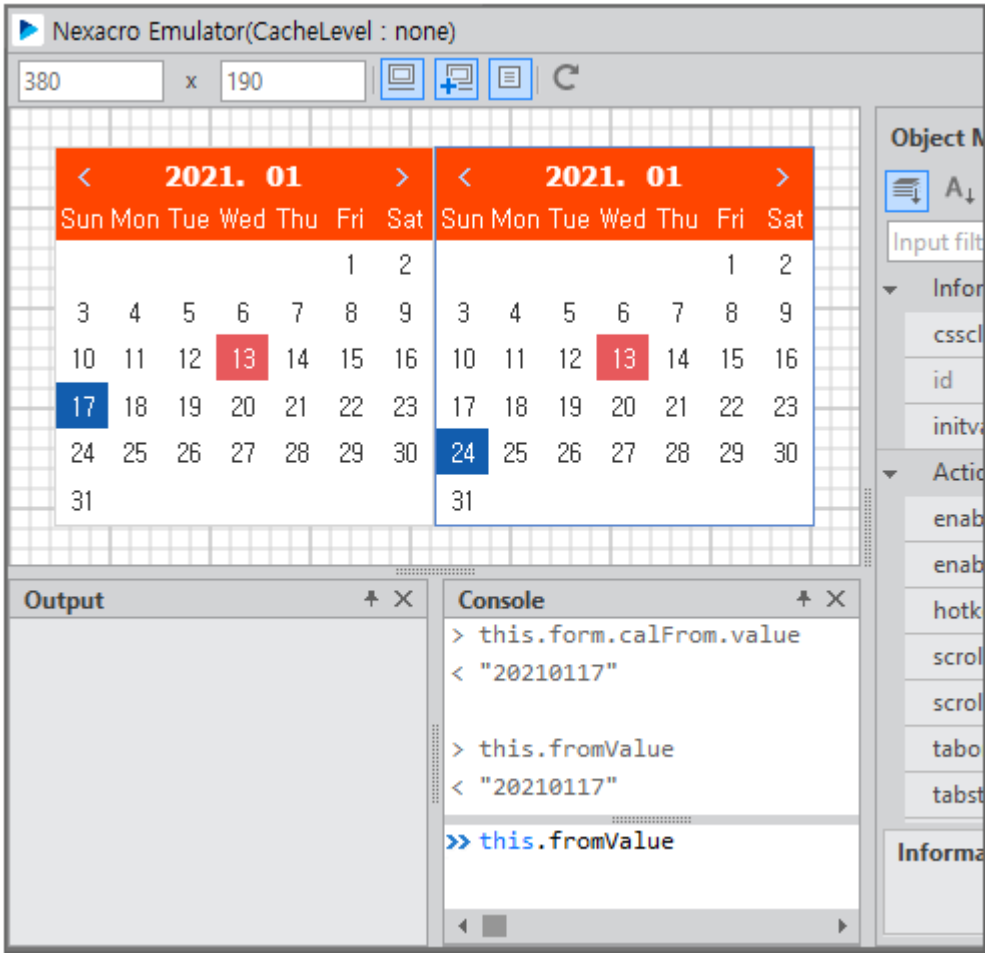
When the selected value in the Calendar component is changed, save the changed value in the fromValue property value.

```
this.calFrom_onchanged = function
(obj:nexacro.Calendar,e:nexacro.ChangeEventInfo)
{
     this.parent.fromValue = e.postvalue;
};
```



In the Script, the keyword, this, refers to the Form object. Therefore, to access the properties of the composite component, you must access via this.parent.

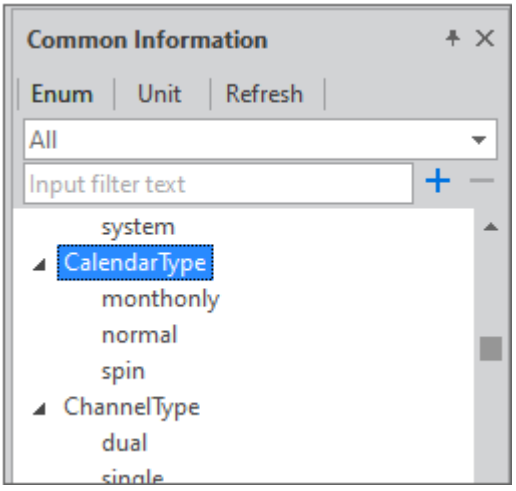
7 Run the emulator, select the date in the first Calendar component, and check the property value in the console window.



8 Add the property corresponding to the second Calendar component and add code.

4.1.2 Adding Enum Property

Enum property refers to a property that can specify a value in the form of enumeration. For example, the type property value of the Calendar component can be selected from 3 values of "normal | monthonly | spin", which is used by adding a property value that can be specified in advance as the EnumInfo property.

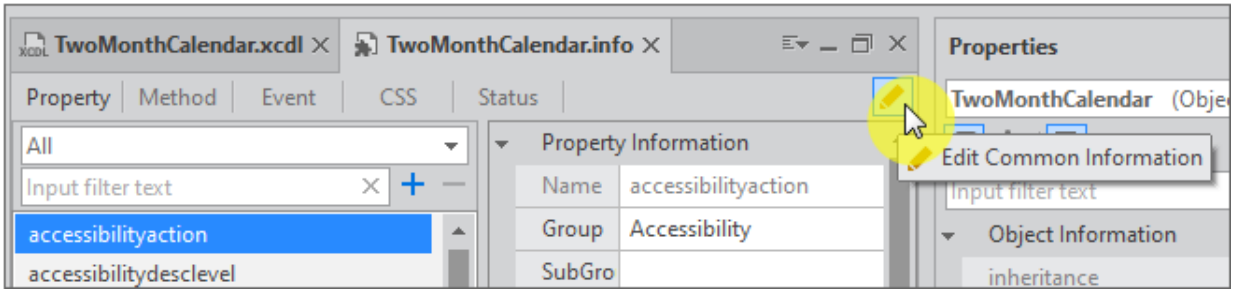


Add a property that can change the style of the Calendar component. The name of the property is colorTheme. Before adding the property, EnumInfo must be added first.

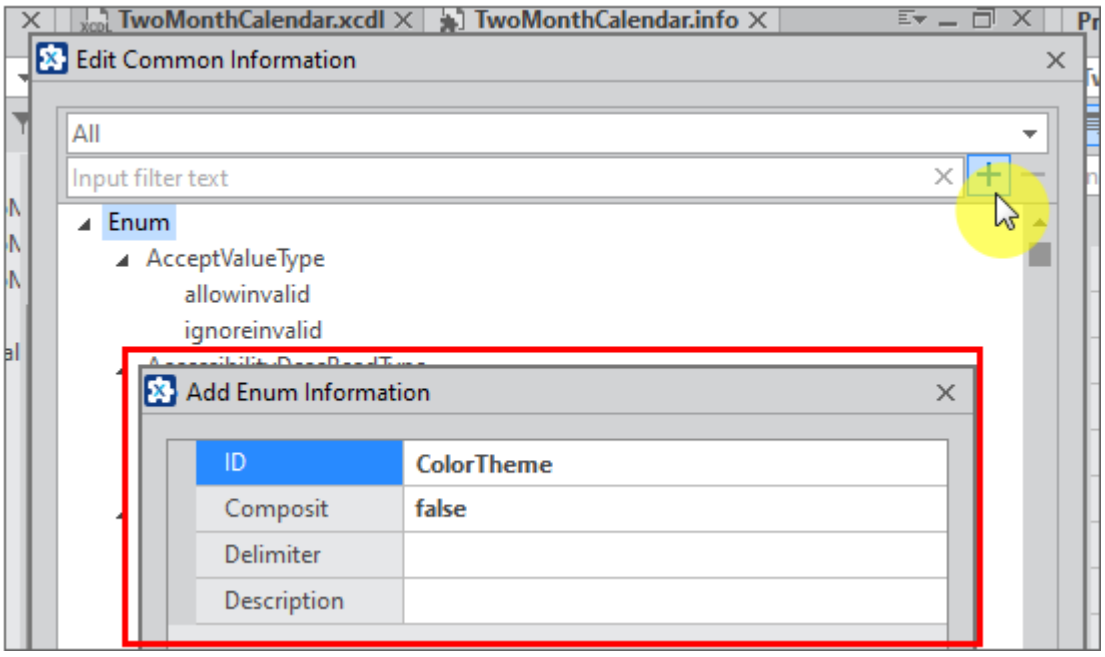
Please refer to the link below for style-related codes. In the below description, it is assumed that the xcss code has been written in advance.
Specifying Individual Style of Components

1. In Project Explorer, double-click the object name under the MetaInfo item or select [Edit] item from the context menu to run the metainfo editor.

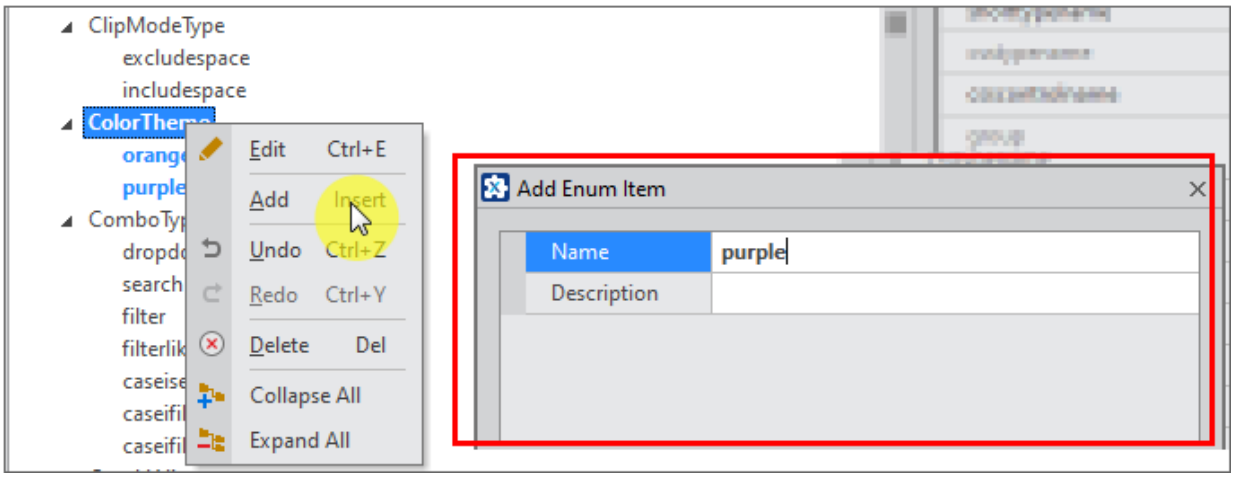
2. Click the [Edit Common Information] icon on the right side of the editor.



3. Click the [+] button, select [Add Info Item], and add an item called "ColorTheme" in the edit window.

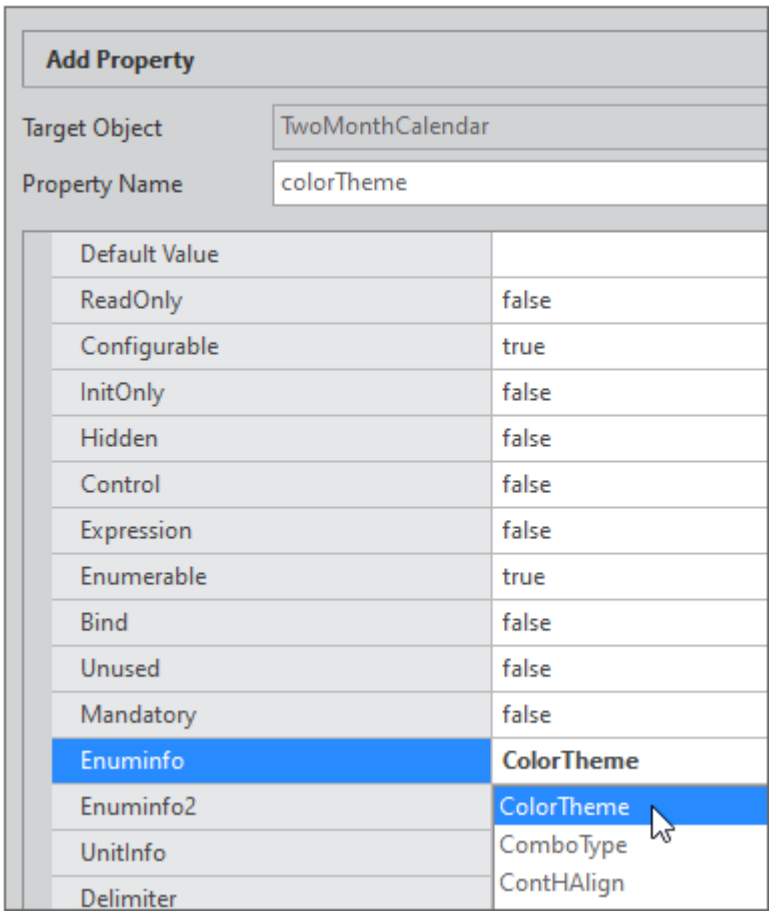


4 Select the added Enum item, select the [Add] item from the context menu, and then add the "orange" and "purple" items in the [Add Enum Item] window.



5 Select [Edit > Add > Property] item from the menu and run Add Property Wizard.

6 Change the Edit Type item to "Enum" in the Property Information item, and then select the "ColorTheme" item created earlier for the Enuminfo item.



7 Modify the script as follows.

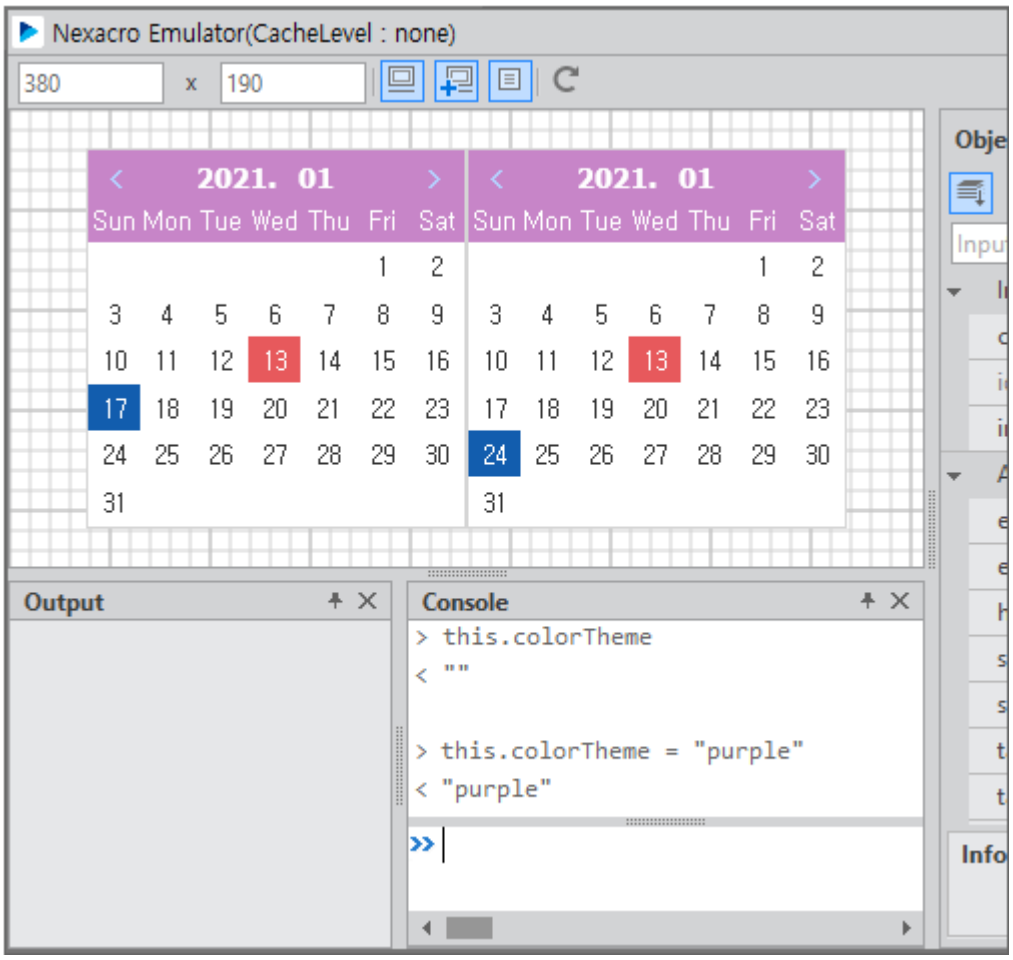
Change the cssclass property value to be used in the Calendar component according to the selected value.

```
nexacro.TwoMonthCalendar.prototype._p_colorTheme = "";
nexacro.TwoMonthCalendar.prototype.set_colorTheme = function (v)
{
    var _cssclass = "";
    if(v=="purple")
    {
        _cssclass = "simple_module, simple_module_purple";
    }
    else
    {
        _cssclass = "simple_module";
    }

    if(this.form.calFrom && this.form.calTo)
    {
```

```
        this.form.calFrom.cssclass = _cssclass;
        this.form.calTo.cssclass = _cssclass;
    }
    if(this._p_colorTheme != v)
        this._p_colorTheme = v;
};
```

8 Run the emulator and check if the style is applied appropriately when the colorTheme property value is changed.



4.1.3 Processing Property Value when Running Components

Add an interface function as shown below so that the value set as the property value is reflected for the operation.

```
nexacro.TwoMonthCalendar.prototype.on_create_contents = function ()
{
    nexacro._CompositeComponent.prototype.on_create_contents.call(this);
    this.fromValue = this.fromValue;
    this.toValue = this.toValue;
    this.colorTheme = this.colorTheme;
};
```

4.2 Adding Methods

4.2.1 Adding Parameterless Method (getDayCount)

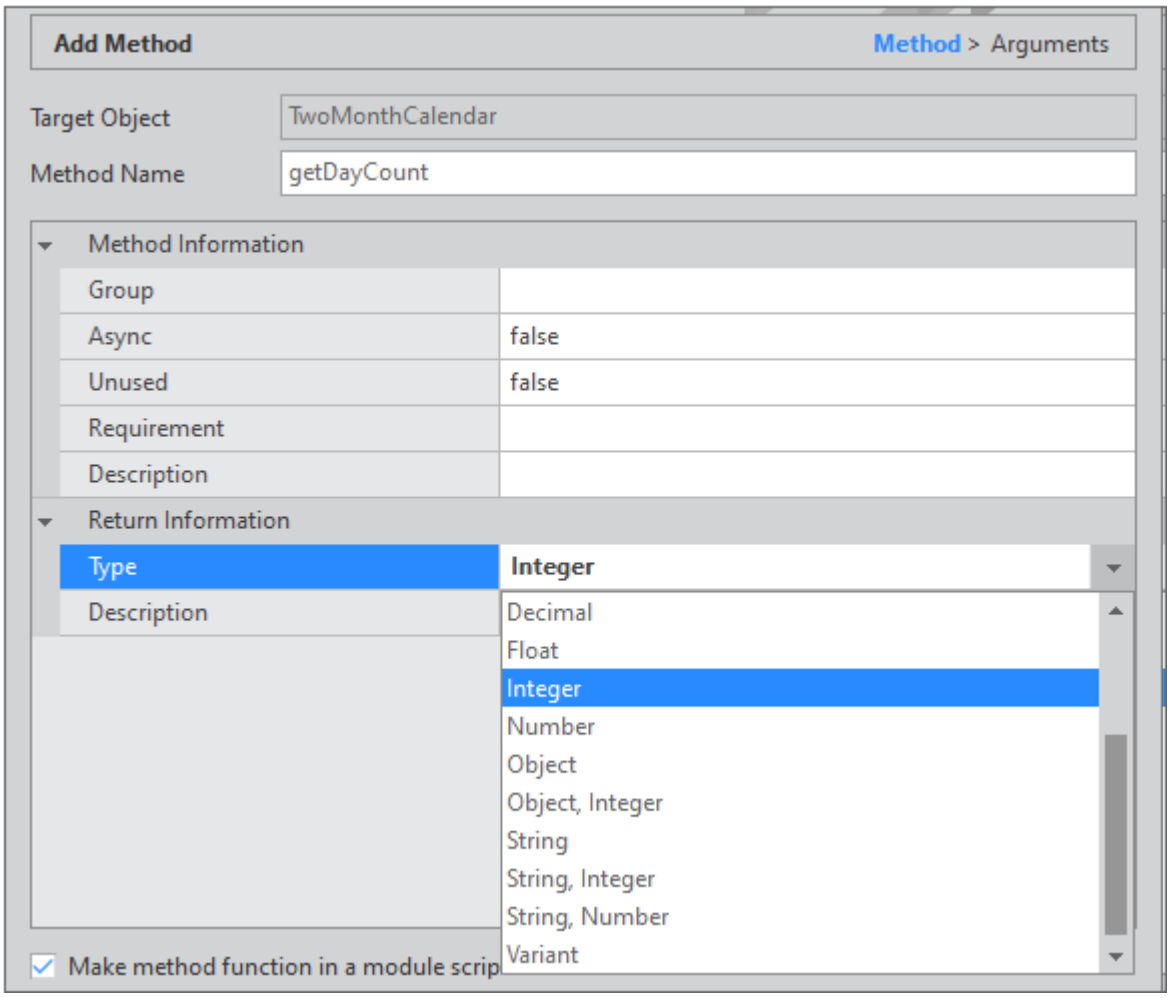
Add a method that calculates and returns the difference between the selected value in the Calendar component. The name of the method is getDayCount.

1. Select [Edit > Add > Method] item from the menu and run Add Method Wizard.

   You can also select an object in Project Explorer and select [Add > Method] item from the context menu.

2. Check if the object displayed in the Target Object item is the one selected and enter the method name to be added in the Method Name item.

   You can set method related information in the Method Information item. In the example, we will generate a method that calculates and returns the difference between the selected dates of two Calendar components. Set the Type value to "Integer" in the Return Information item.



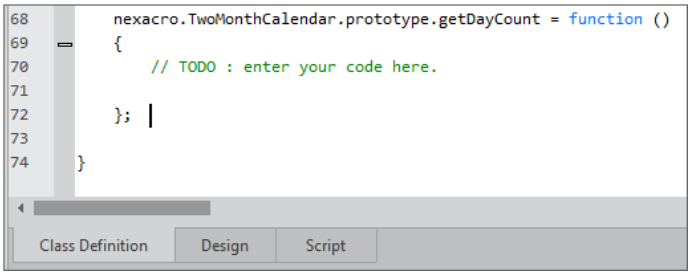
"Make setter function in a module scripts" item sets whether to automatically generate the Function skeleton code of the method. If the metainfo, unused item, is set to true, the corresponding checkbox is disabled.



If the checkbox is disabled or unchecked, the corresponding code will not be added to the xcdl file in the next step.

3 Click the [Finish] button to add the specified method and the Class Definition tab is activated.

If you click the [Next] button, you can specify the parameters used in the method. As in the example, we do not use any parameters, the corresponding step has been skipped.



4 Modify the script as follows.

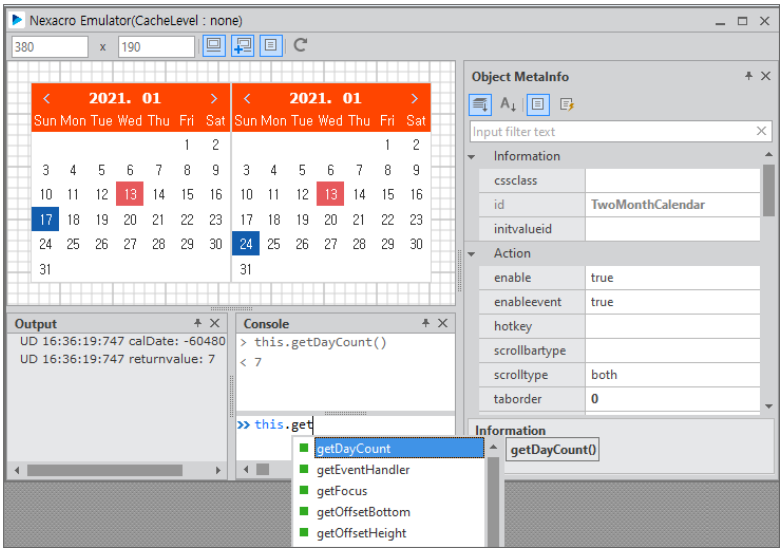
Compare the values of 2 Calendar components and return the difference value.

```
nexacro.TwoMonthCalendar.prototype.getDayCount = function ()
{
    if (this.form.calFrom.value && this.form.calTo.value) {
        const fromDate = new Date(
            this.form.calFrom.getYear(),
            this.form.calFrom.getMonth() - 1,
            this.form.calFrom.getDay()
        );
        const toDate = new Date(
            this.form.calTo.getYear(),
            this.form.calTo.getMonth() - 1,
            this.form.calTo.getDay()
        );

        const dayDifference = Math.abs(fromDate - toDate);
        return dayDifference / (1000 * 60 * 60 * 24);
    }
    return -1;
};
```

5 Run the emulator, select the dates from the first and second Calendar components, call the added method in the console window, and check the returned value.

When entering the method name in the console window, you can also see that the IntelliSense feature is running.



4.2.2 Adding Parameterized Method (addDay)

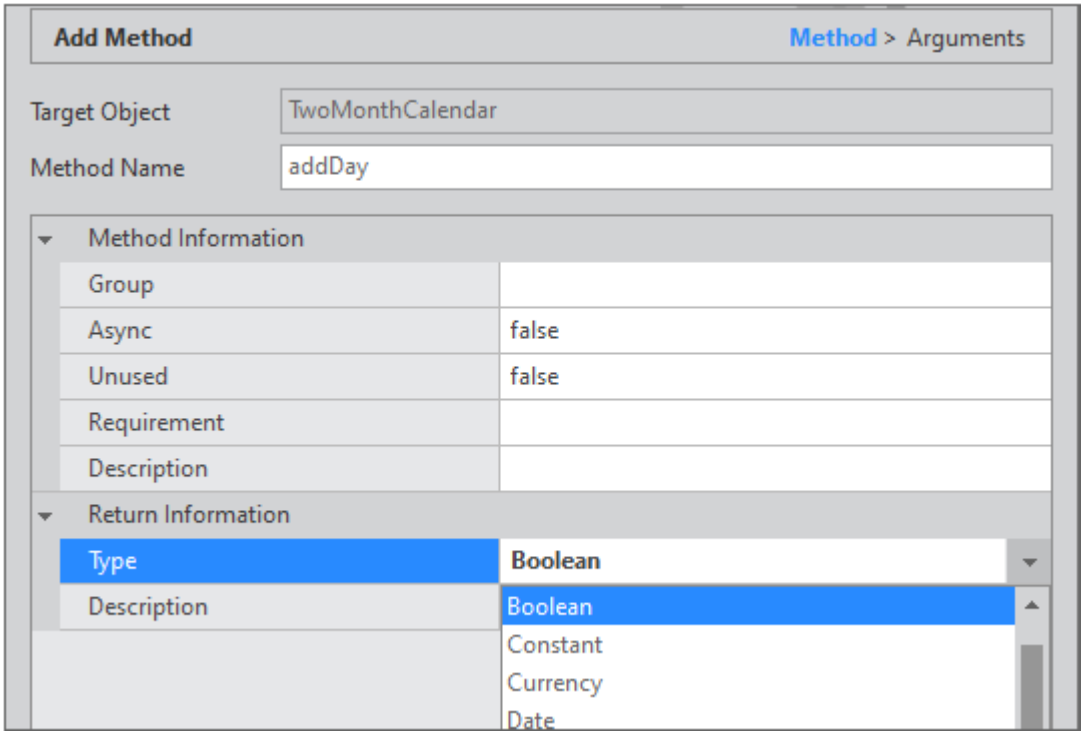
Add a method that adds the date passed as a parameter to the value selected in the first Calendar component and sets it as the value of the second Calendar component. The name of the method is addDay.

1. Select [Edit > Add > Method] item from the menu and run Add Method Wizard.

   You can also select an object in Project Explorer and select [Add > Method] item from the context menu.

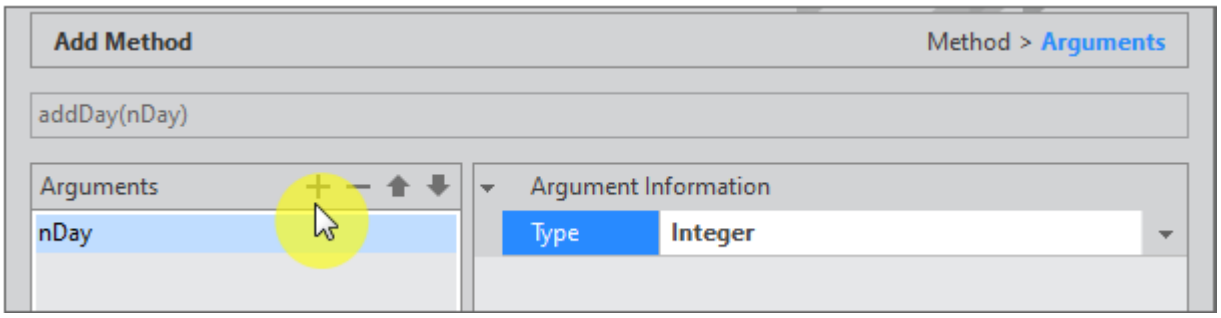
2. Check if the object displayed in the Target Object item is the one selected and enter the method name to be added in the Method Name item.

   You can set method related information in the Method Information item. In the example, the number passed as the parameter is added to the first Calendar date to be set in the second Calendar component and the result value is returned. Set the Type value to "Boolean" in the Return Information item.

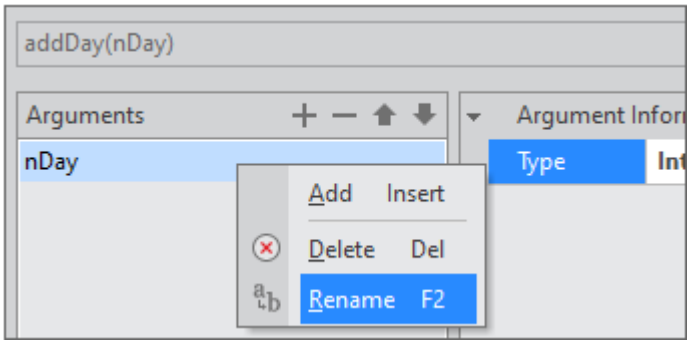


3. Click the [Next] button and the screen where you can specify the parameter is displayed.

   Click the [+] button in the Arguments item to add the parameter and set the Type property to "Integer".

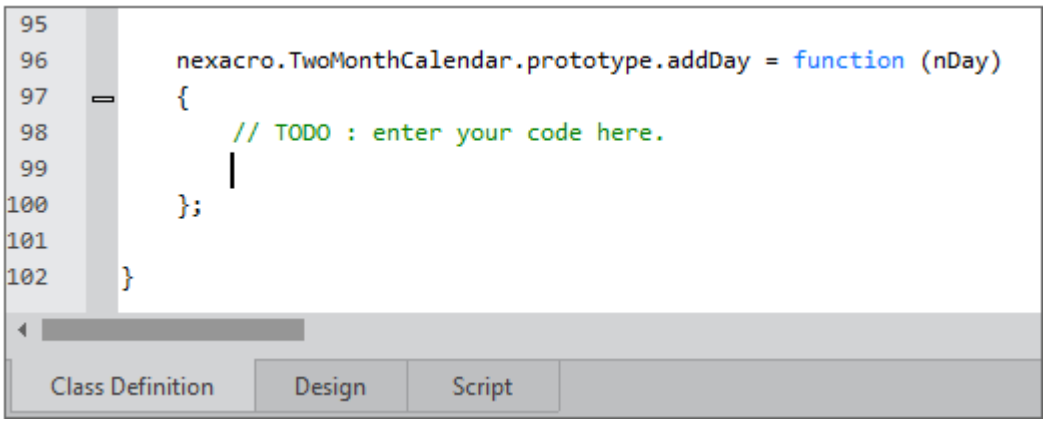


When adding parameters, the name is given in the format of arg00. You can change the name by selecting the item and selecting [Rename] item from the context menu. In the example, we changed the name to nDay.



If there are multiple parameters, the order can be changed by using the up and down arrows.

4 Click the [Finish] button to add the specified method and the Class Definition tab is activated.



5 Modify the script as follows.

After adding the value passed as the parameter to the value of the first Calendar component, specify it as the value of the second Calendar component.

```
nexacro.TwoMonthCalendar.prototype.addDay = function (nDay)
{
    if (!this.form.calFrom.value) {
        trace("calFrom value is null");
        return false;
    }

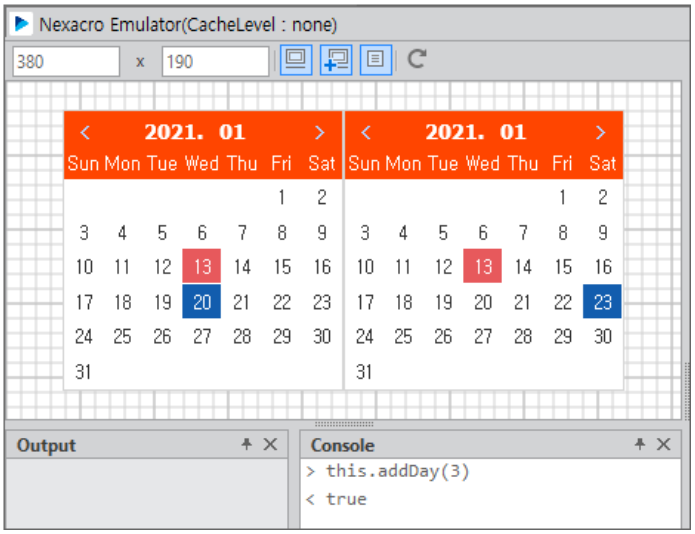
    const fromDate = new Date(
        this.form.calFrom.getYear(),
        this.form.calFrom.getMonth() - 1,
        this.form.calFrom.getDay()
    );

    const toDate = new Date(fromDate);
    toDate.setDate(fromDate.getDate() + nDay);

    const formattedDate = [
        toDate.getFullYear().toString().padStart(4, "0"),
        (toDate.getMonth() + 1).toString().padStart(2, "0"),
        toDate.getDate().toString().padStart(2, "0"),
    ].join("");

    this.form.calTo.value = formattedDate;
    return true;
};
```

6 Run the emulator, select the date in the first Calendar component, call the added method in the console window, and check that the value is set in the second Calendar component.



4.3 Adding Events

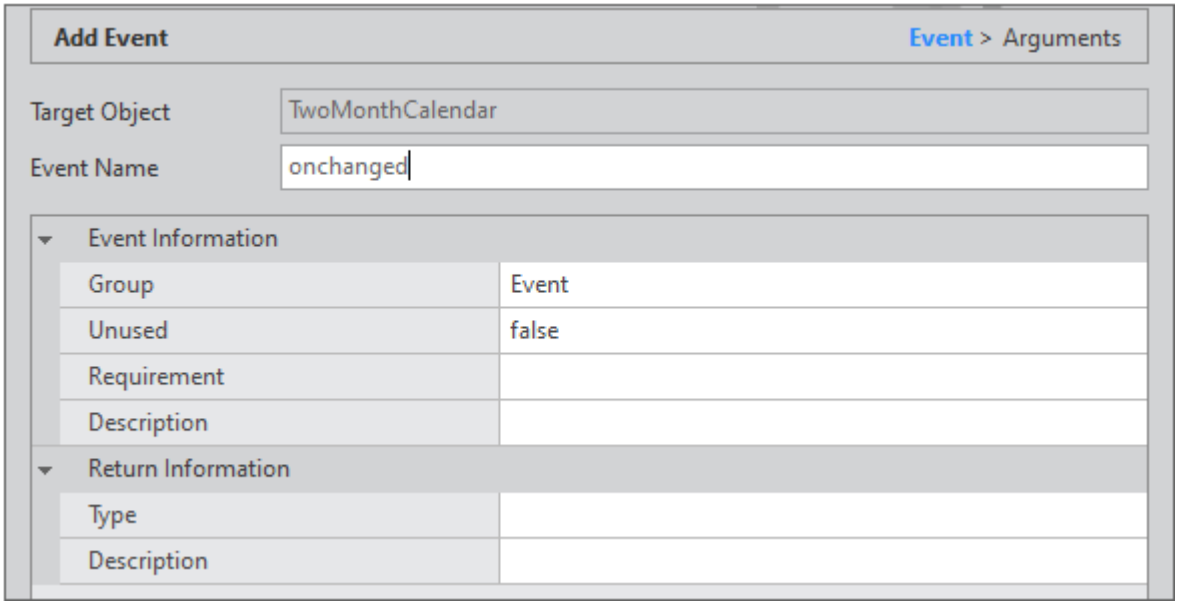
In the Calendar component, add an event to check when the value has changed (onchanged). The name of the event is onchanged.

1. Select [Edit > Add > Event] item from the menu and run Add Event Wizard.

   You can also select an object in Project Explorer and select [Add > Event] item from the context menu.

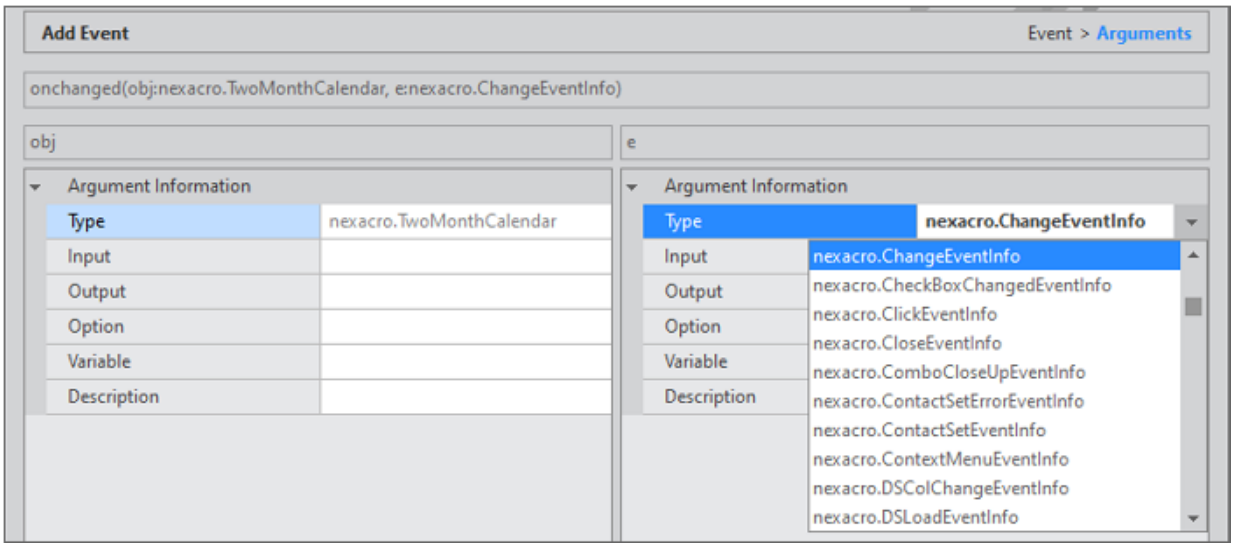
2. Check if the object displayed in the Target Object item is the one selected and enter the event name to be added in the Event Name item.

   If there is a value to return according to the event, then set the Type of Return Information. In the example, as there is no value to return, we did not specify the corresponding item.



3. Click the [Next] button and specify the object where the event occurs and the EventInfo object. If the [Finish] button is clicked without specifying the corresponding item, nexacro.EventInfo object is applied as default.

   In the example, the item is changed as nexacro.ChangeEventInfo.



4 If you click the [Finish] button, a basic script is added as shown below and the Class Definition tab is activated.

First, call a method called this.addEvent in the function that initializes the composite component. Inside the component, there is a variable that manages the event list. When the addEvent method is run, add the event to the corresponding list.

```
7   //===========================================
8   if (!nexacro.TwoMonthCalendar)
9   {
10      //===========================================
11      // nexacro.TwoMonthCalendar
12      //===========================================
13      nexacro.TwoMonthCalendar = function (id, left
14      {
15          nexacro._CompositeComponent.call(this, id
16          this.addEvent("onchanged");
17      };
18
```

Then, add the following event function at the bottom of the code.

```
124     nexacro.TwoMonthCalendar.prototype.on_fire_onchanged = function (
125     {
126         if (this.onchanged && this.onchanged._has_handlers)
127         {
128             var evt = new nexacro.ChangeEventInfo(this, "onchanged");
129             return this.onchanged.fireEvent(this, evt);
130         }
131         return false;
132
133     };
134
```

5 Modify the script as follows.

```
nexacro.TwoMonthCalendar.prototype.on_fire_onchanged = function (obj,
pretext, prevalue, posttext, postvalue)
{
    if (this.onchanged && this.onchanged._has_handlers)
    {
        var evt = new nexacro.ChangeEventInfo(obj, "onchanged", pretext,
prevalue, posttext, postvalue);
        return this.onchanged.fireEvent(this, evt);
    }
    return false;
};
```

6 Add the onchanged event function of the Calendar component and add the following code.



The onchanged event function was added through the example of "Adding Properties". If it has already been added, add only the changed code.

```
this.calFrom_onchanged = function
(obj:nexacro.Calendar,e:nexacro.ChangeEventInfo)
{
     this.parent.fromValue = e.postvalue;
     this.parent.on_fire_onchanged(obj, e.pretext, e.prevalue, e.posttext,
e.postvalue);
};

this.calTo_onchanged = function
(obj:nexacro.Calendar,e:nexacro.ChangeEventInfo)
{
    this.parent.toValue = e.postvalue;
    this.parent.on_fire_onchanged(obj, e.pretext, e.prevalue, e.posttext,
e.postvalue);
};
```

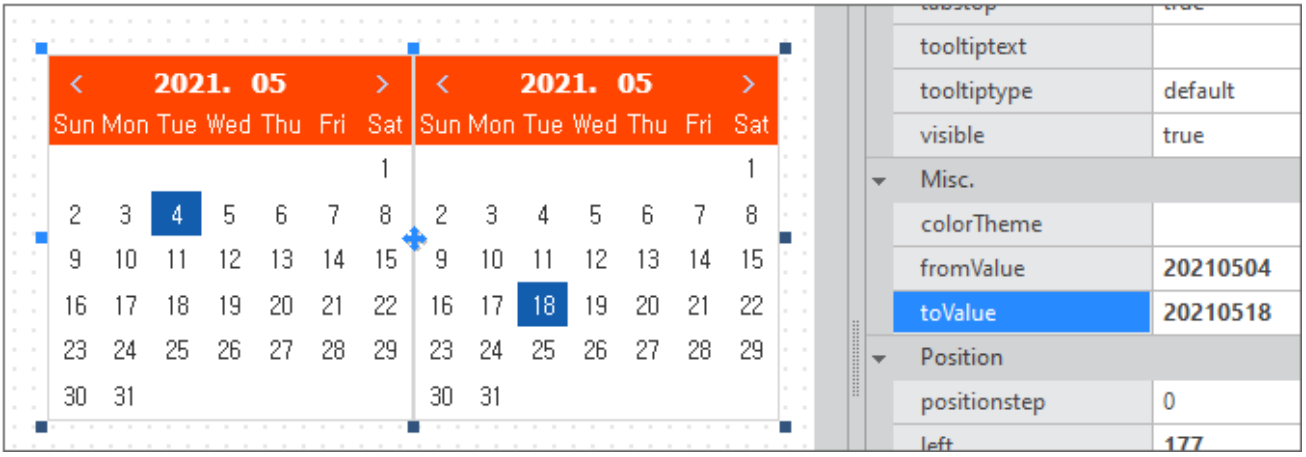
The added event cannot be tested in the emulator, but it can be checked in the app after module deployment.

4.4 Checking Added Properties, Methods & Events after Module Installation

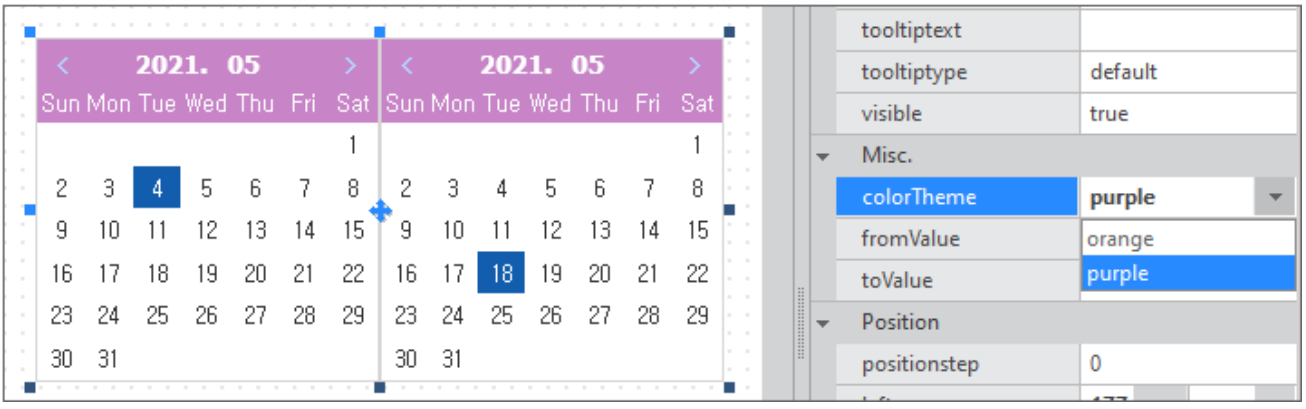
Let's take a look at how you can use added properties, methods, and events after installing the module in Nexacro Studio.

4.4.1 Displaying Added Properties & Methods in Property Window

The added properties and events are displayed in the property window. You can see the value change on the Design tab when you specify the value.

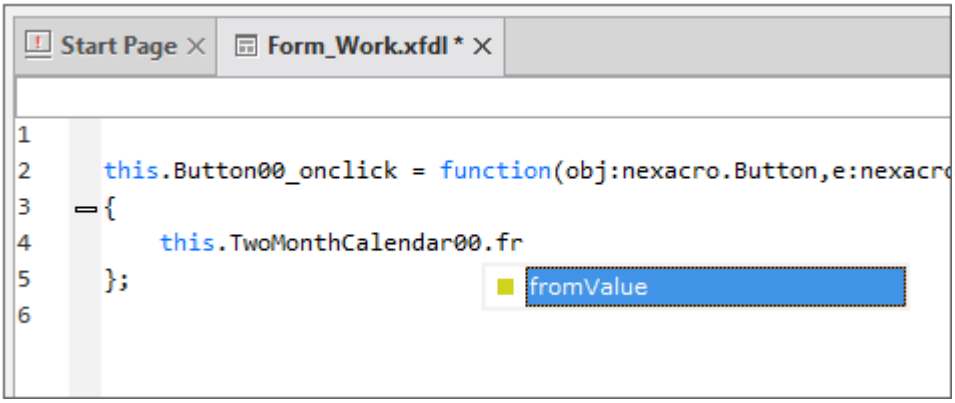


When you select the colorTheme property item, you can check whether the property value specified in the drop-down list is visible. Selecting the value applies to the changed style in the Nexacro Studio design window.



4.4.2 Displaying Added Properties, Methods, & Events when IntelliSense Is Activated

In the script tab, you can also see the added properties, methods, and events when IntelliSense is activated.



4.4.3 Checking Method Operation

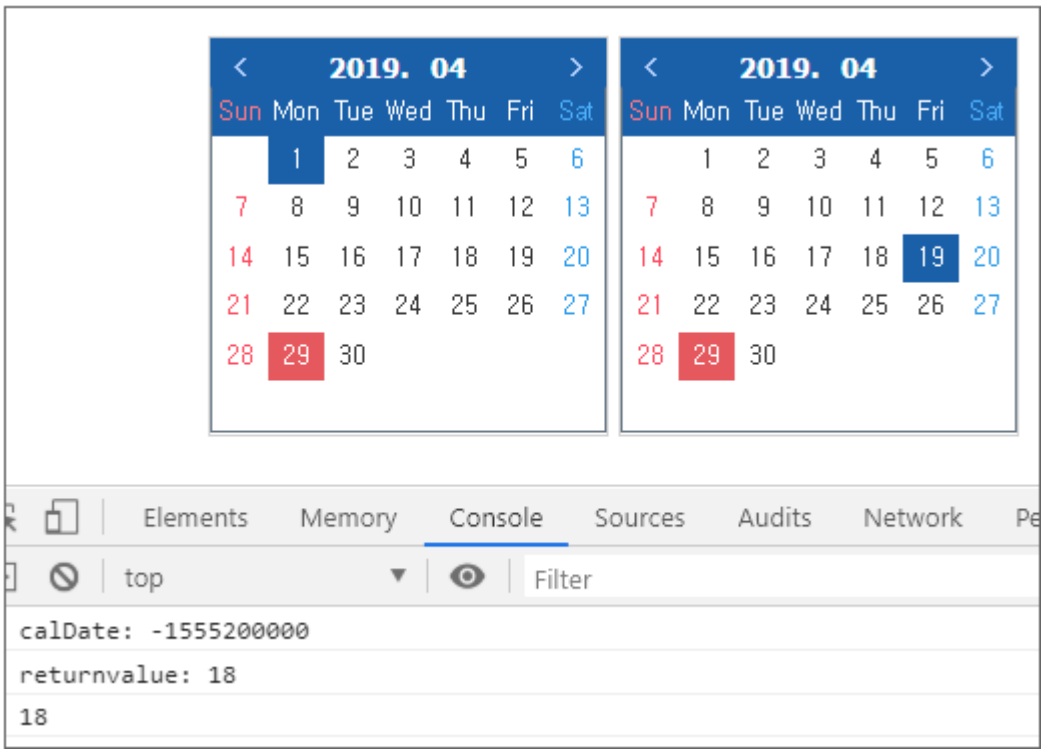
You can also check that the value is returned normally when calling the method.

1. Generate the Form screen in Nexacro Studio.
2. Place the composite component on the screen.
3. Place the Button component on the screen and write the onclick event function as follows.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    trace(this.TwoMonthCalendar00.getDayCount());
};
```

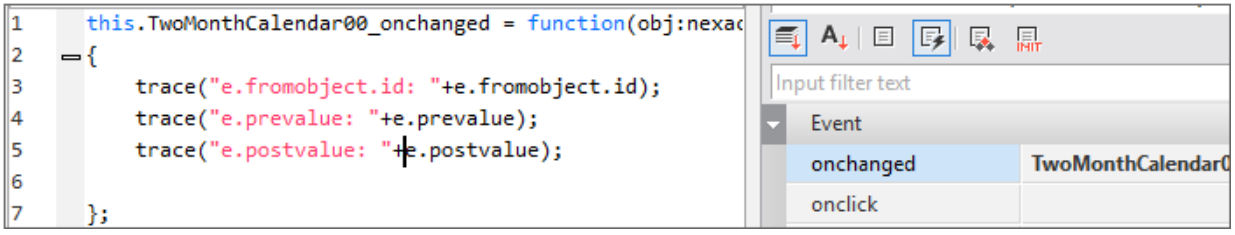
4. Run the app with QuickView(Ctrl + F6), select dates from the two calendars, and click Button.

   The method operation result can be checked on the browser developer tool console screen.



4.4.4 Checking Event Operation

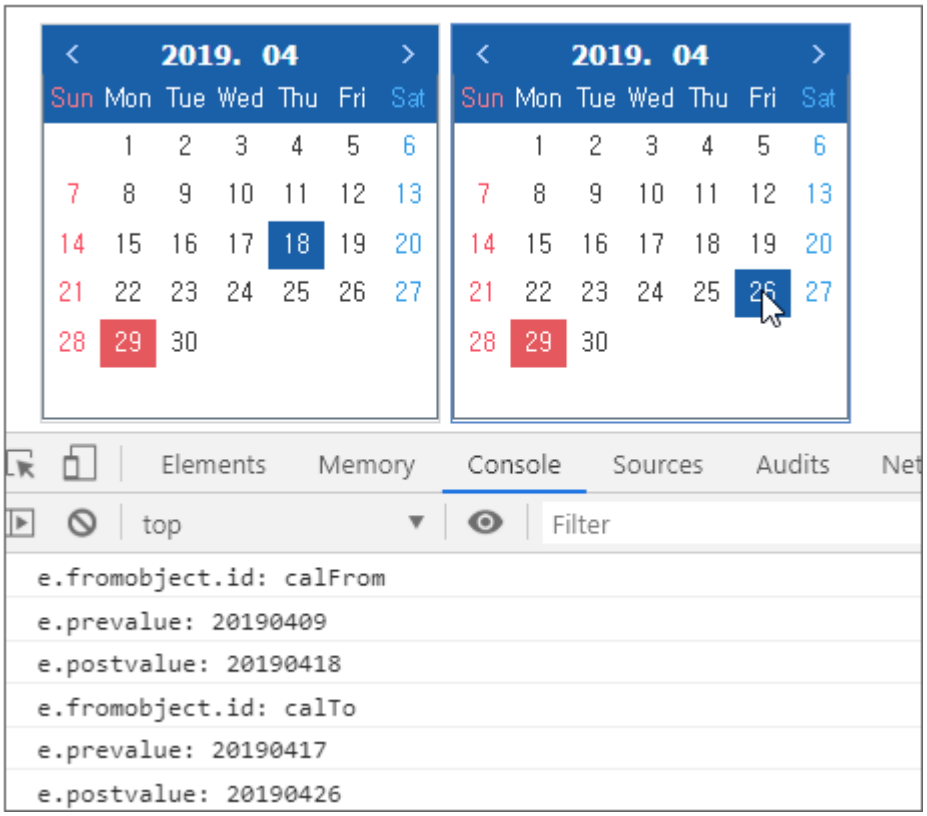
1. Generate the Form screen in Nexacro Studio.
2. Place the composite component on the screen.
3. Write the onchanged event function of the composite component as follows.




```
this.TwoMonthCalendar00_onchanged = function(obj:nexacro.TwoMonthCalendar,
e:nexacro.ChangeEventInfo)
{
    trace("e.fromobject.id: "+e.fromobject.id);
    trace("e.prevalue: "+e.prevalue);
    trace("e.postvalue: "+e.postvalue);
};
```

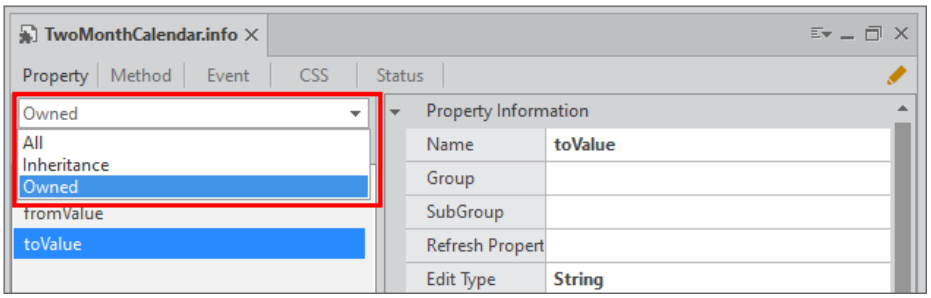
4 Run the app with QuickView(Ctrl + F6) and check if the event is generated when selecting dates from the two calendars.

The method operation result can be checked on the browser developer tool console screen.



4.5 Metainfo Editing (Properties, Methods, & Events)

Detailed items can be set in the wizard that adds properties, methods, and events. This is called the metainfo. You can edit the detailed items by selecting the Property, Method, or Event items of the object under the MetaInfo file in Project Explorer and double-clicking it.

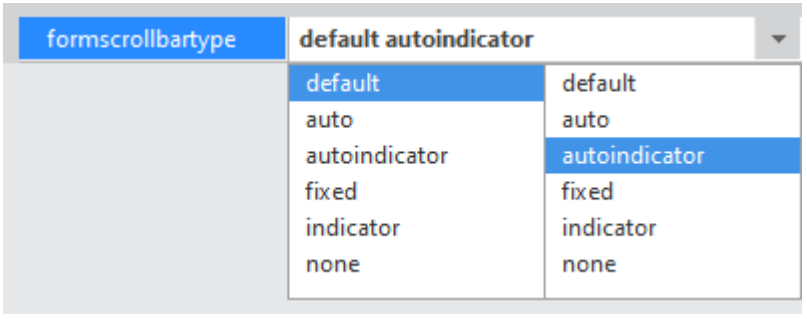


You can select and display the displayed list according to the option.

Item	Description
All	Displays all items.
Inheritance	Displays only the inherited items.
Owned	Displays only the newly-added items.

4.5.1 Properties

Item	Description
Group	It specifies the category of the property. If not specified, it is processed as Misc. Please refer to the note below the table for detailed items that can be specified.
Subgroup	It is used when editing innerdataset properties in Nexacro Studio. When the edit window is opened by clicking the button to the right of the Combo component innerdataset property item in the property window, the property specified in the subgroup item value in "bind(innerdataset)" format is exposed.
Edit Type	It specifies the type of property window editing. For example, in the case of the left property, it is specified as positionBase, and you can specify the number, unit, and positionBase property in the property window. Please refer to the note below the table for detailed items that can be specified.
Default Value	It specifies the default value of the property. When adding properties, it specifies the default value when generating a script. If the defaultvalue item value is modified after the script generation, it has no effect.
Unused	It specifies whether to use the property in the child when inheriting.
ReadOnly	It specifies whether the property is read-only.
Configurable	This is one of the descriptor items of the Object.defineProperty method. The default value when adding a property is true. This is not reflected when generating a metadata file.
InitOnly	It specifies whether to use only the initial setup. For example, the initvalueid property value is used only to set the initial value before being displayed on the screen.
Hidden	It specifies whether to display the property window. It is not displayed in the property window but can be accessed via script. For example, the form property of the composite component can be accessed from the script but it is not displayed in the property window.
Control	It specifies whether the property itself is an object (or control). If the corresponding item value is true, the readonly item is set to true and the hidden item is set to false.

Item	Description
	For example, the form property of the composite component is an object, so it is set to true.
Expression	It specifies whether to use expr. When true, the [set expression] button is displayed in the property window and the expr value can be edited.
Enumerable	This is one of the descriptor items of the Object.defineProperty method. The default value when adding a property is true. This is not reflected when generating a metadata file.
Bind	It specifies whether to use the Column binding feature of the bound Dataset object. If it is bound to the Dataset object, you can select the Column item to be connected in the property window from the drop-down list.
Mandatory	When placing components in Nexacro Studio, it specifies whether to generate the corresponding property in the XML code.
Refresh Properties	It is specified when there are properties that are affected by a change in property value. The affected property information is separately managed in the RefreshInfo.info file. The actual property value change must be implemented in the set_[value] function.
Enuminfo	It specifies the EnumInfo information that can be selected when the edittype item is specified as "Enum". In the property window, it is displayed as the drop-down list to select items.
Enuminfo2	If the edittype item is specified as "Enum2", then specify the EnumInfo information that can be selected. In the property window 2 items can be selected from the drop-down list as shown below. 
UnitInfo	It specifies the UnitInfo information that can be selected when the unit you specify in the property is fixed. You can display the unit on the right side of the property window or specify the minimum and maximum values. It can only be applied when the edittype setting value is "Number".
Delimiter	If the edittype item is set to "Number2", more than one value can be specified as the property value. It specifies each property value separator at this time.
Requirement	It specifies the supported operating environment with comma separators.

Item	Description
	The supported operating environment is displayed in the format of tooltip text when the mouse cursor hovers over the property name in the property window or when the IntelliSense feature is run in the script tab.
Css Property Name	In the case of the style property, it specifies the CSS property name. The CSS property names have a different naming convention from general property names, so for convenience of development, general property names and CSS property names are managed separately. For example, the CSS property name of the border property is -nexa-border.
Normal Property Name	It is the name specified in the XFDL file in Nexacro Studio. If omitted, the same value as the name item is used.
Description	It is a brief description of the property. Text is displayed in the description area at the bottom when selecting the property item in the property window or it is displayed when the IntelliSense feature is activated in the script tab.



Values that can be specified as group items are as follows.
Accessibility, Action, Appearance, Binding, Collection, Communication, Control, EventInfo, Hidden, Information, Misc, Normal, Position, SandBox, Style



Values that can be specified as edittype items are as follows.
Accessibility, Boolean, ColumnID, Contents, ContentsParam, CssClass, DataObjectID, DataObjectPath, DatasetID, Enum, Enum2, FileName, FormatID, Hotkey, ID, IconID, ImageID, InitvalueFileID, InitvalueID, InnerColumnID, InnerDatasetID, Line, LiteDBConnectionID, LiteDBParameters, ModelServiceID, MultiEnum, MultilineString, Number, Number2, Number2Unit, NumberUnit, Position, PositionBase, ProgID, ScreenID, String, StringEnum, SvgPath, ThemeID, URL, UserFontID, Variant, ViewChildObjList, ViewObjList

4.5.2 Methods

Item	Description
Group	It specifies the category of the method. Currently, it does not affect the module operation.
Unused	It specifies whether to use the method in the child when inheriting.
Async	It specifies whether to process synchronization. Currently, it does not affect the module operation.
Requirement	It specifies the supported operating environment with comma separators. The supported operating environment is displayed in the format of tooltip text

Item	Description
	when the IntelliSense feature is run in the script tab.
Description	It is a brief description of the method. It is displayed when the IntelliSense feature is activated in the script tab.

4.5.3 Events

Item	Description
Group	It specifies the category of the event. The default value is specified as "Event".
Unused	It specifies whether to use the event in the child when inheriting.
Requirement	It specifies the supported operating environment with comma separators. The supported operating environment is displayed in the format of tooltip text when the mouse cursor hovers over the event name in the property window or when the IntelliSense feature is run in the script tab.
Description	It is a brief description of the event. It is displayed when the IntelliSense feature is activated in the script tab.

5.

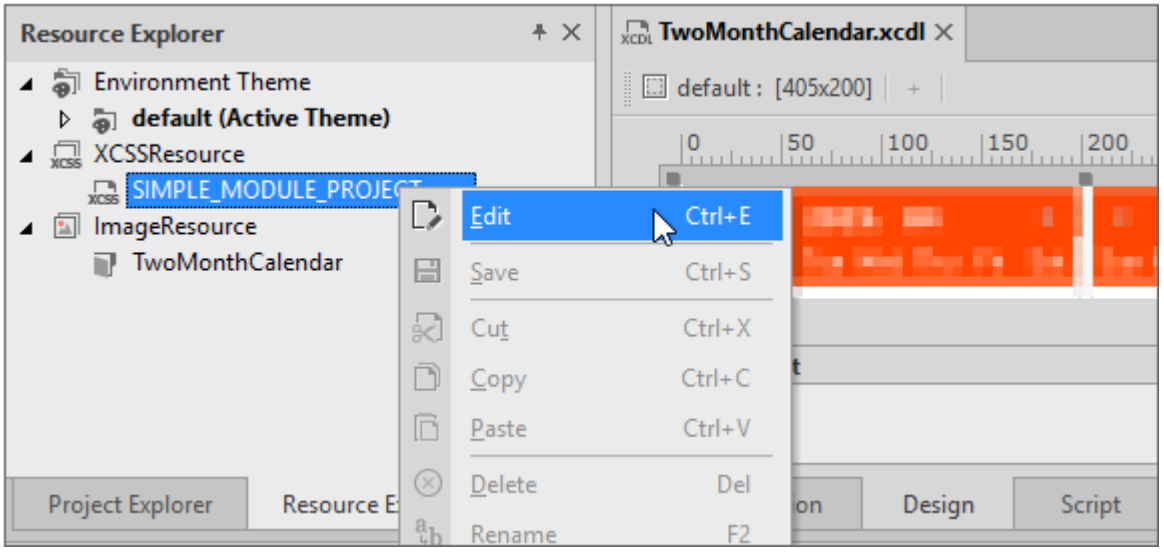
Specifying Individual Style of Components

Individual components placed within the composite component apply the component style of the default theme. If you want a style that applies only to the composite component you are deploying, you can use the XCSS file and the cssclass property.

5.1 Editing XCSS File & Applying to Screen

When generating a module project, the XCSS file is automatically generated with the project name. The generated file can be found in the Resource Explorer tab.

1. Select the xcss file in the Resource Explorer tab and click [Edit] item from the context menu to open the CSS editing window.



2. Modify the code as shown below in the Text tab. Add the Calendar selector and add an item called simple_module as the class selector.

```
.Calendar.simple_module .datepicker .head
{
    background : orangered;
}
.Calendar.simple_module .datepicker .body .weekband
{
    background : orangered;
}
.Calendar.simple_module .datepicker .body
{
    -nexa-border : 1px none transparent;
}

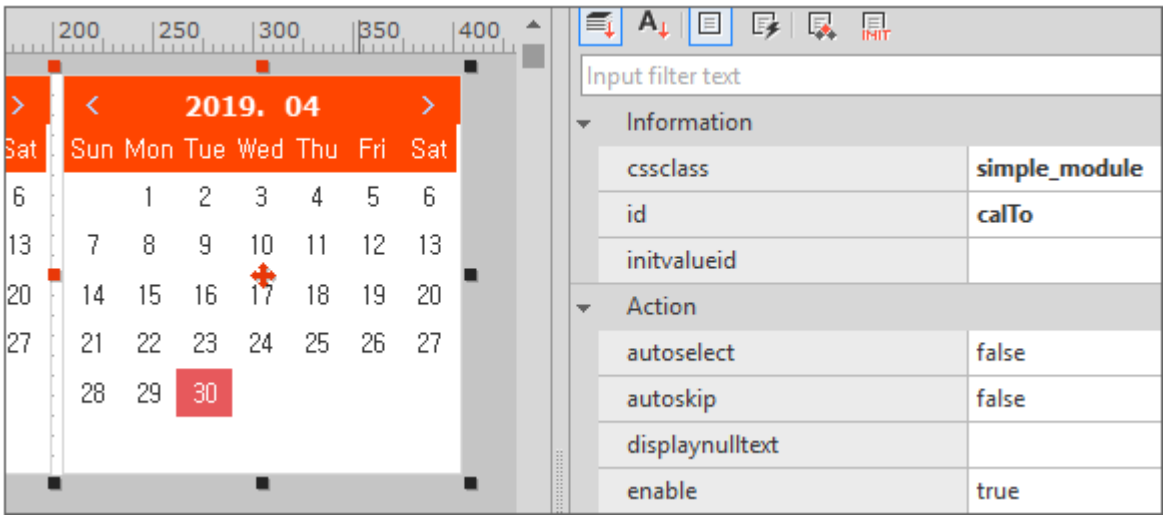
.Calendar.simple_module .datepicker .body .dayitem[userstatus=saturday]
{
    color : #333333;
}
.Calendar.simple_module .datepicker .body .dayitem[userstatus=sunday]
{
    color : #333333;
}
.Calendar.simple_module .datepicker .body .weekitem[userstatus=saturday]
{
    color : #ffffff;
}
.Calendar.simple_module .datepicker .body .weekitem[userstatus=sunday]
{
    color : #ffffff;
}
.Calendar.simple_module_purple .datepicker .head
{
    background : #C785C8;
}
.Calendar.simple_module_purple .datepicker .body .weekband
{
    background : #C785C8;
}
```



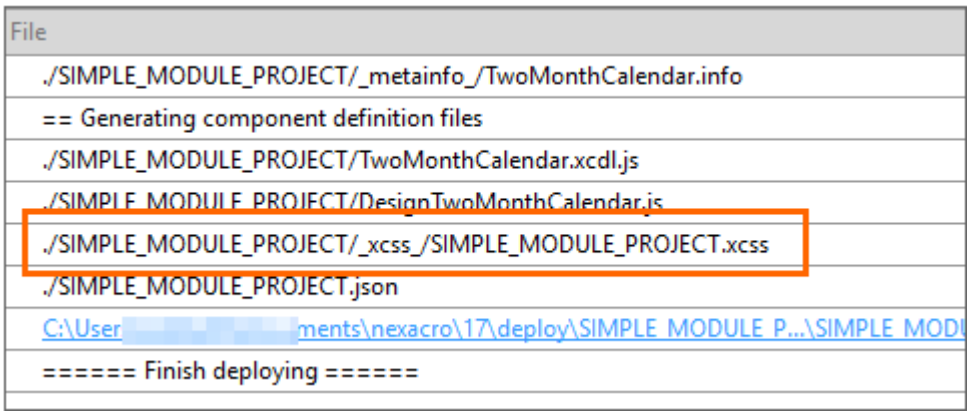
Please refer to the link below for the CSS editing window feature.
Editing XCSS

3 Select the xcdl file in the Project Explorer tab and click the [Edit] item from the context menu to open the composite component screen.

4 Specify the cssclass property value of 2 Calendar components to "simple_module". You can see that the style of the components has been changed.



5 Generate a module installation file. You can see that the xcss file is deployed together without any additional settings.

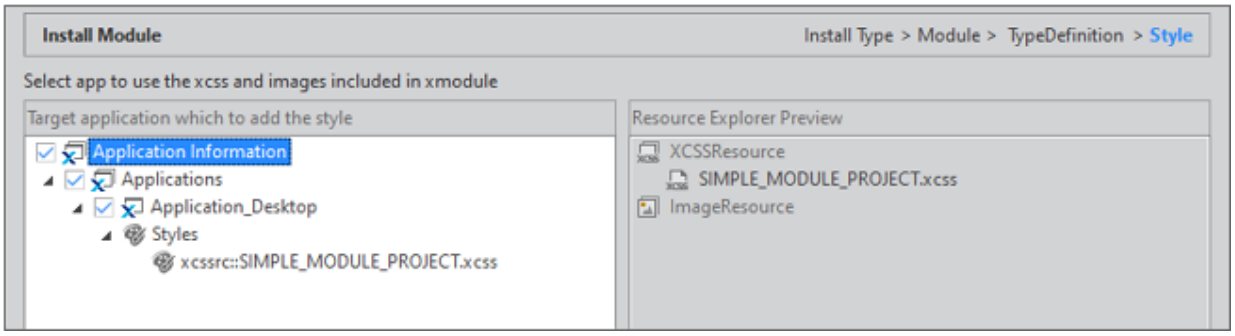


5.2 Installing Module & Checking Applied Style

The steps to install the module are the same until the step of selecting the composite component. In the corresponding step, click the [Next] button and select the App to apply Style.

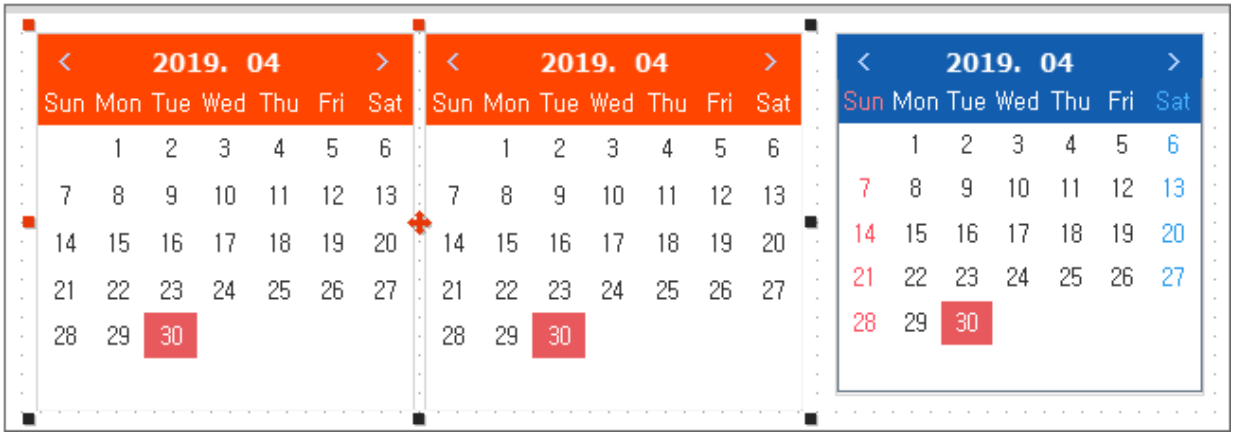
1. Check the name of the composite component to install and click the [+] button and the [Next] button.

2. Specify the App object to apply in the style setting screen. You can see that the xcss file has been added in bold text.



3. Click the [Install] button to install the module.

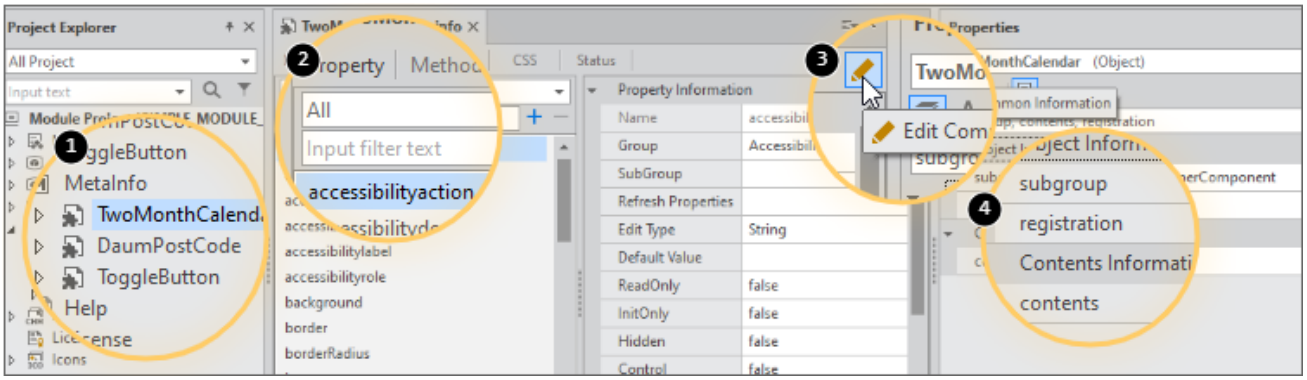
4. Open the Form screen where the composite component is placed or generate a new Form to place the composite component and check whether the specified style is applied.



6.

Editing Metainfo

Let's look at each item of object metainfo that was not set when generating a project or a new composite component. You can select and double-click the object under the MetaInfo item in the Project Explorer tab or select the [Edit] item from the context menu to edit the metainfo setting value.



	Description
1	Select the object under the MetaInfo item. You can edit the relevant information according to the selected object.
2	Edit the properties, methods, events, CSS, and Status metainfo information of the selected object. Please refer to the Metainfo Editing (Properties, Methods, & Events).
3	Open the Edit Common Information window and edit the information of Enum, Unit, and Refresh commonly used in the module project.
4	Edit the metainfo information of the selected object.

6.1 Editing Object Metainfo

6.1.1 Items Automatically Set when Generating Object

The items below are applied when generating the object and cannot be modified.

- inheritance
- classname
- csstypename
- group

6.1.2 Items that Can Be Set

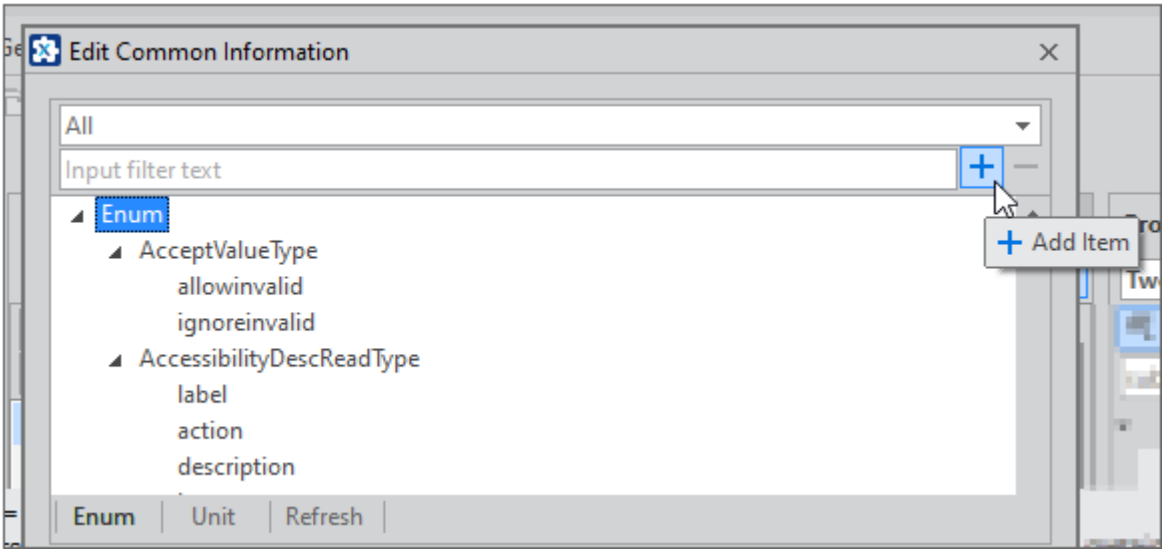
Item	Description
shorttypename	It specifies the TypeName of the object. It is used as an ID and PrefixID in TypeDefinition when installing a module. The default value is set to the same value as the Object ID.
csscontrolname	It specifies the CSS TypeName of the object control. It is exposed as a list of components that can be added as a selector when editing the XCSS file. The default value is set in the format of [Object ID]_Control.
subgroup	It specifies the subtype of the component. In the case of the composite component, it is set to the default value (Component).
csspseudo	Specifies whether to use the Status information of the object. Composite Component and Visible Object are set as true values when created.
container	It specifies whether the child can contain other objects.
tabstop	It specifies whether to move the focus to the object using the TAB key.
cssstyle	It specifies whether to use the object and object control as a selector when editing the XCSS file. When set to false, it is not exposed in the component list.
defaultwidth	It specifies the default width when placing the component on the screen after registering the module. It is also reflected as the default width when running the emulator.
defaultheight	It specifies the default height when placing the component on the screen after registering the module. It is also reflected as the default height when running the emulator.
registration	It specifies whether the object can be registered. If it is set to "deny", the object is not displayed and cannot be registered when installing the module.
edittype	Sets the object editing type. Uses the information inherited from the created component by inheriting the Menu component.
popup	Sets the component displayed as a pop-up when executed.

Item	Description
	For the component created by inheriting the PopupMenu, PopupDiv components or the component displayed in a pop-up form, set to true.
edittypecomponent	Sets the features of the text editing area. Able to set in the case of the component created by inheriting the Edit, TextArea components or the component having the text editing area.
dblclickevent	It specifies the event function to be generated or moved when the component is double-clicked on the design tab.
requirement	Sets the support scope of the created object. It does not affect the actual operation.
finalclass	Sets whether the created object can be inherited when other objects are created. When set to false, it is displayed in the list of objects that can be inherited when creating a new object according to the object type.
icon	It specifies the icon to be displayed in Nexacro Studio after module installation. Please refer to Deployment after Setting Icon to Module Object.
contents	It specifies whether to use the contents editor. Please refer to Setting with Metainfo Contents Property.

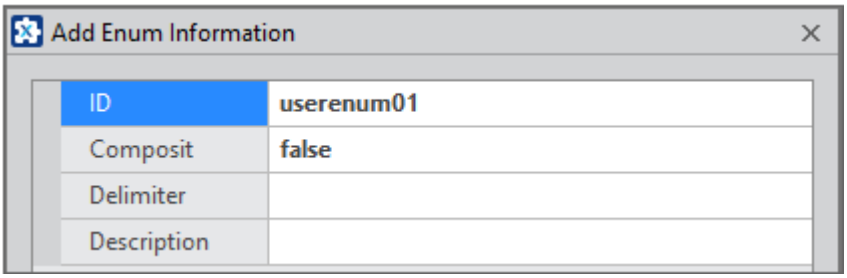
6.2 Common Information Metainfo Editing

6.2.1 Adding Items

Clicking the [+] button in the Edit Common Information window displays a window where you can add items.

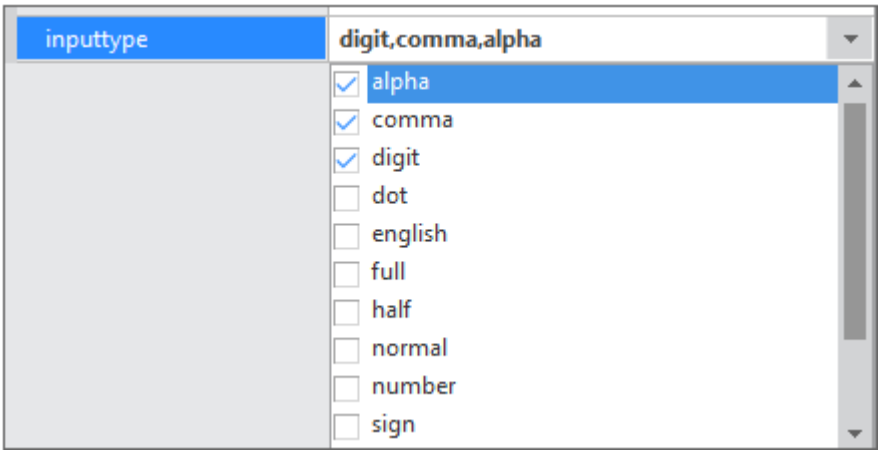


Enum

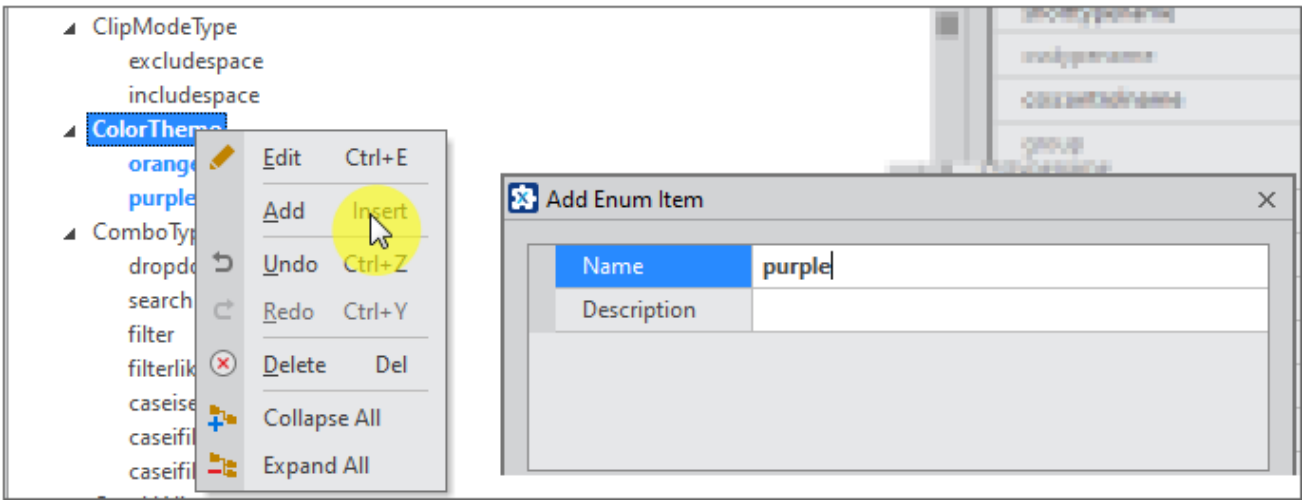


Item	Description
ID	It sets the value to be specified as Enuminfo or Enuminfo2 of the object property.
Composit	It sets whether you can select more than 1 item. For setting the true value, the property value is set according to the value set by Delimiter.
Delimiter	It sets the separator when selecting 2 or more items when the Composit value is true.

When the Composit value is true, a checkbox is displayed so that you can select more than 2 items when selecting the object property in Nexacro Studio. The selected value processes the property value using the separator set by Delimiter. The example below is the screen where Enum information is displayed when selecting the inputtype property of the Edit component.

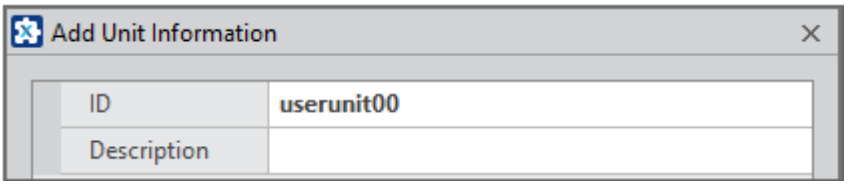


You can add an item by selecting the added item and [Add] item from the context menu.



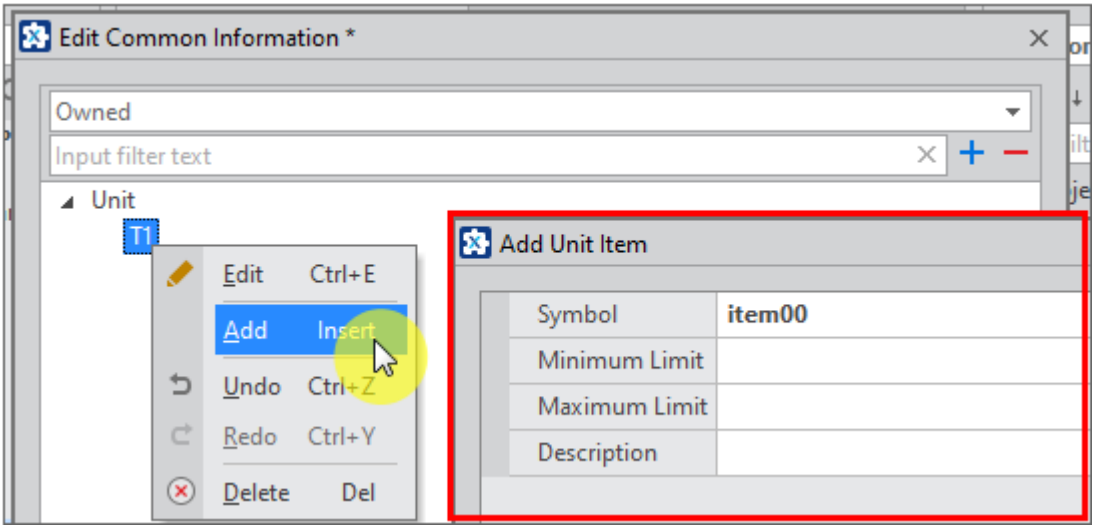
Item	Description
Name	It sets the value to be used as the property value.

Unit



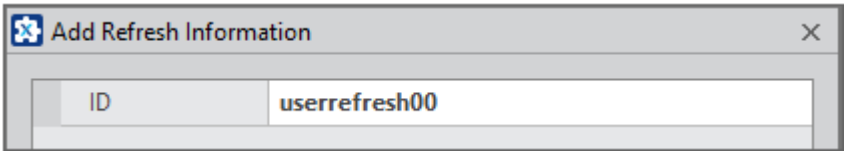
Item	Description
ID	It sets the value to be specified as UnitInfo of the object property.

You can add an item by selecting the added item and [Add] item from the context menu.



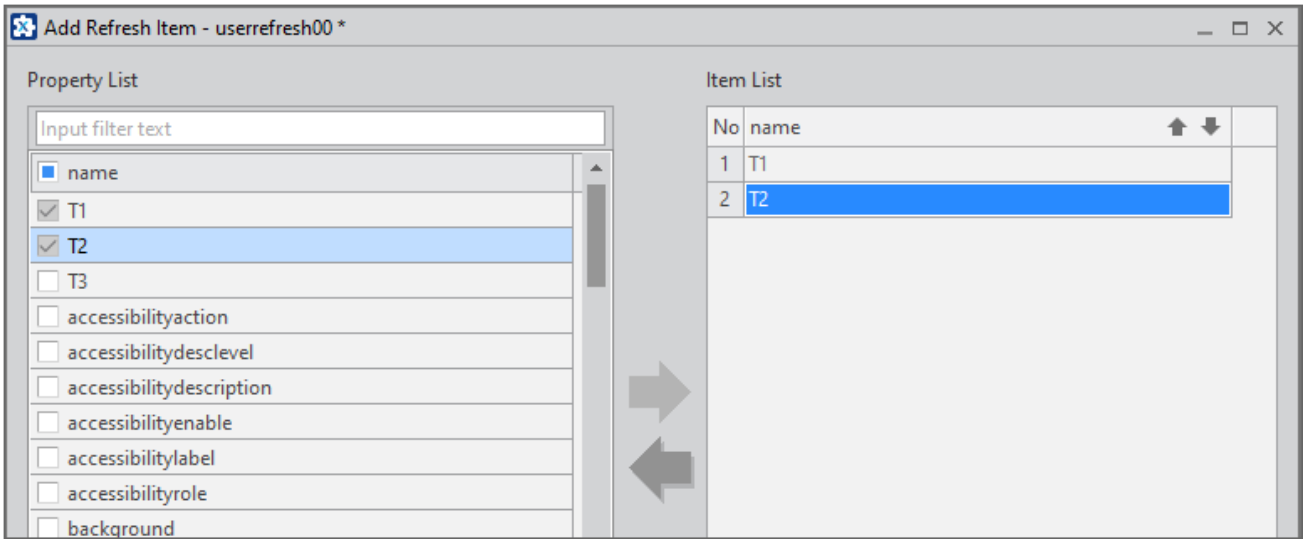
Item	Description
Symbol	It sets the unit value to be displayed in the property window.
Minimum Limit	It sets the minimum value.
Maximum Limit	It sets the maximum value.

Refresh



Item	Description
ID	It sets the value to be specified as RefreshInfo of the object property.

You can add an item by selecting the added item and [Add] item from the context menu. You can apply items among properties that the object has inherited or added and you can modify the order of the applied items.



For example, in the metainfo information of the Combo component, the refreshinfo item value of the index property is "selectitem". The selectitem item is composed as follows.

```
<RefreshInfo id="selectitem">
    <Property name="text" />
    <Property name="value" />
    <Property name="index" />
</RefreshInfo>
```

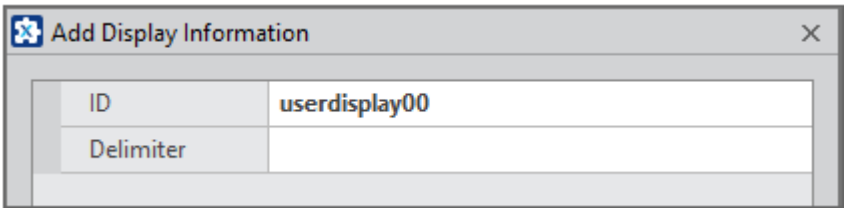
If the index property value is modified in the Nexacro Studio property window, the property value is updated in

the order of text, value, and index, according to the order specified in refreshinfo. Also, in the Generate Code, the set_[value] function is called in the order specified in refreshinfo.

```
obj = new Combo("Combo00",...;
obj.set_taborder("7");
obj.set_text("Combo00");
obj.set_value("3");
obj.set_index("-1");
this.addChild(obj.name, obj);
```

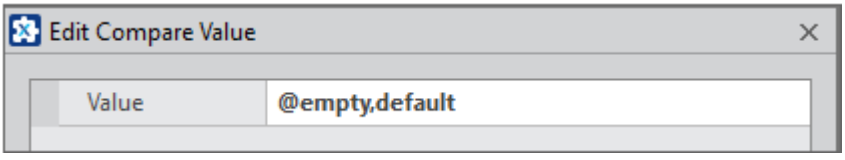
Display

When setting a specific value for a property, it can be set as non-editable.



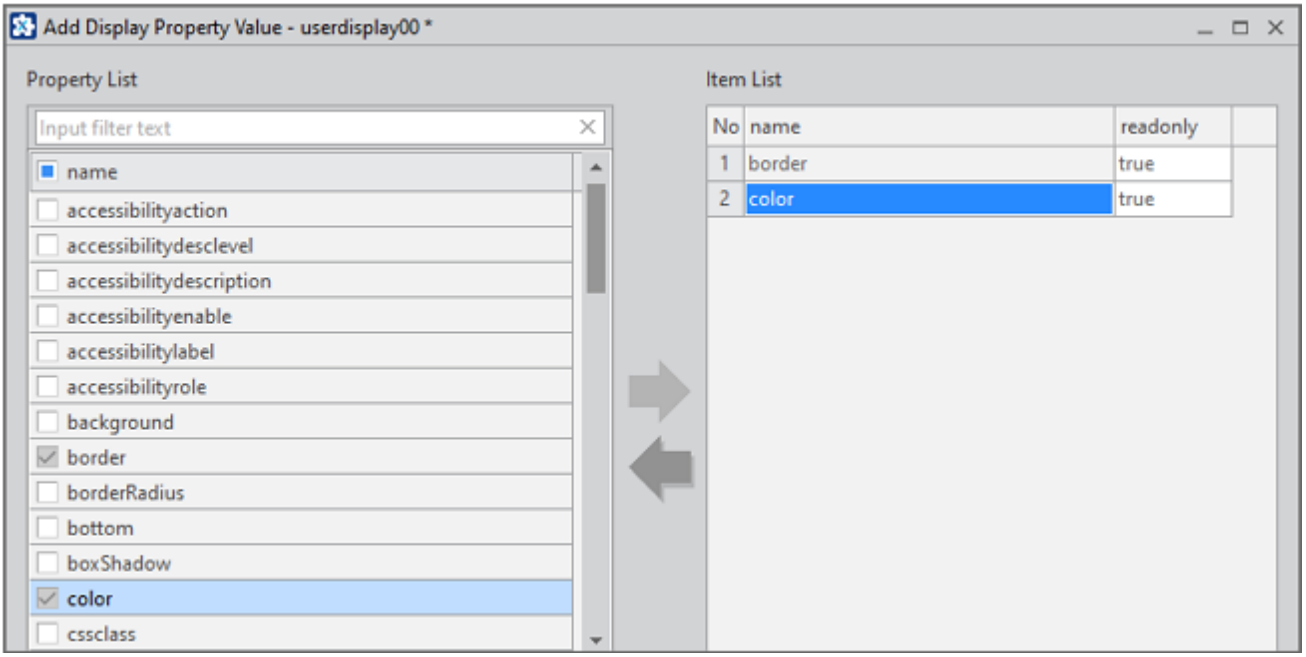
Item	Description
ID	Set the value to be specified as DisplayInfo of object properties.
Delimeter	Set the character to be used as a value separator for compare value.

After selecting the ID of the added item, select the [Add] from the context menu to add a Compare Value.



Item	Description
Value	Enter the attribute value to process the value set as the delimeter as a separator. Use the value "@empty" for a state where no value can be entered. If "@empty,default" is entered, it will be processed if the property value is empty or "default".

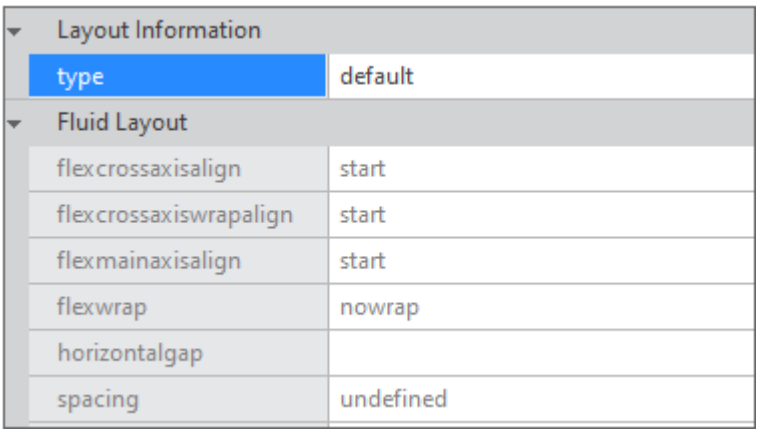
After selecting the value of the added item, select the [Add] from the context menu to add an item to be processed as readonly.



For example, in metainformation of the layout object, the displayinfo item value of the type property is "layouttype". The layouttype item is composed as follows.

```
<DisplayInfo id="layouttype" delimiter=",">
    <Value compare="@empty,default">
      <Property name="spacing" readonly="true" />
      <Property name="horizontalgap" readonly="true" />
      <Property name="verticalgap" readonly="true" />
      <Property name="flexwrap" readonly="true" />
      <Property name="flexmainaxisalign" readonly="true" />
      <Property name="flexcrossaxisalign" readonly="true" />
      <Property name="flexcrossaxiswrapalign" readonly="true" />
      <Property name="tabletemplate" readonly="true" />
      <Property name="tabletemplatearea" readonly="true" />
      <Property name="tabledirection" readonly="true" />
      <Property name="tablecellalign" readonly="true" />
      <Property name="tablecellincompalign" readonly="true" />
    </Value>
    <Value compare="horizontal,vertical">
      <Property name="stepindex" readonly="true" />
...
```

When the type property value is modified in the Nexacro Studio property window, the compare value is checked in the DisplayInfo and the property items that match are disabled.



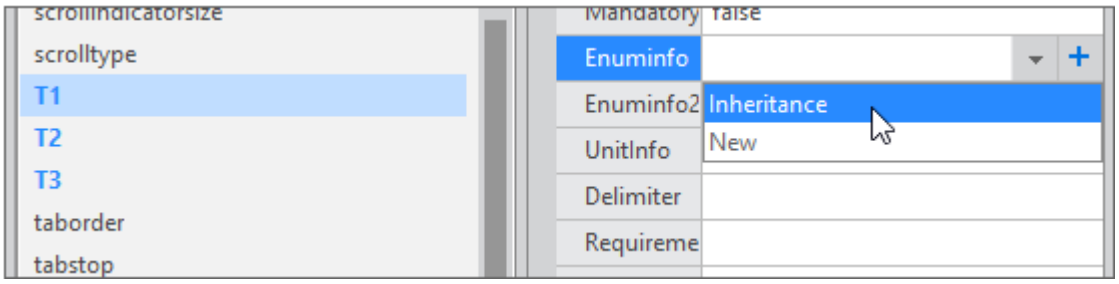
6.2.2 Editing by Copying Existing Item

A new item can be added when selecting the properties of the selected object and selecting the item values of Enuminfo, Enuminfo2, UnitInfo, and Refresh Properties. At this time, you can copy the existing item to modify and apply it.

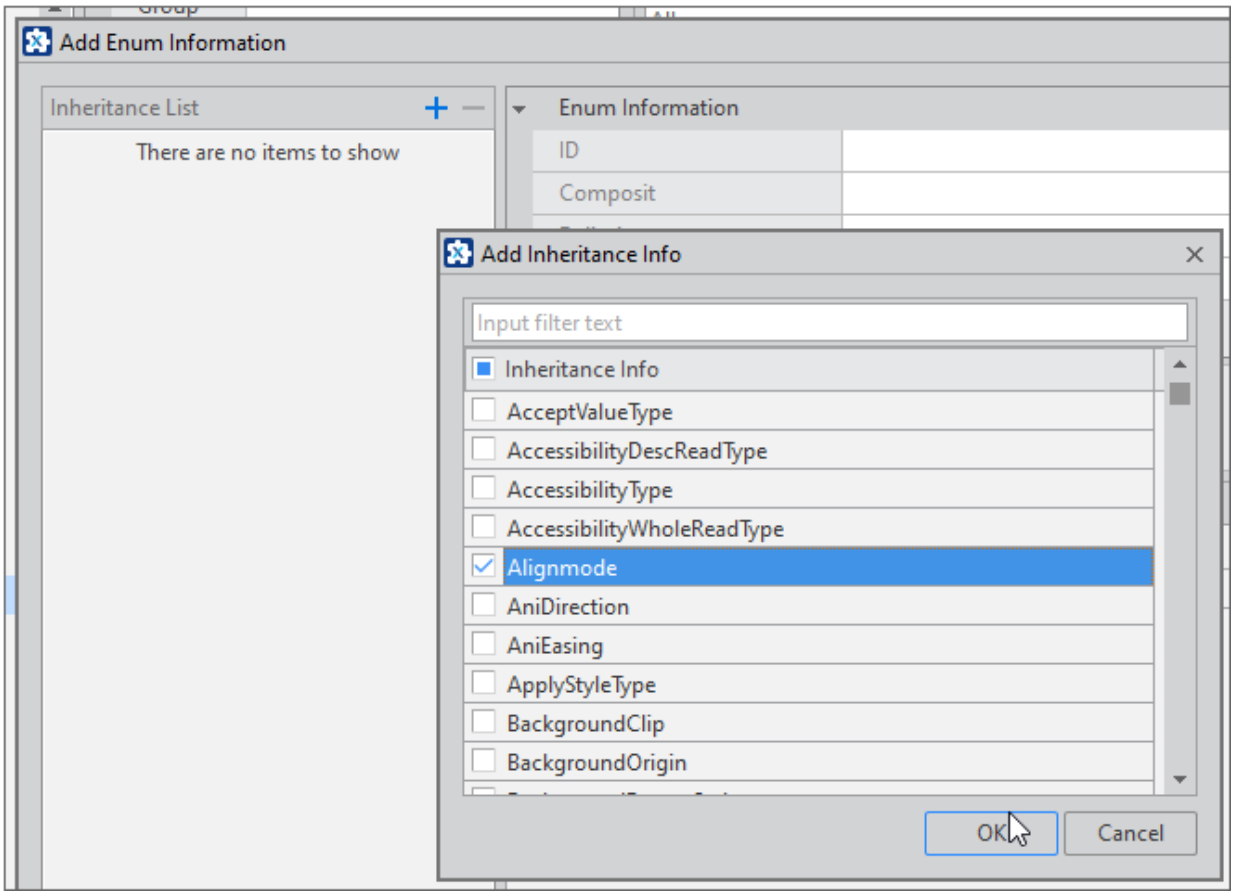
Enum, Unit, and Refresh all operate in the same way and the demonstration below is for Enum.

1. After selecting the property, click the [+] button on the item to be added.

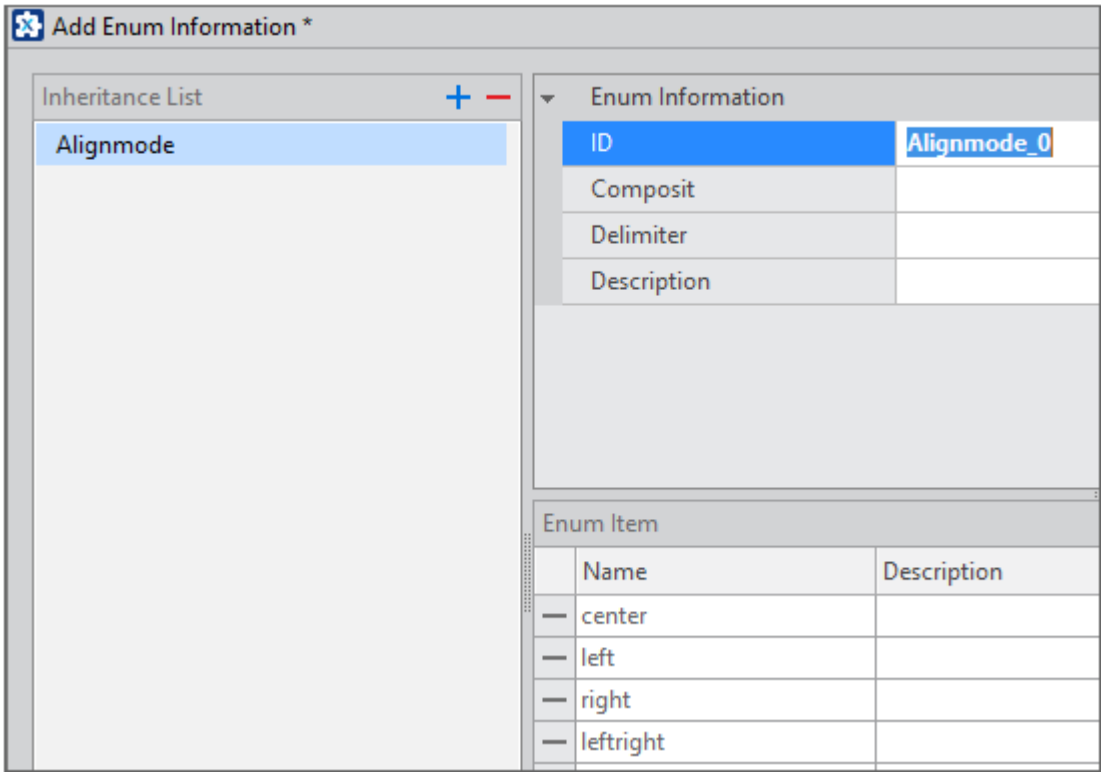
2. Select [Inheritance] from the expanded list.



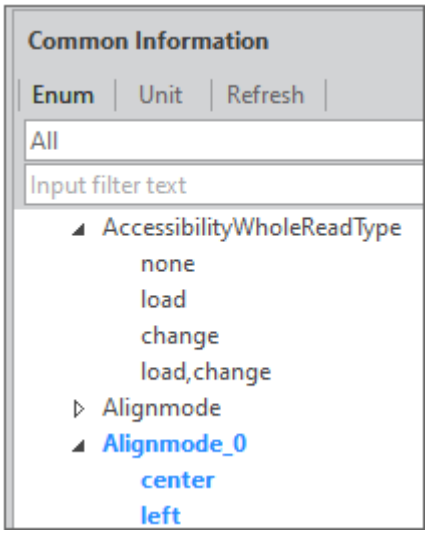
3. Click the [+] button in the Add Enum Information window and select an existing item.



4 Edit ID and item and save.



5 The modified item can be checked in the Common Information window.



7.

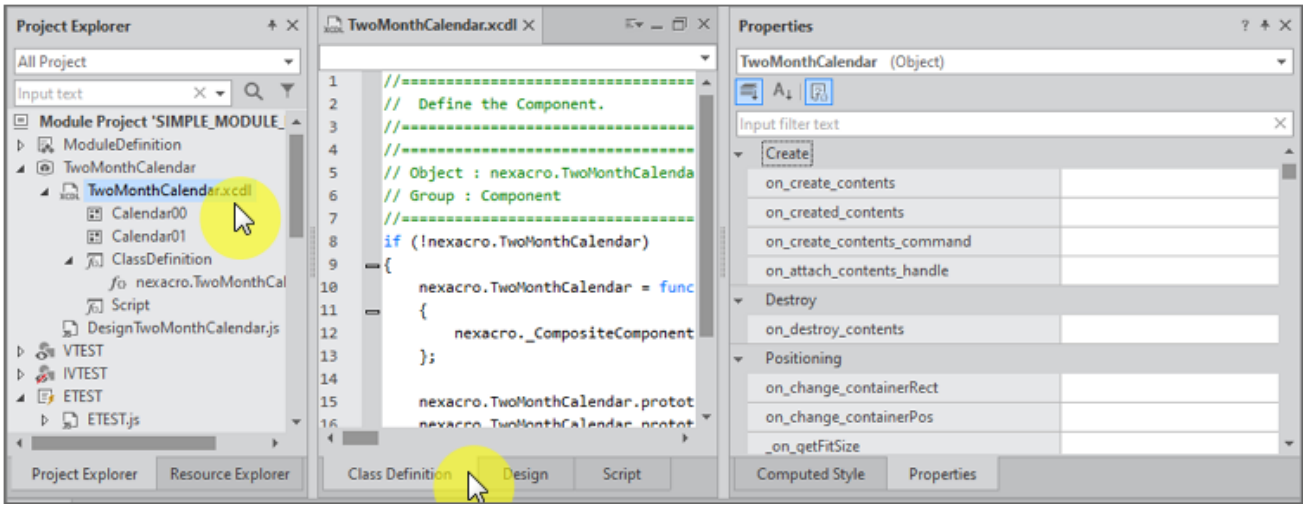
Utilizing Object Interface Functions

If you open the .xcdl file(Composite Component) or [object name].js file in the Project Explorer tab, you can check the object interface list in the property window. A basic template is generated by double-clicking each item in the same way as adding events in Nexacro Studio.

7.1 Checking Object Interface List in Property Window

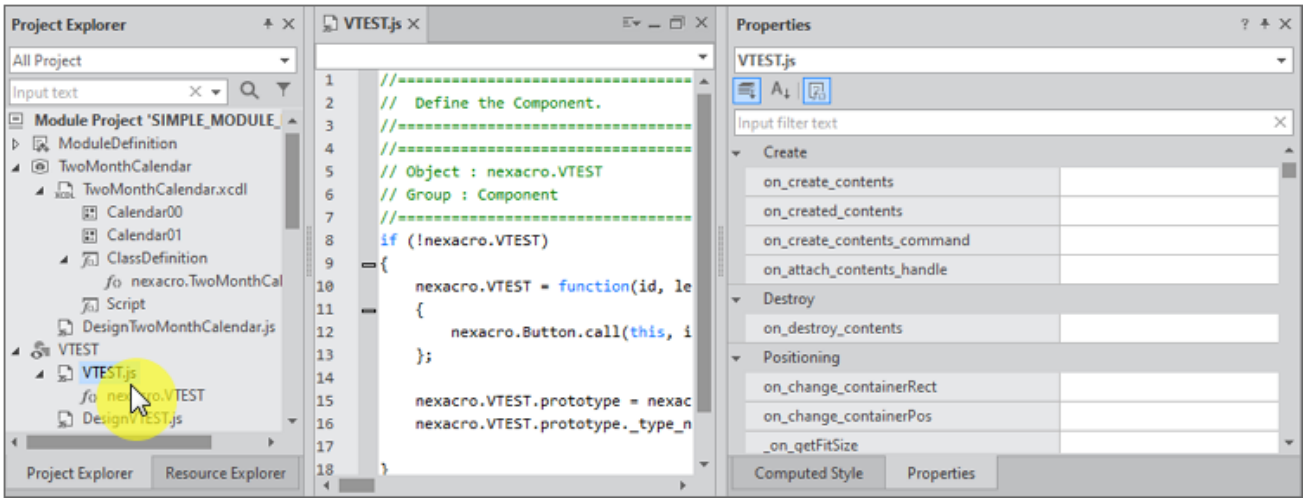
7.1.1 XCDL File (Composite Component)

If you open the .xcdl file in the Project Explorer tab and the Class Definition tab is selected, you can check the object interface list in the property window.



7.1.2 JS File (Object Other than Composite Component)

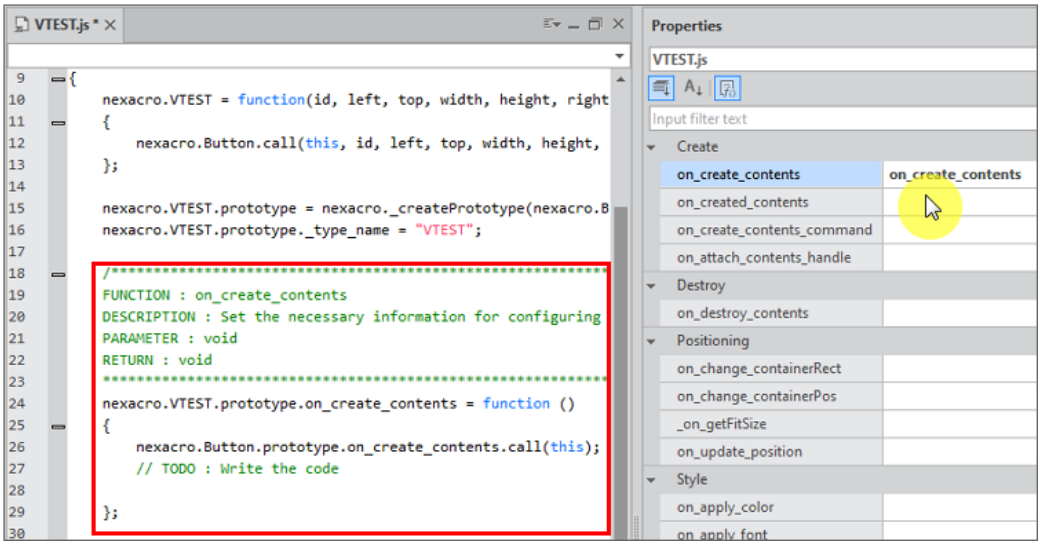
You can see the object interface list in the property window by opening the .js file of the object in the Project Explorer tab.



7.2 Adding Object Interface Function

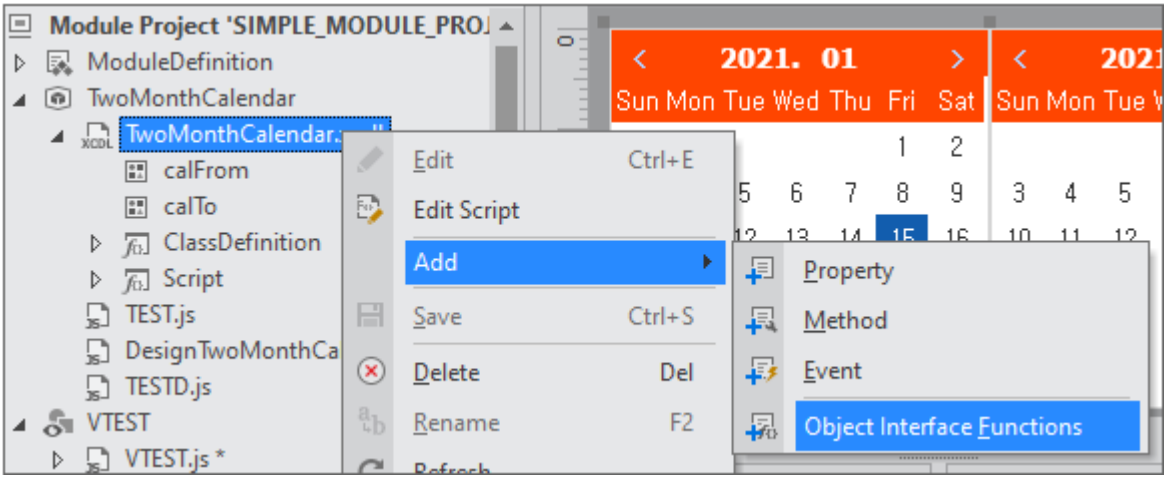
7.2.1 Adding in Property Window

In the same way as adding events in Nexacro Studio, the default template is created by double-clicking each item. A brief description of the interface function and the code to write are provided in the comment form.

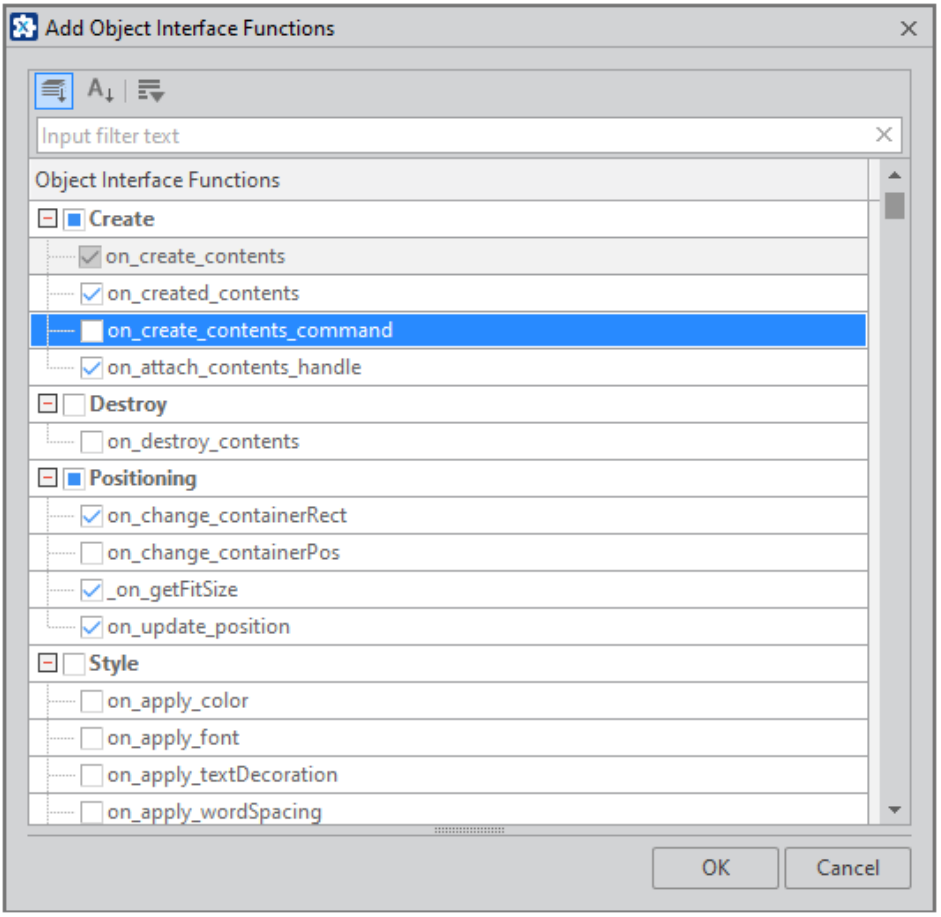


7.2.2 Adding from Menu

You can add the object interface function by selecting the file in Project Explorer and selecting the same item from the menu [Edit > Add > Object Interface Functions] or the context menu.



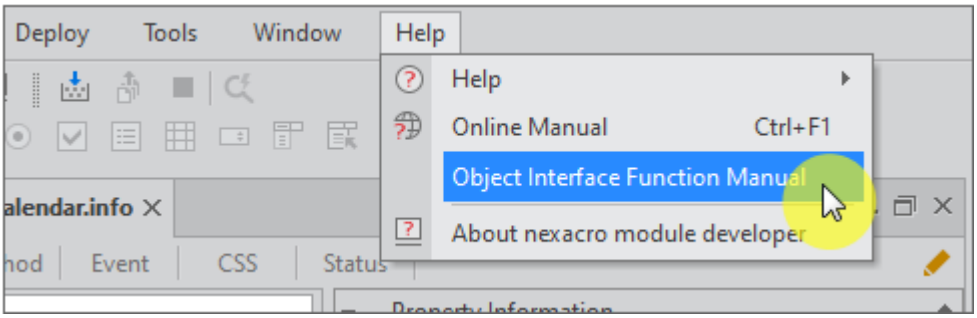
You can add only one object interface function at a time in the property window, but you can select and add multiple items in the [Add Object Interface Functions] window. Select the object interface function to add and click the [OK] button.



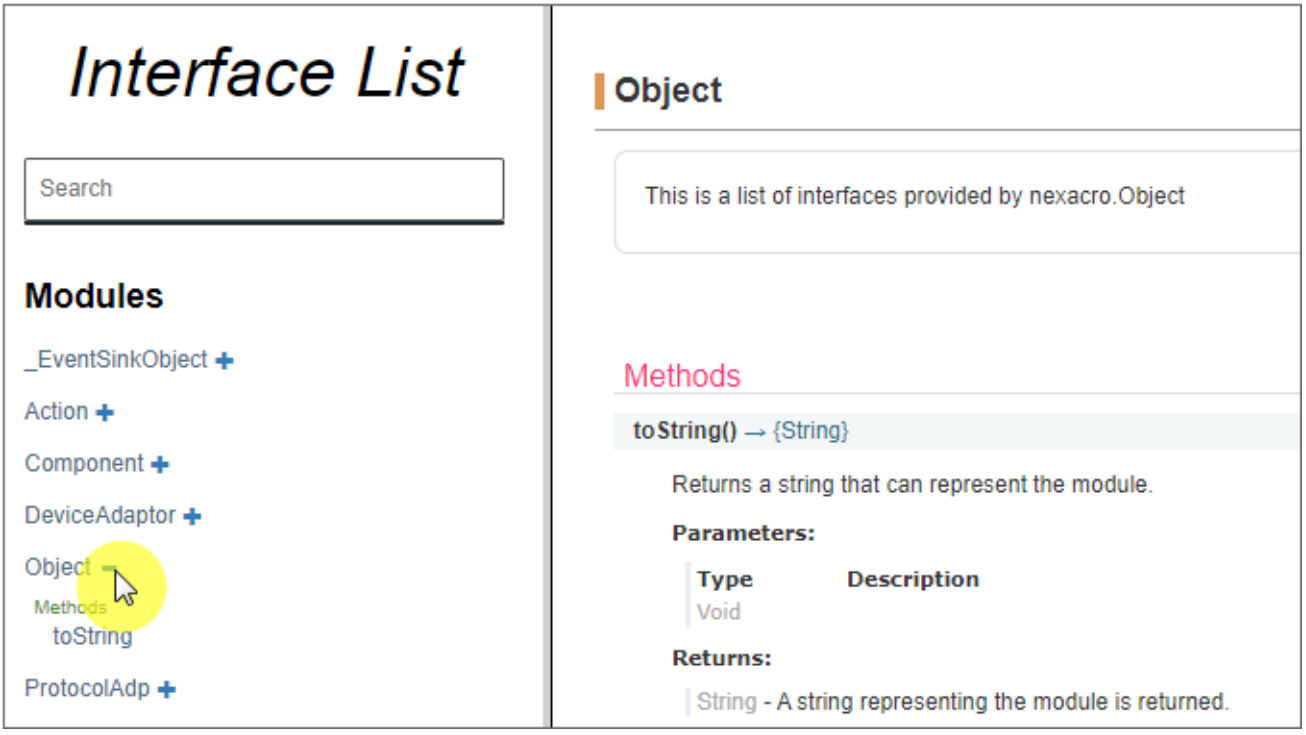
7.3 Checking Object Interface Function Help

The object interface function help is provided in the form of a separate JSDoc. You can check the list of object interface functions that can be used commonly in each object.

If you select the menu [Help > Object Interface Function Manual] item, then the JSDoc file (index.html) from the installation folder is loaded from the web browser.



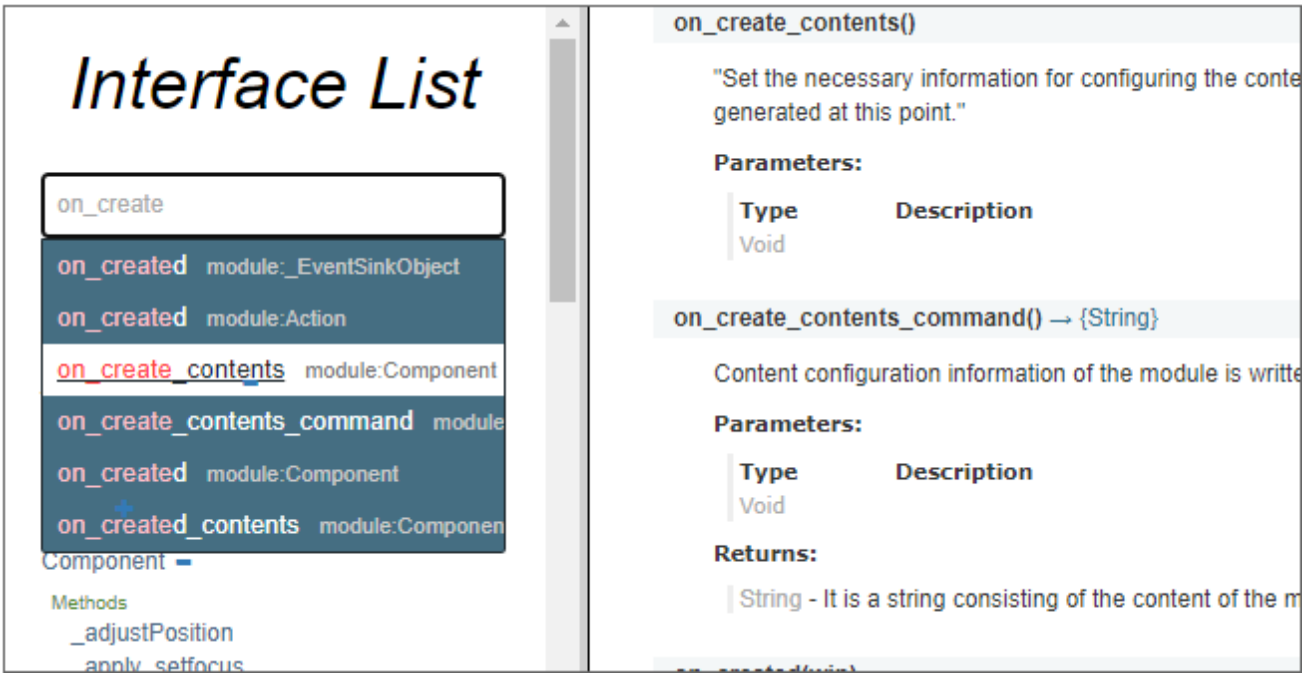
If you select the object type from the Modules list, then the list of object interface functions provided is displayed.



Modules	Object
_EventSinkObject	Invisible Object (except when inheriting the Object, Action objects)
Action	Invisible Object (when inheriting the Action object)
Component	Composite Component, Visible Object

Modules	Object
DeviceAdaptor	Device Adaptor
Object	All objects are implemented by inheriting Object, and the toString() method can be used in common with all objects.
ProtocolAdp	Protocol Adaptor

The function to search for interface function names is provided. It searches for the function name that contains the character string entered in the search window and moves to the corresponding location when clicked.



8.

Contents Editor

In case of the Grid and ListView components, the format of the content display can be set using the format of the Content Editor. This allows components or data to be displayed in various formats. nexacro Module Developer provides a feature to set the contents editor for the module development.

To set the metainfo Contents property, three types of contents editor are used that are provided by default. If you are generating a Content Editor project and want to set it up, you can set it with the object contenteditor property.

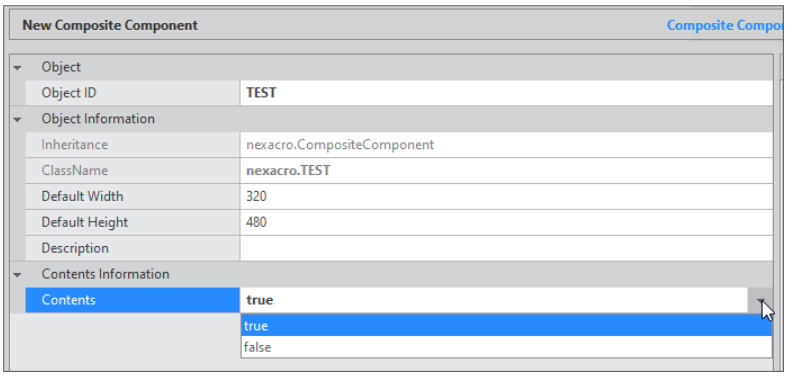
8.1 Setting with Metainfo Contents Property

By setting the Contents property, you can use three types of Content Editor provided by default when setting object properties.

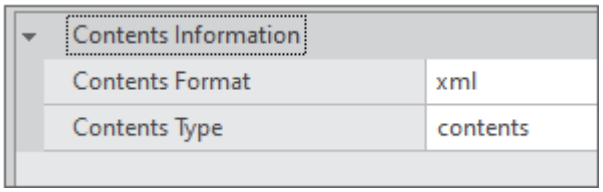
8.1.1 Setting Contents Properties

Setting in Component Wizard

1. Change the setting value of the Contents item to true when generating an object.



2 Click the [Next] button and set Contents Format and Contents Type.



The set property value can be checked again in metainfo.

3 Check Class Definition of the generated object.

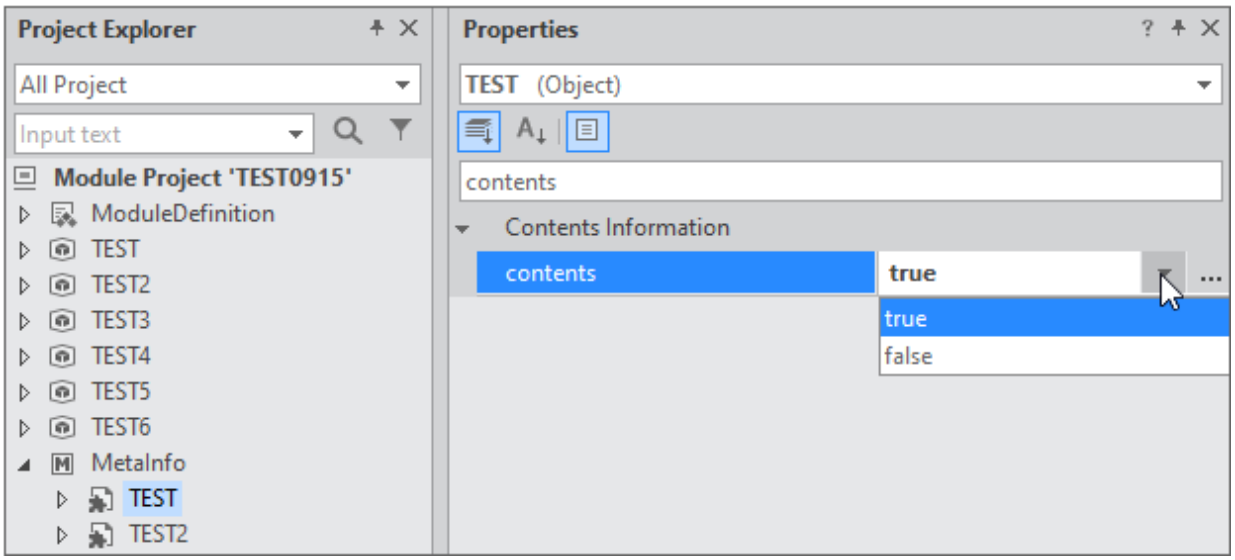
Related functions are automatically generated according to Contents Type as shown below.

```
18      nexacro.TEST.prototype._p_contents = "";
19      nexacro.TEST.prototype.set_contents = function (v)
20      {
21          if(this._p_contents != v)
22          {
23              this._p_contents = v;
24              this._setContents(this._p_contents);
25          }
26
27      };
28
29      Object.defineProperty(nexacro.TEST.prototype, "contents", {
30              get: new Function("return this._p_contents"),
31              set: nexacro.TEST.prototype["set_contents"],
32              enumerable : true,
33              configurable : true});
34
35      nexacro.TEST.prototype._setContents = function (v)
36      {
37          // This function is necessary to apply contents in the run environment.
38          nexacro._CompositeComponent.prototype._setContents.call(this, v);
39
40          // TODO : enter your code here.
41
42      };
```

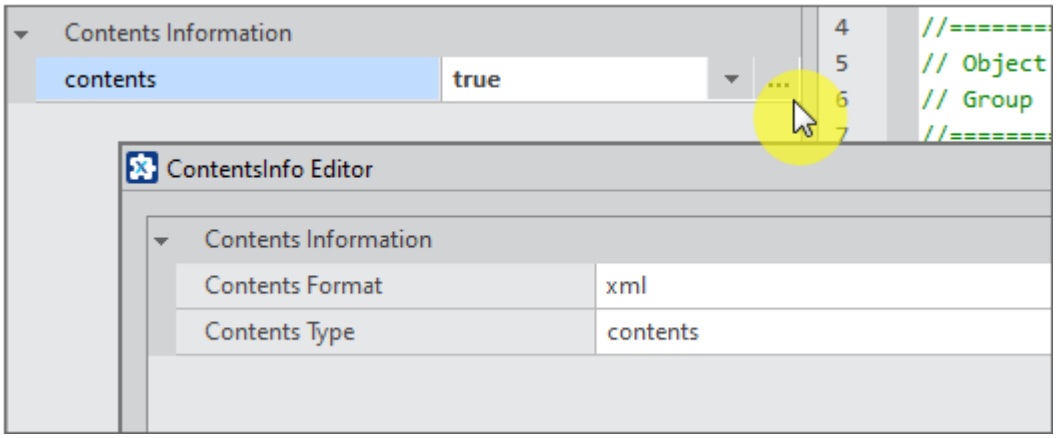
Setting in Metainfo

1 Select an object under MetaInfo in Project Explorer.

2 Set the Contents property value to true in the property window.



3 Click the button of the Contents property item to run ContentsInfo Editor.



4 Set Contents Format, Contents Type.

5 Add the necessary code to the Class Definition of the generated object.

If the Contents property is changed in metainfo, the code is not automatically added. You must add the necessary code yourself.

8.1.2 Setting Contents Format & Contents Type Properties

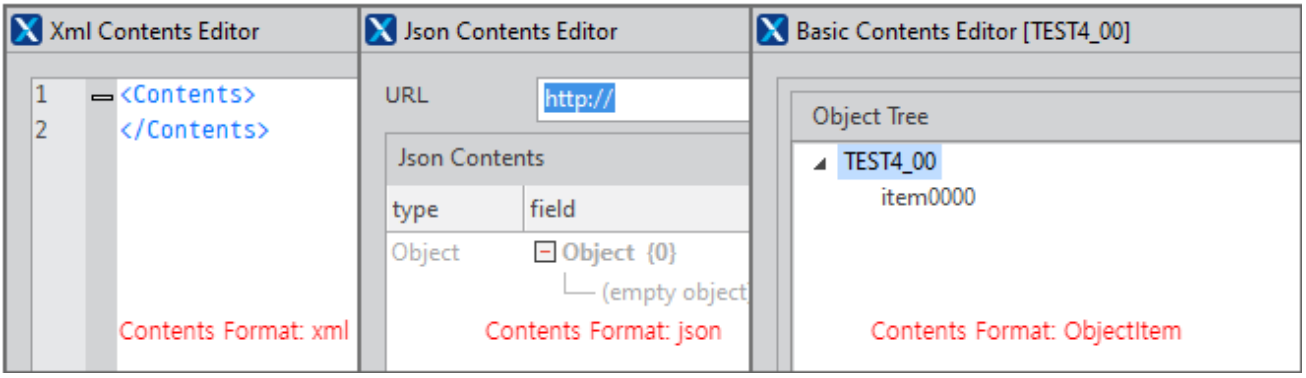
You can set Contents Format and Contents Type properties by setting the Contents property value to true. The Contents Type property settings you can select depend on the Contents Format property.

Contents Format	Contents Type
xml	contents, formats

Contents Format	Contents Type
json	contents
objectitem	contents, formats

Contents Format

Set the format of the property value to be processed. The editor that can be used after module installation varies depending on the selected property value.

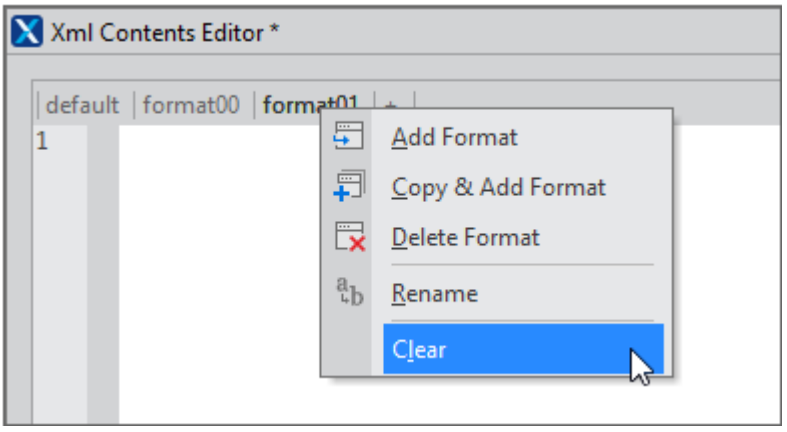


Contents Type

If you set the property value to "formats", you can set more than one data using formatid. In case of Component Wizard, formats and formatid properties and related functions are set as shown below.

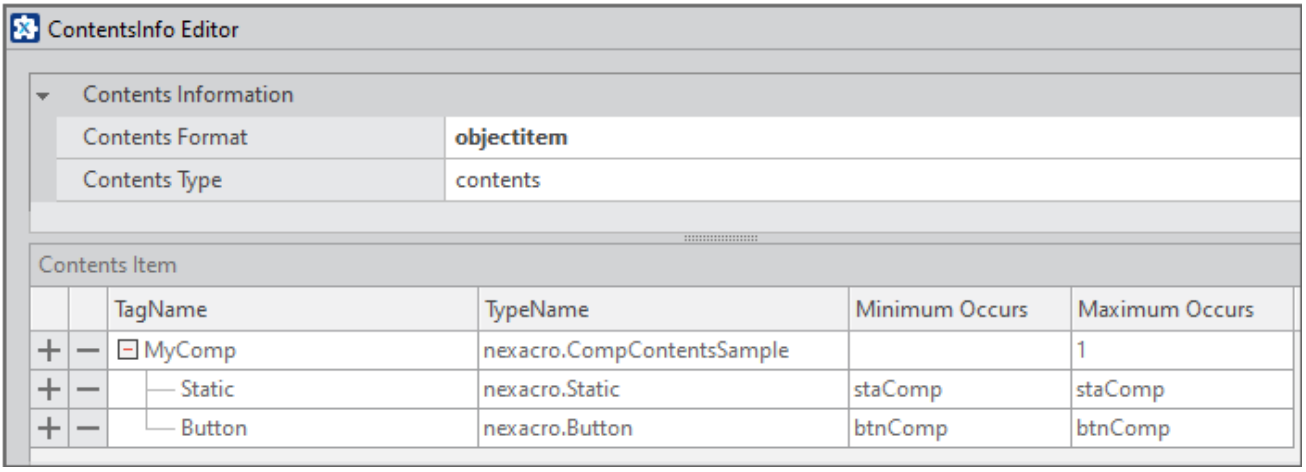
```
44      nexacro.TEST.prototype._p_formats = "";
45      nexacro.TEST.prototype.set_formats = function (v)
46      {
47          if(this._p_formats != v)
48          {
49              this._p_formats = v;
50              this._setContents(this._p_formats);
51          }
52
53      };
54
55      Object.defineProperty(nexacro.TEST.prototype, "formats", {
56              get: new Function("return this._p_formats"),
57              set: nexacro.TEST.prototype["set_formats"],
58              enumerable : true,
59              configurable : true});
60
61      nexacro.TEST.prototype._p_formatid = "";
62      nexacro.TEST.prototype.set_formatid = function (v)
63      {
64          // TODO : enter your code here.
65
66      };
67
68      Object.defineProperty(nexacro.TEST.prototype, "formatid", {
69              get: new Function("return this._p_formatid"),
70              set: nexacro.TEST.prototype["set_formatid"],
71              enumerable : true,
72              configurable : true});
```

A button to add Format is added at the top of the editor that can be used after module installation. The added Format can be deleted or renamed.



8.1.3 Setting Contents Item Properties

If you set the property value of Contents Format to "objectitem", you must set the Contents Item property.



Property Name	Description
TagName	It sets the Tag name used when generating Design Source.
TypeName	It sets the class to display the properties of the selected items in Basic Contents Editor. For the top-level parent item, the current object class is selected, and for the child item, an appropriate class is selected.
Minimum Occurs	If a value of 1 or more is specified, items of more than that number must be set. If no value is set, it is processed as 0 and there is no minimum number limit.
Maximum Occurs	If a value of 1 or more is specified, items of less than that number must be set. If no value is set, it is processed as 0 and there is no maximum number limit.

The properties Minimum Occurs or Maximum Occurs can be set by specifying the id property value.

Property Value	Description
id property value	It can only set one item of the set-id property value. ex) btnContent
id property value | id property value	It sets the property value using the "|" separator. It can only set one item out of two or more id property values. ex) btnContent|btnFotmat
id property value, id property value	It sets the property value using the "," separator. The number of set items out of two or more id property values can be added. Minimum Occurs property must add the set number as the corresponding id property value. Maximum Occurs can add only as many as the set number by the corresponding id property value. ex) btnContent,btnFotmat



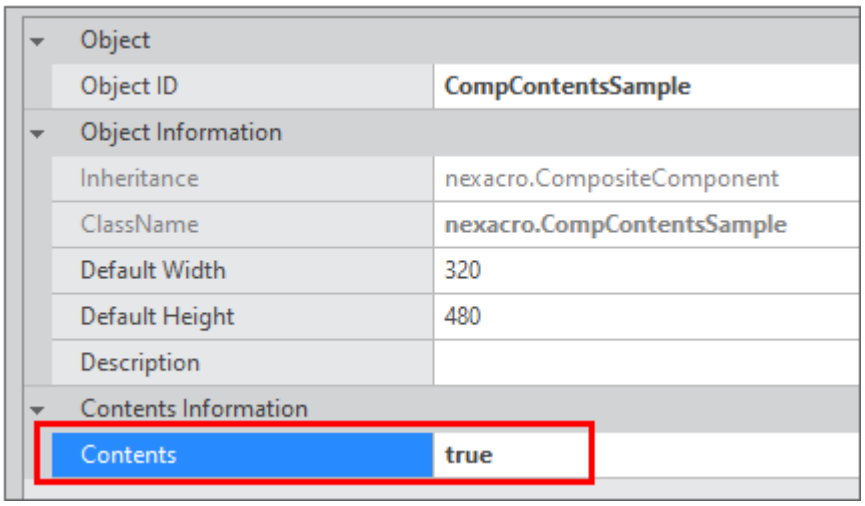
When using "|" or "," separators, blanks must not be included.
If there is a blank character, a message indicating that the character is not allowed is displayed and it is not saved.

8.1.4 Creating Simple Sample

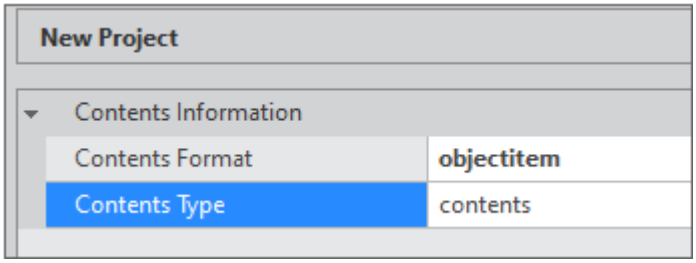
Let's take a look at how to set the Contents property of the object to true, generate a module, and change the Contents property value of the corresponding object in Nexacro Studio.

Generating Module Project

1. Select the menu [File > New > Project] and run Project Wizard.
2. Enter the Project Name and click the [Next] button.
3. Enter Object ID and change the Contents item value to true.



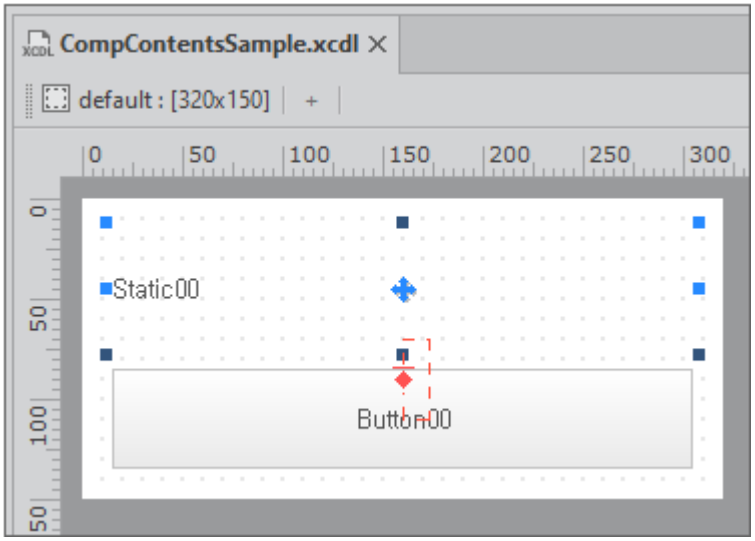
4 Click the [Next] button and change Contents Information as shown below.



5 Click the [Finish] button to generate the project.

Configuring Screen

1 Place the Static and Button components in the design area and change the size of the Form appropriately.



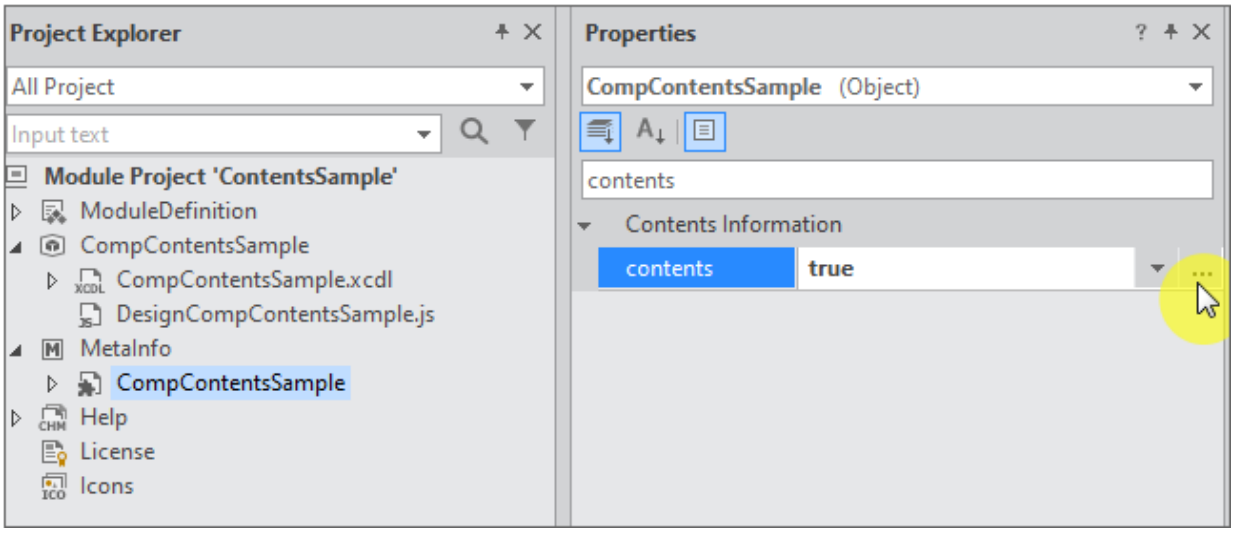
2 Set the component id as follows.

```
Static: staComp
Button: btnComp
```

Setting Contents Item

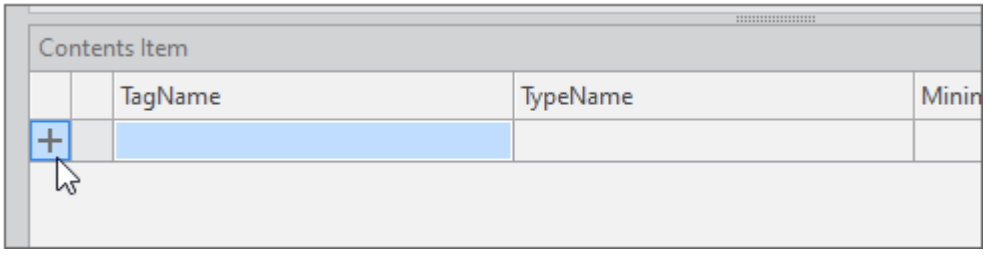
1 Select the object name under the MetaInfo item in Project Explorer.

2 Check the Contents property in the property window and click the button.

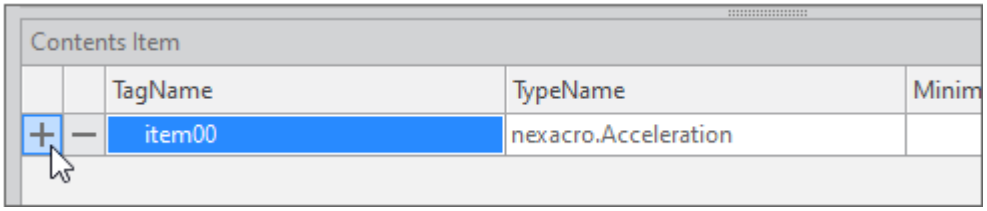


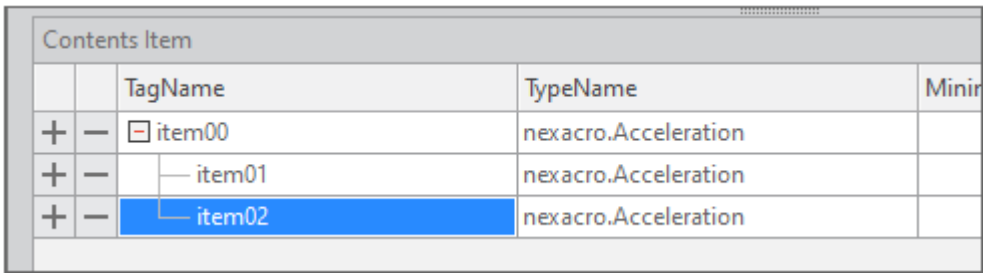
When ContentsInfo Editor is run, add Contents Item.

3 Click the [+] button to add an item.

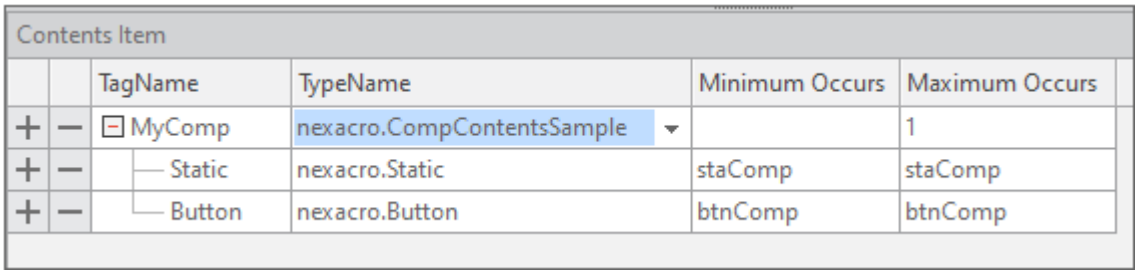


4 Click the [+] button in the added item row to add two child items.





5 Change the items such as TagName and TypeName as follows.



For the top-level parent item's TypeName, select the object class you are working on. In the example, it is "nexacro.CompContentsSample".

When setting the Contents property in Nexacro Studio, each child item can be added one by one and the id value is set to "staComp" and "btnComp".

Writing Script

1 Select the xcdl file under the object name in Project Explorer.

2 Select the Class Definition tab.

When setting the Contents property value, the method of processing the set value must be implemented directly in the script. If the Contents property is set in Component Wizard, the script code is generated as follows.

```
nexacro.CompContentsSample.prototype._p_contents = "";
nexacro.CompContentsSample.prototype.set_contents = function (v)
{
    if(this._p_contents != v)
    {
        this._p_contents = v;
        this._setContents(this._p_contents);
    }
};
```

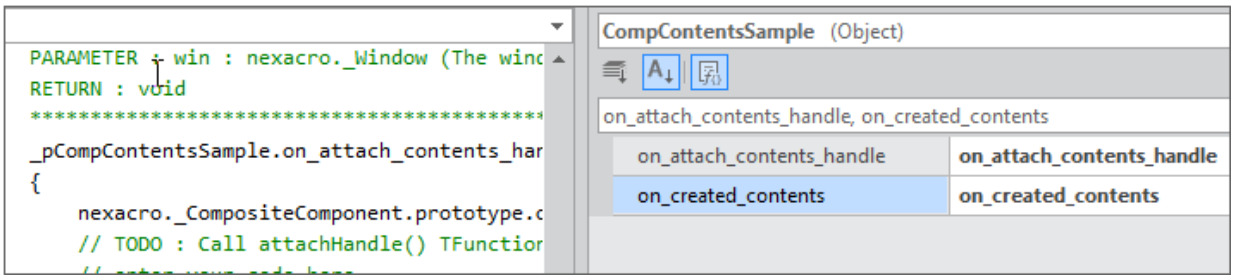
```
Object.defineProperty(nexacro.CompContentsSample.prototype, "contents", {
        get: new Function("return this._p_contents"),
        set: nexacro.CompContentsSample.prototype["set_contents"],
        enumerable : true,
        configurable : true});

nexacro.CompContentsSample.prototype._setContents = function (v)
{
    // This function is necessary to apply contents in the run environment.
    nexacro._CompositeComponent.prototype._setContents.call(this, v);

    // TODO : enter your code here.
};
```

3 Double-click the on_attach_contents_handle, on_created_contents item in the property window to generate the corresponding function.

Since the object interface that occurs in NRE and WRE is different when running the app, we have registered both as follows. If the service target is only WRE, register only the on_attach_contents_handle function.



4 Modify the on_attach_contents_handle, on_created_contents function as follows.

on_created_contents is the function that operates in NRE. It is run when the corresponding component is loaded in Nexacro Studio, resulting in a duplicate function being run. Therefore, check system.navigatorname so that it is not run in Nexacro Studio.

```
nexacro.CompContentsSample.prototype.on_attach_contents_handle = function
(win)
{
    nexacro._CompositeComponent.prototype.on_attach_contents_handle.call(
this, win);
    this._setContents(this.contents);
};
```

```
nexacro.CompContentsSample.prototype.on_created_contents = function (win)
{
    nexacro._CompositeComponent.prototype.on_created_contents.call(this,
win);
    if(system.navigatorname != "nexacro DesignMode")
    {
        this._setContents(this.contents);
    }
};
```

5 Modify the _setContents function as follows.

The _setContents function is called in the following situations -

- Form loading with object added in Nexacro Studio
- Change of object Contents property value in Nexacro Studio
- Calling the on_attach_contents_handle, on_created_contents function when loading the app

The value set with the Contents property is passed as an XML string. It generates the DOMDocument object using DomParser within the function, checks the values set in each item, and reflects the set value.

```
nexacro.CompContentsSample.prototype._setContents = function (v)
{
    // This function is necessary to apply contents in the run environment.
    nexacro._CompositeComponent.prototype._setContents.call(this, v);

    // TODO : enter your code here.
    this.contents = v;
    var objDoc = new nexacro.DomParser();
    var xmlDoc = objDoc.parseFromString(this.contents);
    var obj = null;
    var n, children = null;

    var contentsNode = xmlDoc.firstChild;
    if(!contentsNode)
        return;

    var objectNode = contentsNode.firstChild;
    if(!objectNode || objectNode.tagName != "MyComp")
        return;
```

```
    var childNode = objectNode.firstChild;
    while(childNode)
    {
        var id = childNode.getAttribute("id");
        var obj = this.form[id];
        if(obj == undefined)
            break;

        var attributes = childNode.attributes;
        var nCount = attributes.length;

        for(var i=0; i<nCount; i++)
        {
            var attr = attributes[i];
            var attr_name = attr.name;
            var attr_value = attr.value;

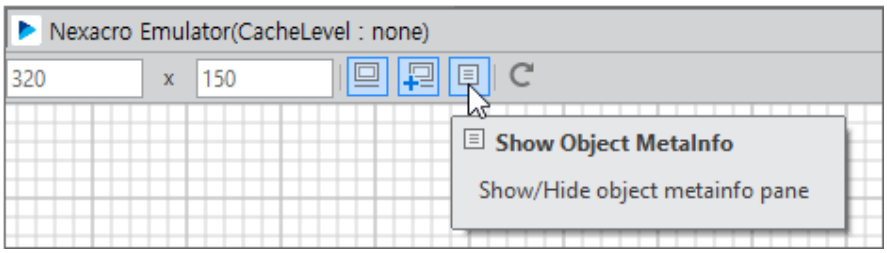
            if(attr_name == "id")
                continue;

            var setter = obj["set_"+attr_name];
            if(setter)
                setter.call(obj, attr_value);
        }
        childNode = childNode.nextSibling;
    }
};
```

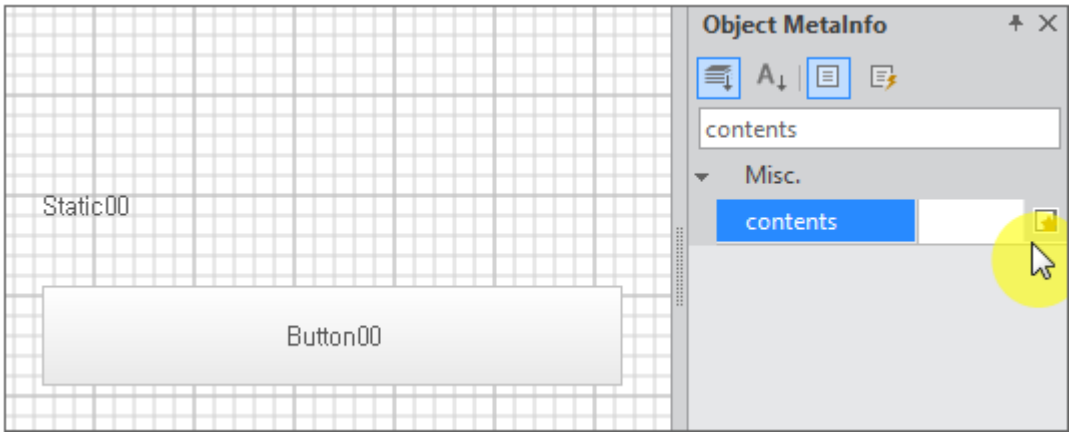
Checking Operation in Emulator

1. Select the menu [Generate > Emulate] and run the emulator.

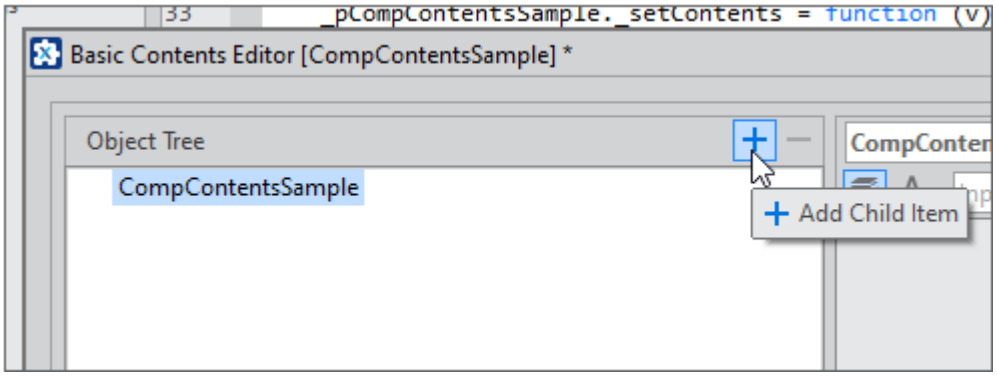
2. If the property window is not visible, click [Show Object MetaInfo] item from the top icons.



3 Check the Contents property in the property window and click the button.



4 When Basic Contents Editor is run, click the [+] button to add an item.

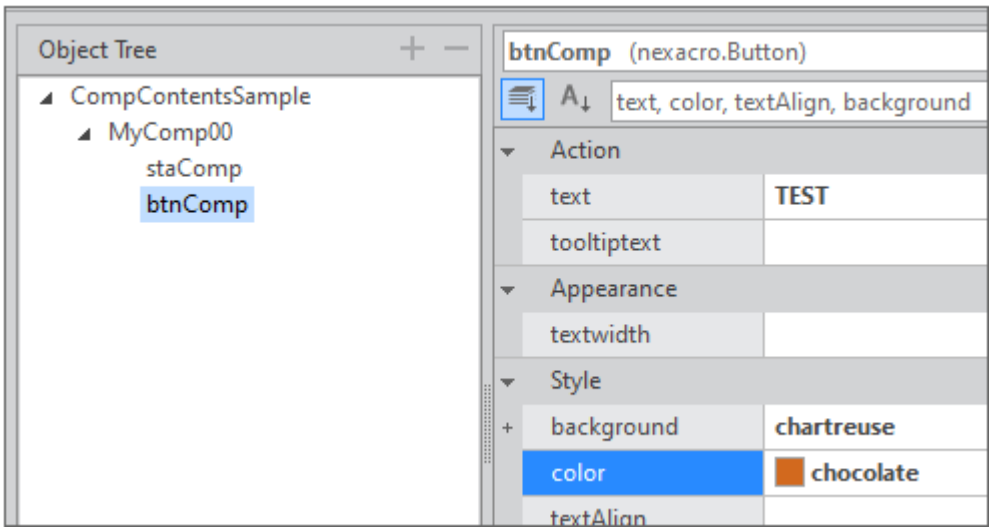


5 Select the top-level parent item added and click the [+] button to add two items.

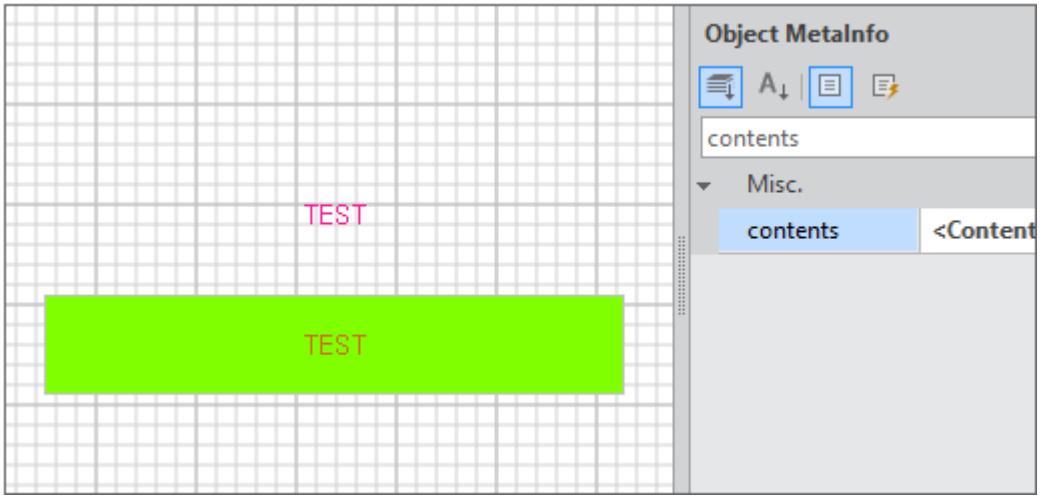
In the previous step, it was set to add only two items with the specified id when setting the Contents Item property, so only two items can be added.



6 Select the child item added and appropriately modify the property value of the component in the property window.



7 Click the [OK] button and check that the set property value is displayed on the emulator.



Installing Module & Checking Operation

Now, deploy the module and check it in Nexacro Studio.

1 Select the menu [Deploy > Module Package] and generate the module installation file.

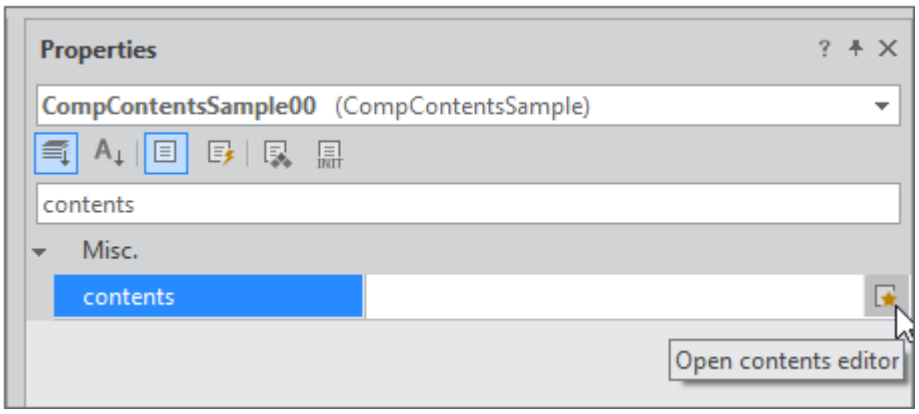
Import the deployed module installation file from Nexacro Studio.

2 Generate a project or run an existing project in Nexacro Studio.

3 Select the menu [File > Install Module Wizard] and install the module file.

4 Place the CompContentsSample component on Form.

5 Check the Contents property in the property window and click the button.



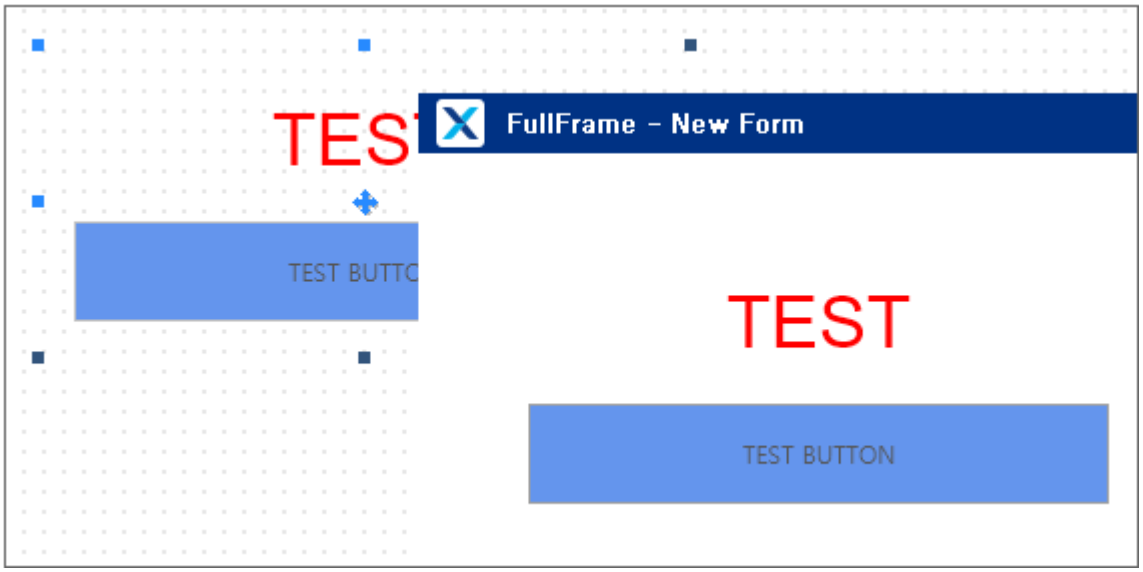
Adding and setting items in the [Contents] tab is the same as in the emulator. In this step, we will look at how to set it in the [Design Source] tab.

6 In the [Design Source] tab, copy and paste the following code to set the property value.

```
<Contents>
  <MyComp id="MyComp00">
    <Static id="staComp" text="TEST" color="red" font="36px/normal
&amp;quot;Arial&amp;quot;" textAlign="center"/>
    <Button id="btnComp" text="TEST BUTTON" background="cornflowerblue"/>
  </MyComp>
</Contents>
```

7 Click the [OK] button and check that the set property value is displayed in the design window.

8 Run Quick View to check if the component is displayed with the set properties in NRE or web browser.



9 Add the Button component to Form and write the onclick event function as follows.

When clicking the button, set the Contents property value of the object and change the text property value within the object.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.CompContentsSample00.set_contents("<Contents><MyComp
id='MyComp00'><Static id='staComp' text='SAMPLE'/><Button id='btnComp'
text='SAMPLE BUTTON'/></MyComp></Contents>");
};
```

10 Run Quick View and check if the text changes when clicking the button.



8.2 Setting with Object Contentseditor Property

Nexacro studio supports connecting and using the contents editor projects that are developed separately.

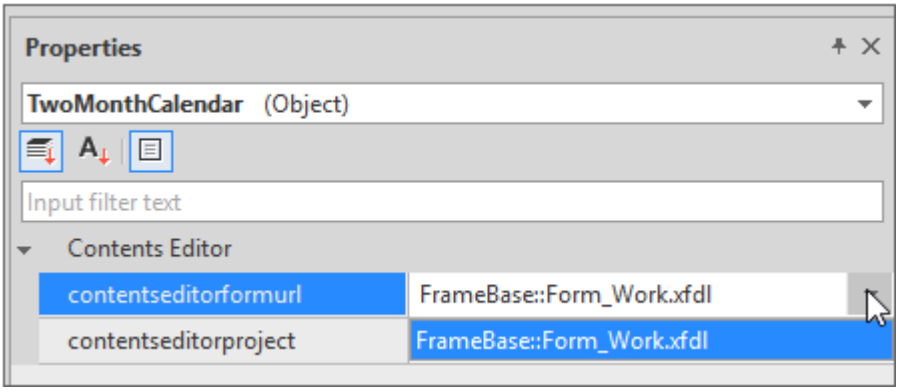
The feature of individually generating contents editor in nexacro Module Developer is not yet supported.

Generating and writing contents editor projects in Nexacro Studio is a feature that is not officially supported. It is the feature that is used by some partner companies that test-produce module components.

When selecting an object in Project Explorer, you can specify the Contents Editor related properties in the

property window.

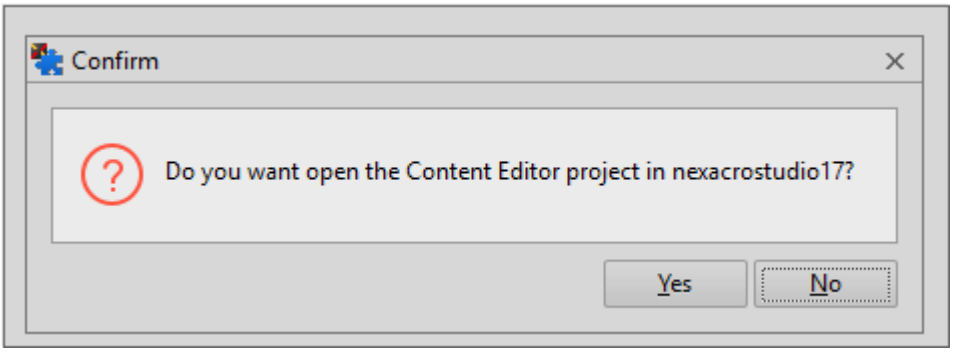
Item	Description
contentseditorformurl	It specifies the XFDL file that designed the contents editor screen. Specify the contentseditorproject item value first and select from the Form list within the corresponding project.
contentseditorproject	It specifies the contents editor project path.



The contents editor project specified in the contentseditorproject item must be located in the module project child-path.

When editing ContentsInfo metainfo, the contentseditor item must be set to "external".

The registered contents editor project is exposed to Project Explorer and when selecting an item, it checks whether to run in Nexacro Studio.



9.

Creating Help

When deploying the module project, you can also deploy a help documentation that the user can refer to. You can give the basic description of components and explain how to use properties, methods, and events.

The current version only supports help documentation in the CHM (Compiled HTML) file format. In this chapter, we will look at how to create and deploy help documentation in the CHM file format.

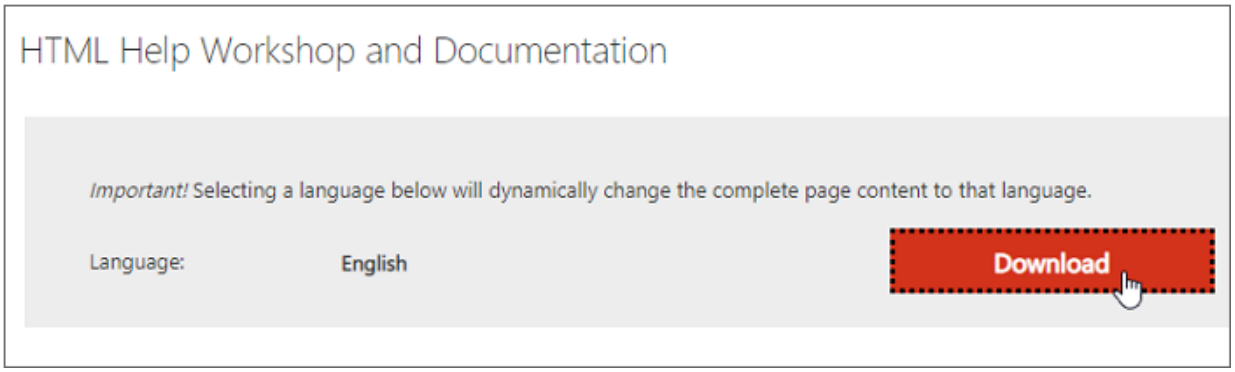
9.1 Installing HTML Help Workshop

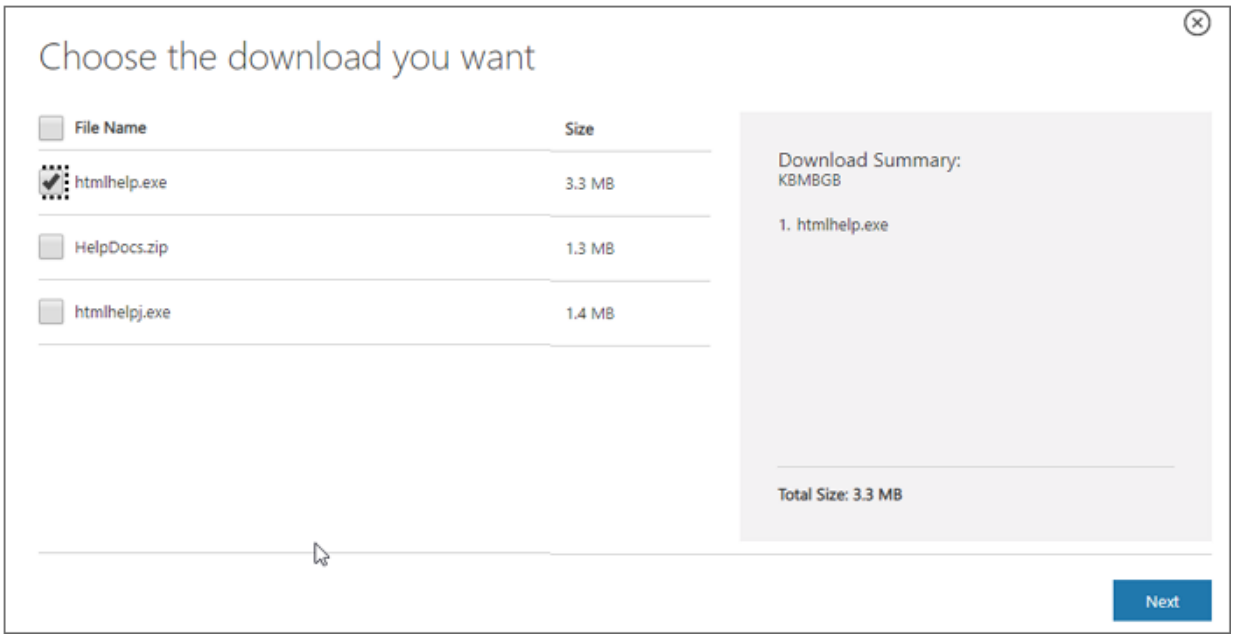
There are several tools that can generate the CHM file. Among them, the HTML Help Workshop is provided free of charge and it lets you generate the CHM file easily.

1. Access the link below.

   > https://www.microsoft.com/en-us/download/details.aspx?id=21138

2. Click the [Download] button, select the "htmlhelp.exe" file from the list of files, and click the [Next] button.

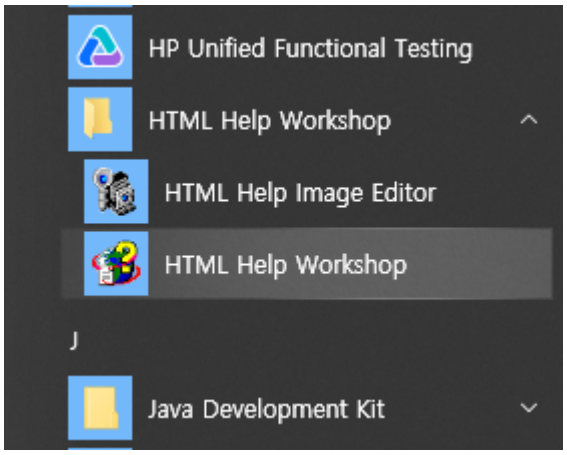




3. Download the htmlhelp.exe file.

4. Run the downloaded htmlhelp.exe file to install HTML Help Workshop.

5. Check and run the HTML Help Workshop in the list of apps.



9.2 Generating Simple CHM File

You can set various things, such as a table of contents and index, in the CHM file, but in this example, we will generate the CHM file with only one page.

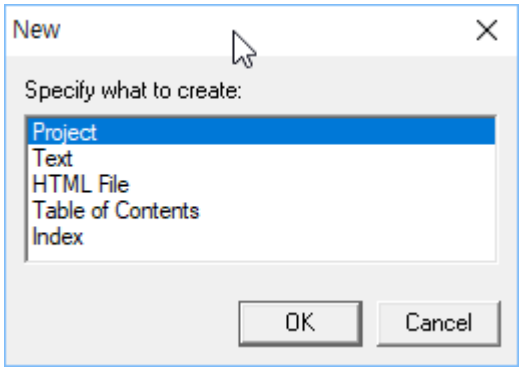
1. Generate the HTML file first in the location to generate the project.

   Configure the HTML file simply using the PRE tag and save as Help.htm.

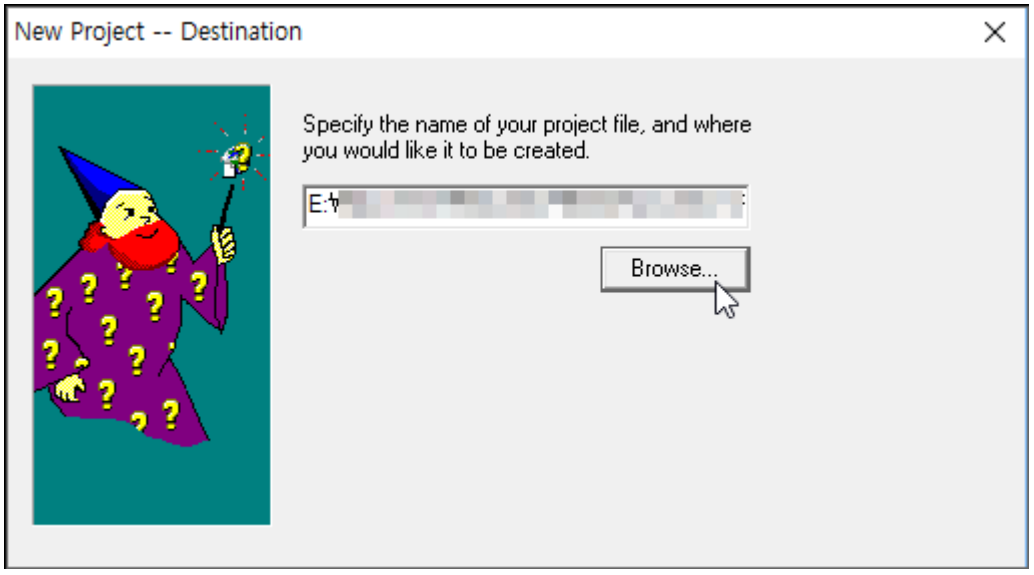
```
<html>
<body>
<pre>
SIMPLE MODULE
- TwoMonthCalendar
</pre>
</body>
```

2. Run HTML Help Workshop.

3. Select [File > New] item from the menu and select [Project] for the type to be generated.



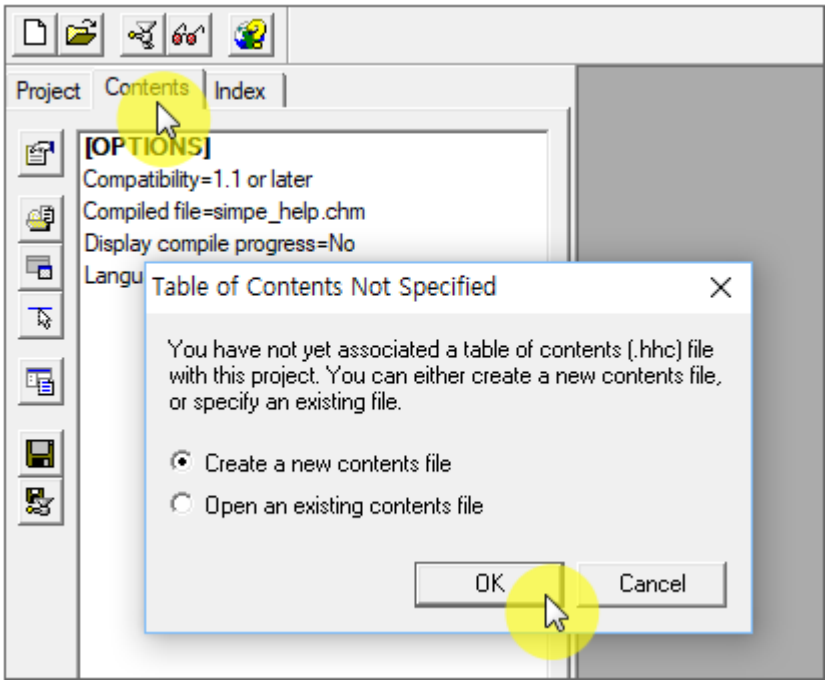
4. Specify the project file path without checking anything else in the Project Wizard.



When the project is generated, basic information is displayed on the Project tab.

5 Select the Contents tab and add content.

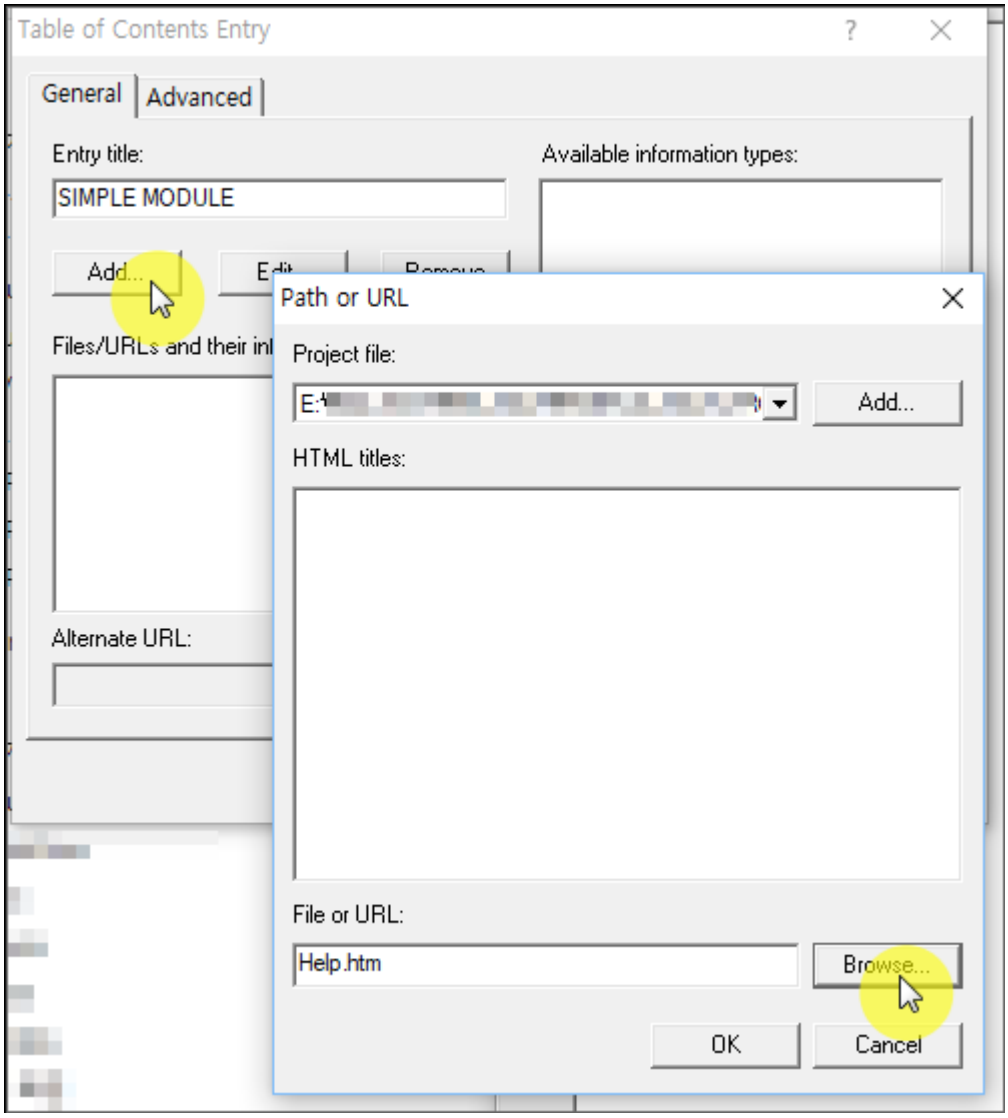
When you select the Contents tab, it will ask if you want to generate a new hhc file. With the [Create a new contents file] item selected, click the [OK] button.



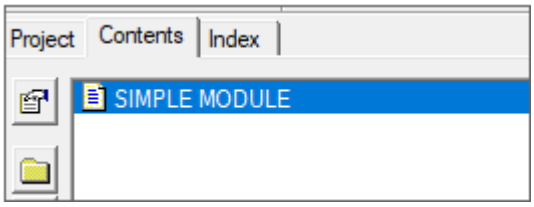
6 Select [insert a page] item from the right toolbar.



7 Enter "SIMPLE MODULE" in the Entry title item and click the [Add] button to display a window for selecting a file. Click the [Browse] button and select the Help.htm file generated earlier.

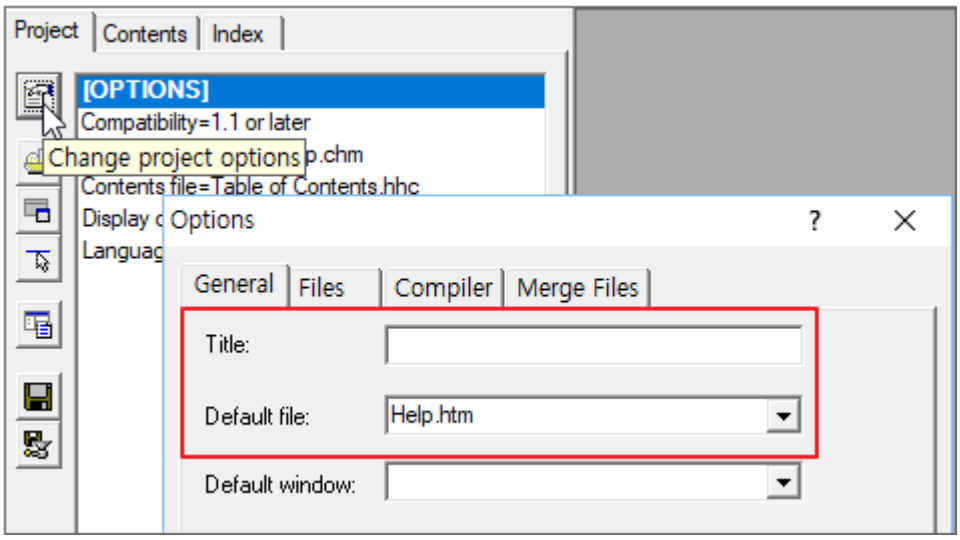


8 Check that the list of pages added in the Contents tab is displayed.



9 Select the Project tab and select [Change project options] item from the right toolbar.

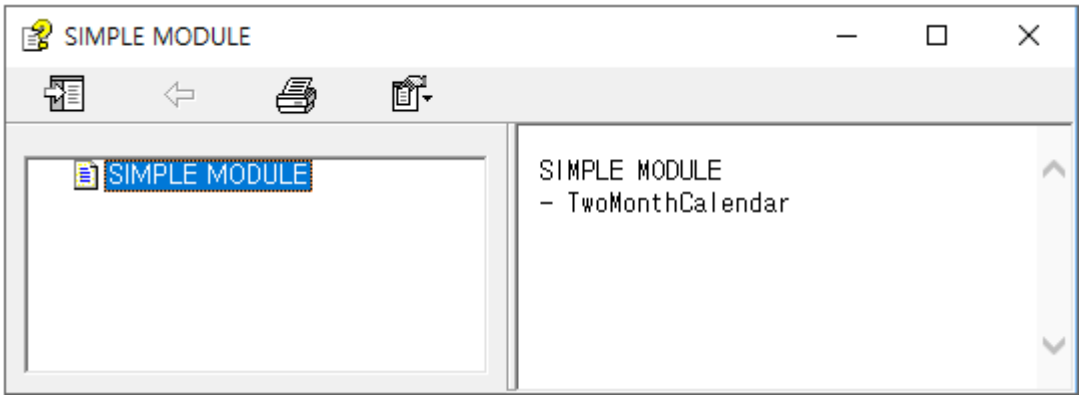
10 Enter the Title item in the Options window and enter the Help.htm file in the Default file item.



11 Select [File > Compile] item from the menu to generate the CHM file.

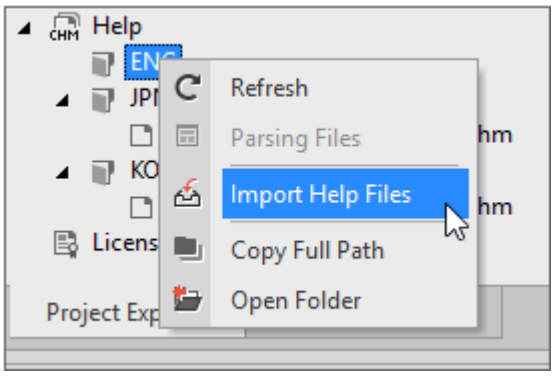
> If there is no file specified when generating the CHM file, an error message is displayed.

12 Run the generated CHM file to check if the content is displayed normally.



9.3 Adding Help to Module Project

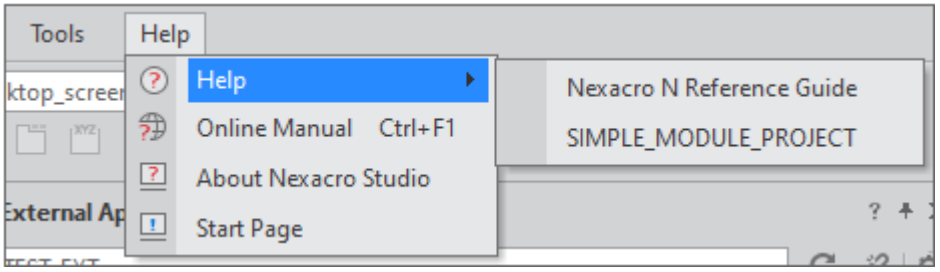
1 On the Project Explorer tab, select the language (CHN, ENG, JPN, KOR) folder under the Help item, and select the [Import Help Files] item from the context menu.



2 Select and add the CHM file. You can add the CHM file for the selected language folder.

> Depending on the language set in the operating system, for Chinese, Japanese and Korean, run the CHM file under the corresponding folder, and for other languages, run the file under the ENG folder.

3 Install the module in Nexacro Studio. You can check the added help in the menu [Help > Help].



4 After placing the composite component on the Form screen, select the component and press the F1 shortcut key to run the module help.

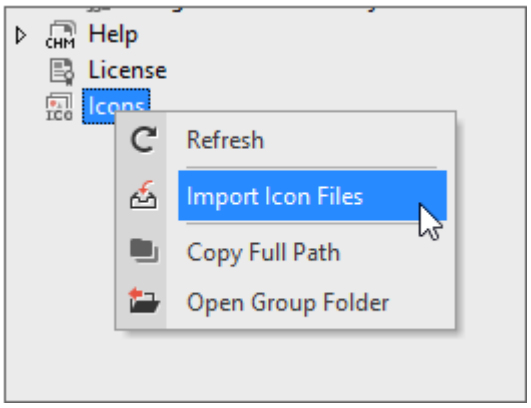
10.

Setting Icons

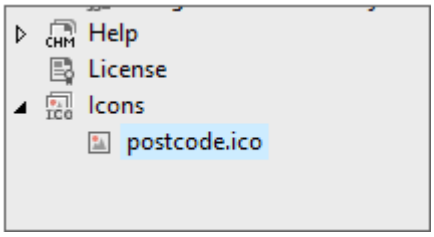
10.1 Importing Icon File

Icon files are managed in project units and each module object selects one of the registered icon files.

1. Select the icons item in Project Explorer and select the [Import Icon Files] item from the context menu.

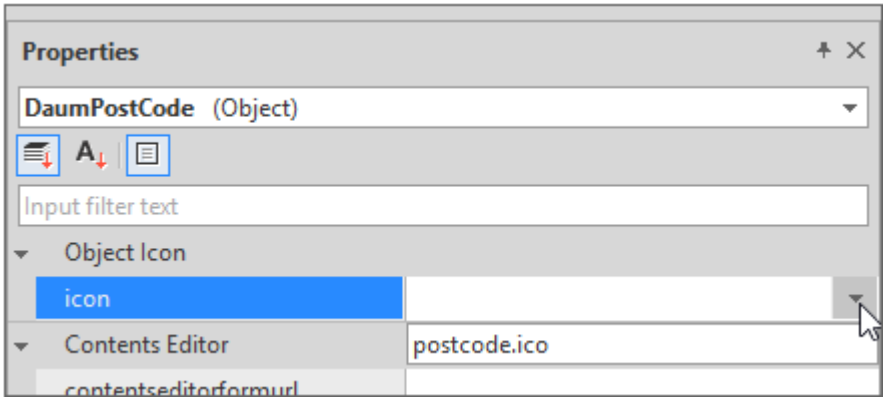


2. Check that the ico file selected in Project Explorer is displayed.

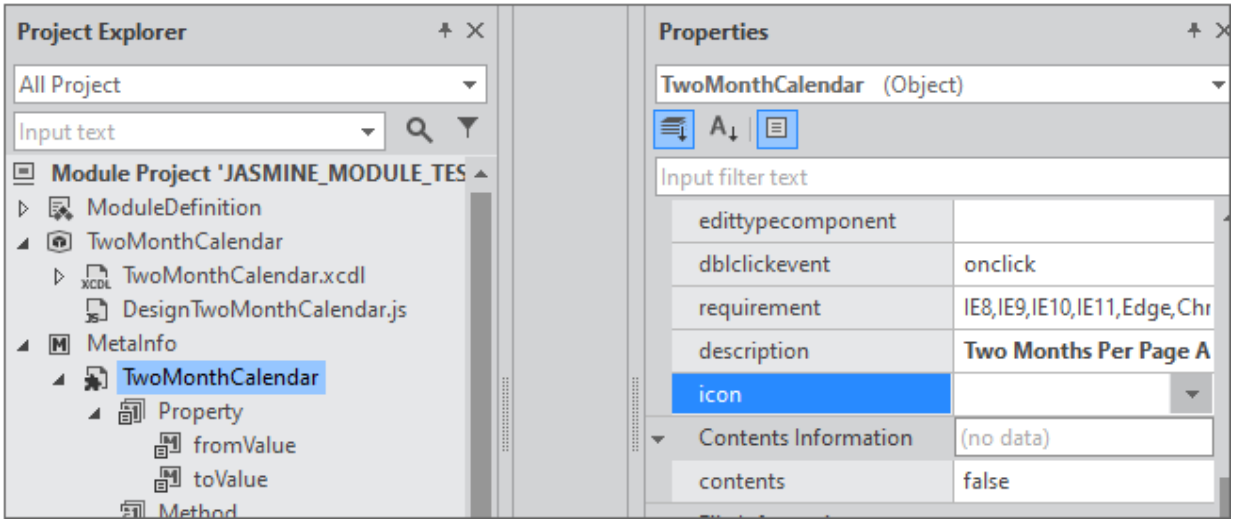


10.2 Deployment after Setting Icon to Module Object

1. If you select a module object in Project Explorer, you can set the icon in the property window. Select the file you want from the icon list.



Alternatively, you can select the module object under MetaInfo and set the icon in the property window.



2. Select the menu [Deploy > Module Package] and deploy the generated module.

3. When installing the module in Nexacro Studio, you can check the icon image specified in "Install Module Wizard". After the module installation, it is indicated as the specified icon on the toolbar.

10.3 Creating Icon File

The default size of the icon file (ico) is 16 pixels, and in a high-resolution environment, a size of 32 pixels is applied. If there is no 32-pixel icon image, a 16-pixel icon image is enlarged and applied.

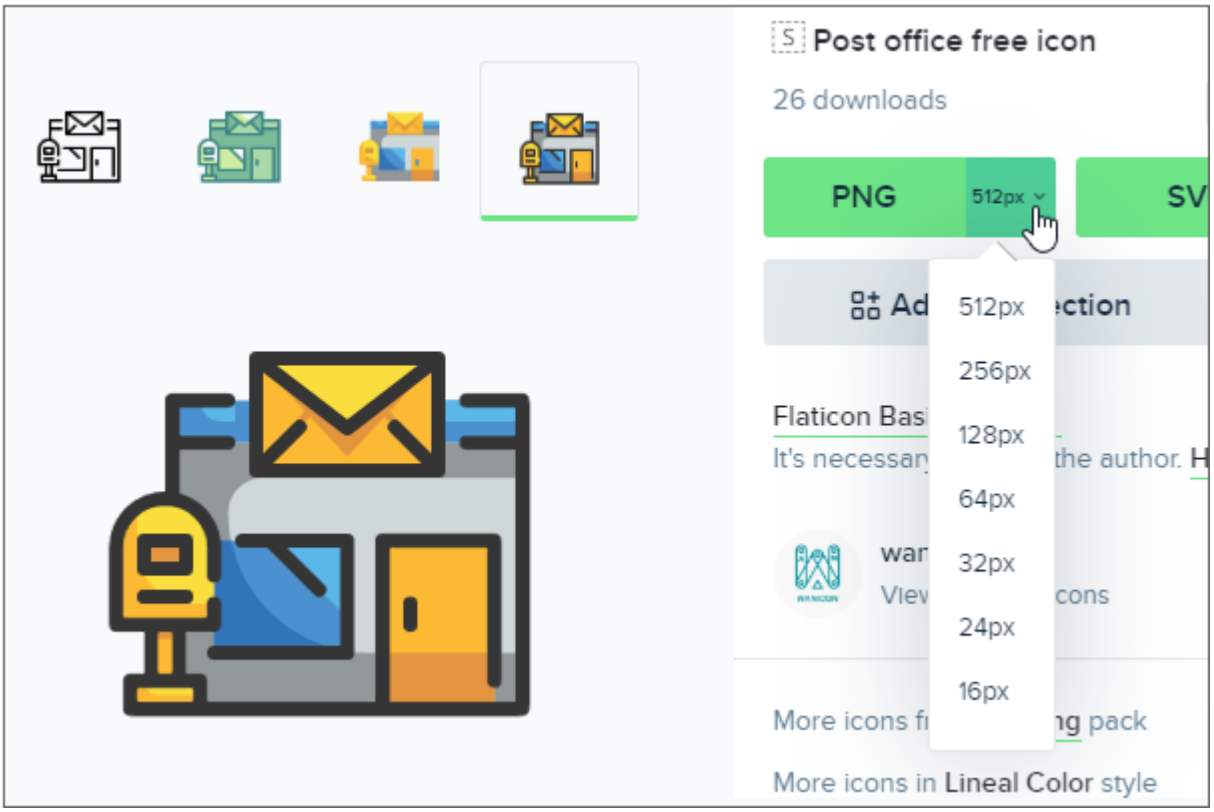
You can create icon files in Photoshop or other design tools. A simpler way is to use an online service that converts image files to ico files.

> The online service mentioned in the example below may not be supported depending on the service situation.

10.3.1 Finding Default Icon Image

Find and download the image you want from the free icon image site. flaticon.com provides icon images by size. Download 16px and 32px PNG images.

> https://www.flaticon.com/



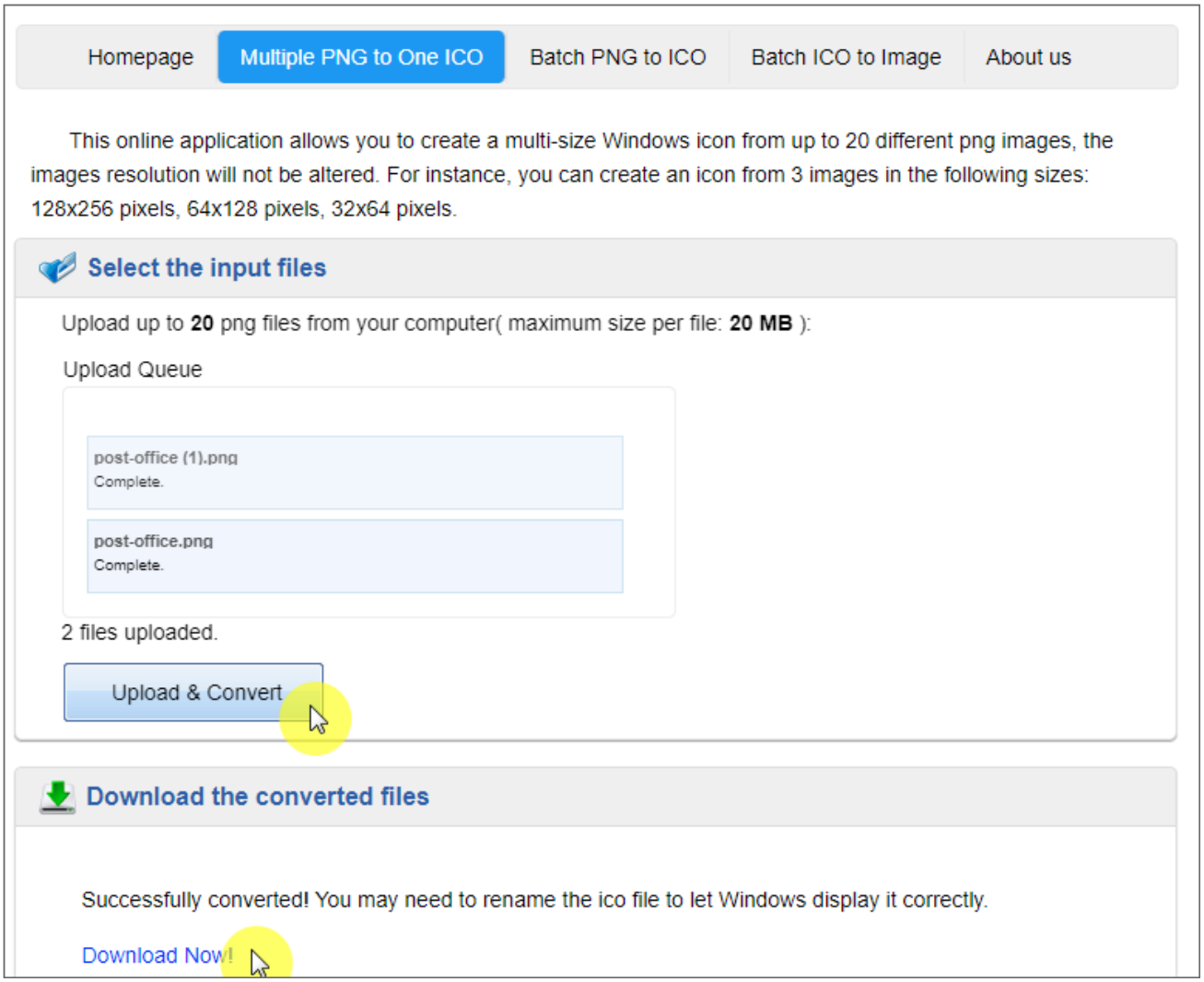
Images provided by flaticon.com must indicate their source when used.
Please indicate the source of the icon image in the license file when deploying the module.
Icons made by wanicon from www.flaticon.com
https://www.flaticon.com/free-icon/post-office_1908148

10.3.2 Generating ico File

Turn two image files of 16 pixels and 32 pixels into one ico file. icoconvert.com provides a service that converts multiple image files into a single icon file.

```
https://icoconvert.com/Multi_Image_to_one_icon/
```

If you upload two image files by clicking the [Upload & Convert] button, it will convert to the ico file, and a link to download the ico file will be displayed.



11.

Option Settings

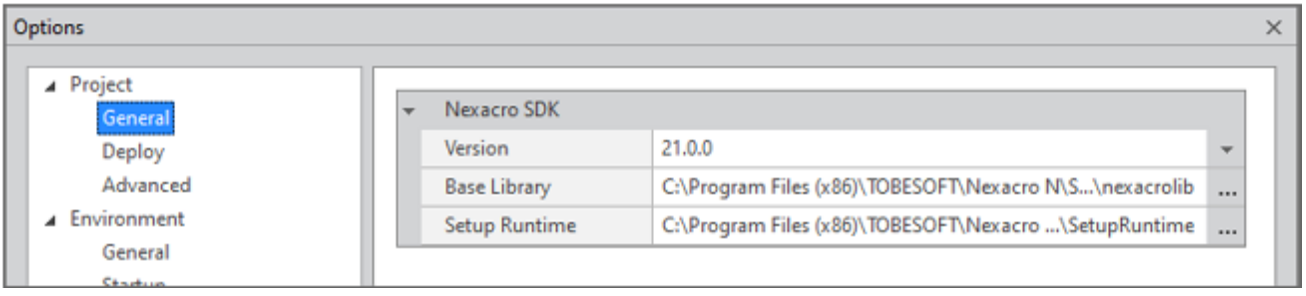
You can set the development environment by selecting the menu [Tools > Option] of nexacro Module Developer.

11.1 Project

The Project option is only visible when the project is open in nexacro Module Developer and the set values of each project are managed separately.

11.1.1 General

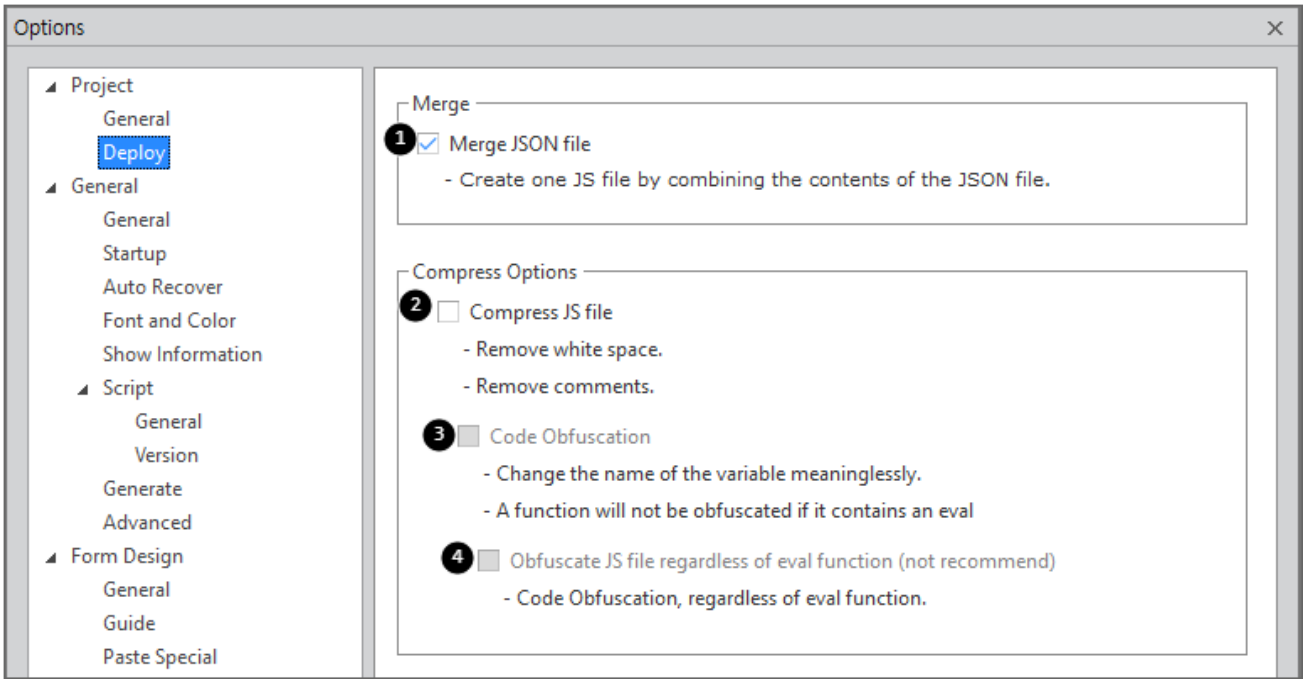
It sets the project default options.



Property	Description
Nexacro SDK	Select the SDK version to use for the project. You can choose among the SDK versions installed in the SDK folder. - Base Library: Displays the Base Library folder corresponding to the selected SDK. - Setup Runtime: Displays the Setup Runtime folder corresponding to the selected SDK.

If needed, you can select a folder different from the selected SDK version for the Base Library and Setup Runtime folders.

11.1.2 Deploy



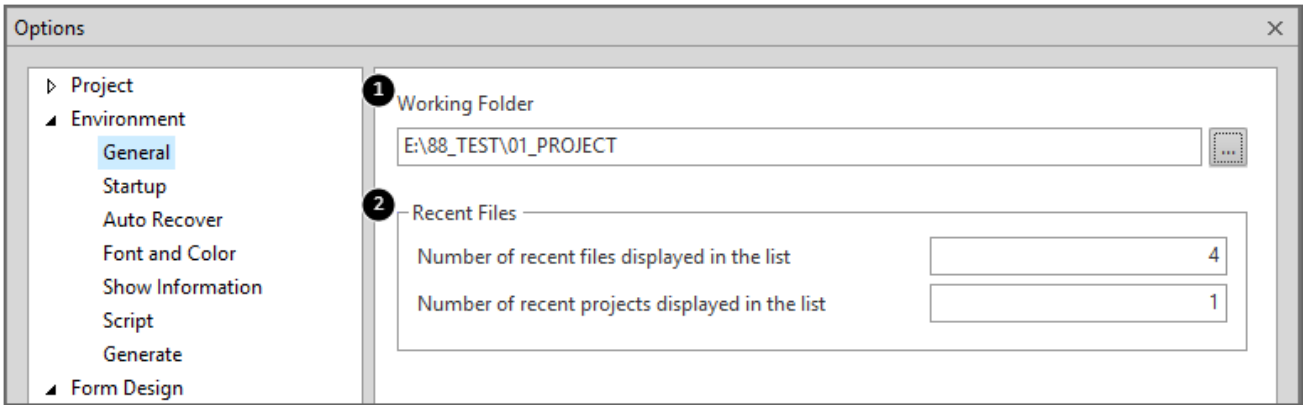
	Property	Description
1	Merge	(1) Set whether to merge the to-be-distributed list of Javascript files registered in the JSON module file into one file.
2	Compress	(2) Remove spaces and notes. (3) Obfuscate the JavaScript files. (4) Obfuscate regardless of the eval function (this is not a recommended option).

11.2 Environment

It sets the default environment of nexacro Module Developer.

11.2.1 General

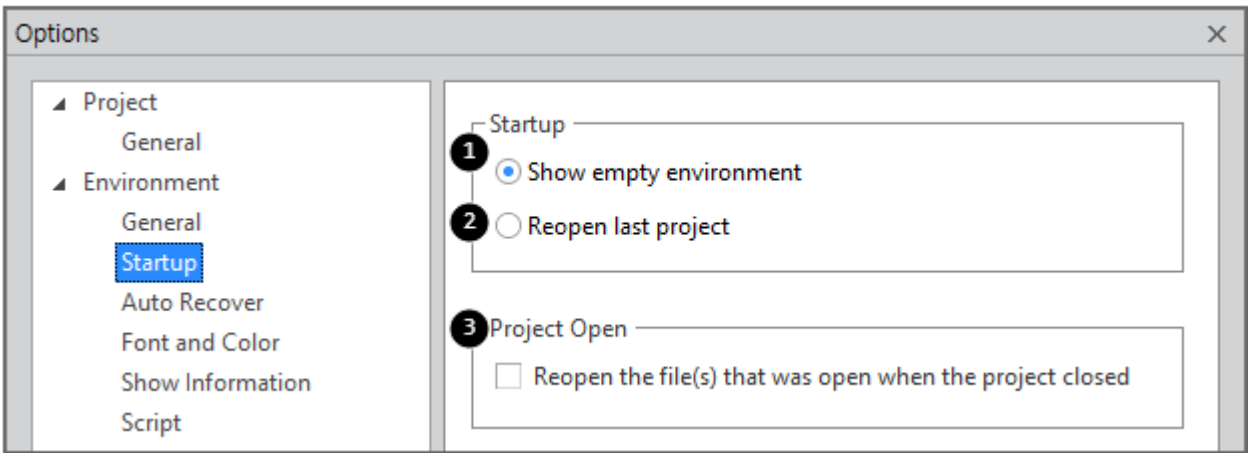
It sets the default environment property values such as the default folder in which the project is generated or the number of the recent task list.



	Property	Description
1	Working Folder	It specifies the folder to save the project when generating a new project.
2	Number of recent files displayed in the list	It sets the number of file lists displayed in the menu [File > Recent Files]. It can set up to 16.
3	Number of recent projects displayed in the list	It sets the number of project lists displayed in the menu [File > Recent Projects]. It can set up to 16.

11.2.2 Startup

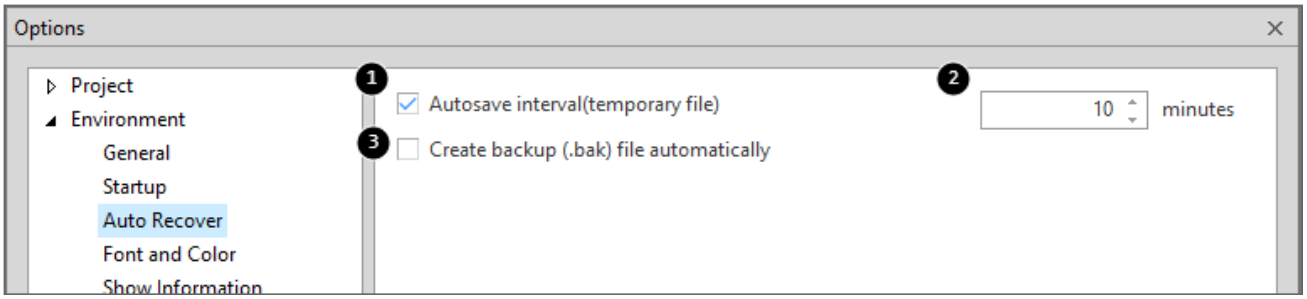
It sets the operating environment when running nexacro Module Developer.



	Property	Description
1	Show empty environment	It starts with a blank screen when starting.
2	Reopen last project	It automatically opens the last project you worked on when starting.
3	Reopen the file(s)...	It gives an option to open the last opened file.

11.2.3 Auto Recover

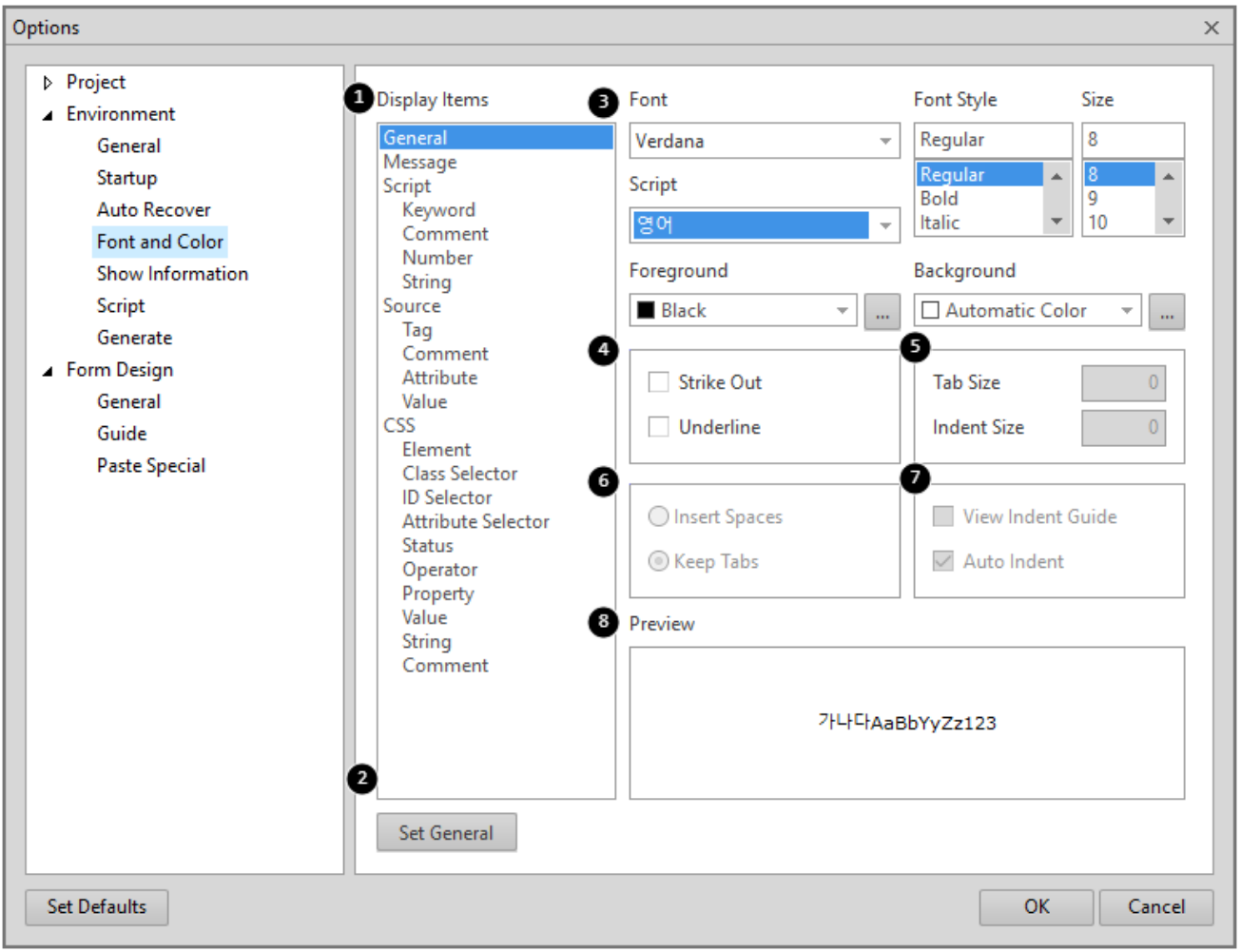
It sets the automatic recovery option when editing files.



	Option	Description
❶	Autosave interval (temporary file)	If checked, a temporary file will be created.
❷		Specifies the autosave interval
❸	Create backup (.bak) file automatically	Creates a backup file

11.2.4 Font and Color

It sets Font and Color used in each window.

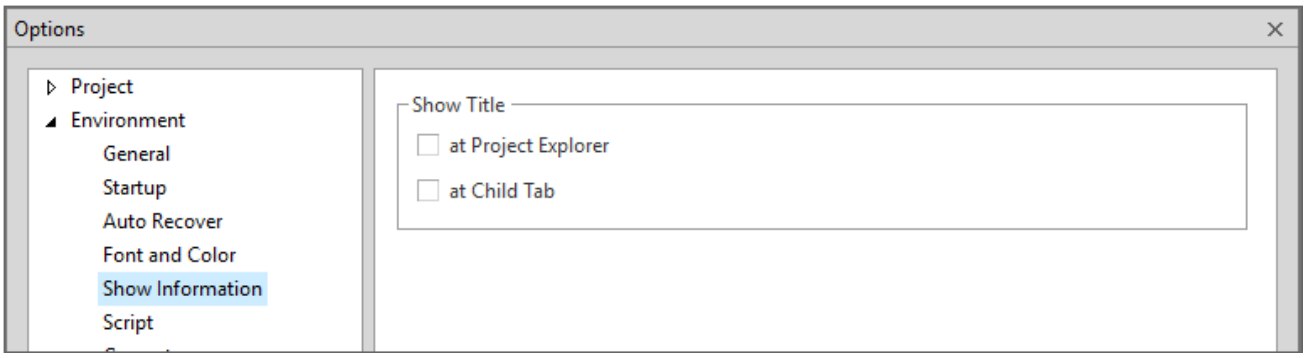


	Option	Description
1	Display items	Selects the items whose display font and color are being changed
2	Set General	The value set as the 'General' item value in Display Items is reflected in the rest of the Display Items. It can be used when you want to set the basic Font, Color style and directly modify the rest of the Display Items. For example, if you change the Size value of the 'General' item to 9 and click the 'Set General' button, the Size values of all other items are changed to 9.
3	Font	Select font
	Font Style	Sets font style
	Size	Sets font size
	Script	Sets the language scripts that can be used in the selected font
	Foreground	Sets the font color
	Background	Sets the font background color
4	Strike Out	If checked, causes text to be displayed in "strike through"
	Underline	If checked, causes text to be underlined

	Option	Description
5	Tab Size	Sets the tab size
	Indent Size	Sets the indent size
6	Insert Spaces	When selected, causes tabs to be converted to spaces
	Keep Tabs	When selected, causes tabs to be maintained as tabs
7	View Indentation Guide	When selected, caused the indentation guide to be displayed
	Auto Indent	When selected, enables automatic indenting
8	Preview	Displays sample text formatted as specified by the options selected

11.2.5 Show Information

It sets the option of whether to display the tile of the XCDL file. If no option is set, only the file name is displayed.

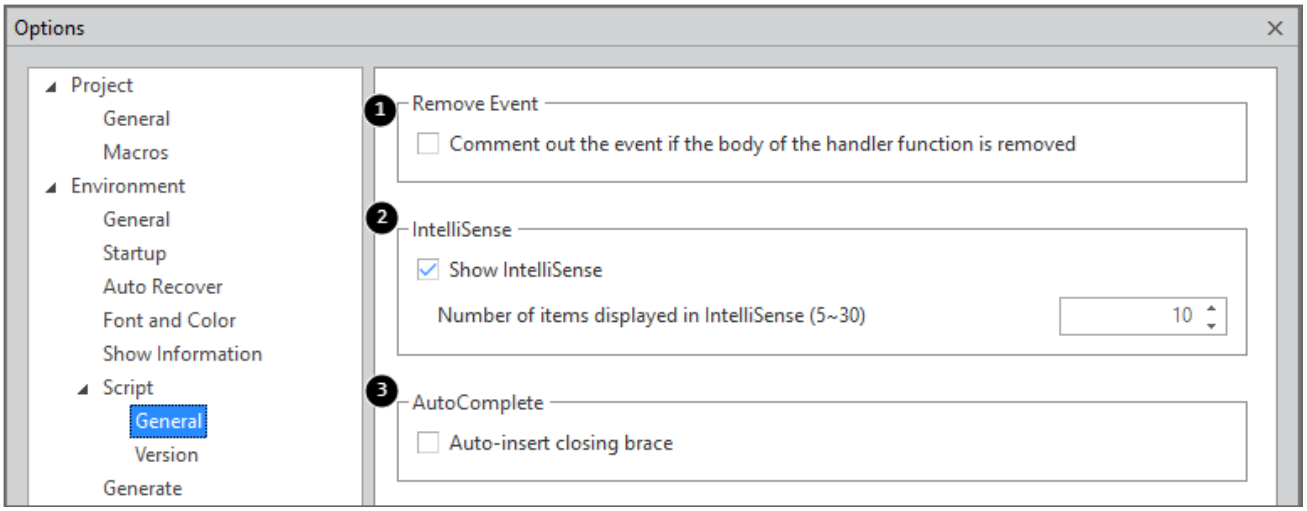


	Property	Description
	at Project Explorer	It displays the property value of the Form titletext of the XCDL file in Project Explorer.
	at Child Tab	It displays the property value of Form titletext XCDL of the XCDL file in Tab of the edit screen.

11.2.6 Script

It sets the Option used in the Script editing screen.

General

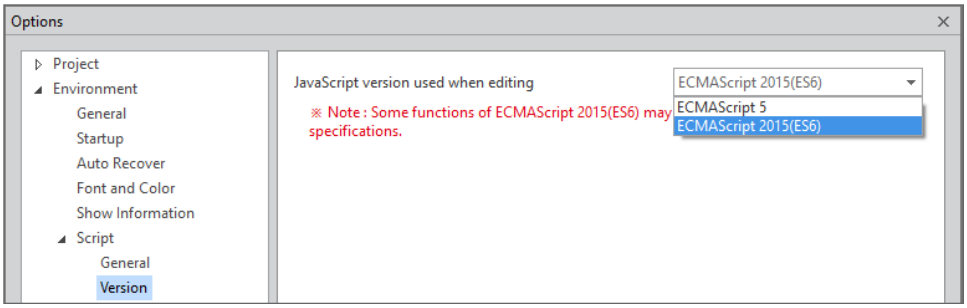


	Option	Description
1	Remove Event	Determines whether to comment out an event function when the relevant event is removed through the Properties pane
2	IntelliSense	Determines whether to display IntelliSense Sets up the number of items displayed in the IntelliSense list
3	Auto Complate	Sets up whether to insert a closing brace automatically

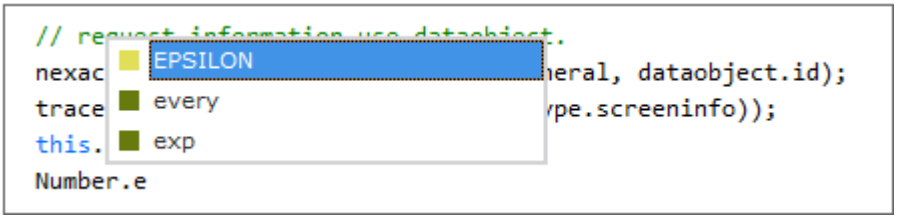
Version

Select the JavaScript version to use when writing the script. Depending on the version selected, the items supported by Script IntelliSense will change, and the script verification rules are applied differently for Generate.

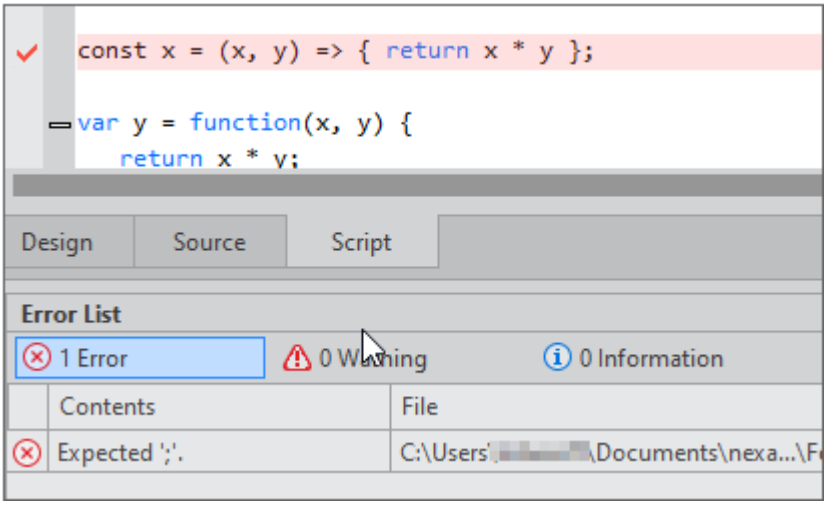
If you select "ECMAScript 2015(ES6)" as the Version option, then a script error may occur depending on the execution environment. You need to write the code in consideration of the user environment.
For example, when running the ES6 code in the IE11 browser, a syntax error may occur and the screen may not be displayed.



This is the IntelliSense screen displayed when "ECMAScript 2015(ES6)" is selected as the Version option and "Number.e" is entered in the script edit window. The "Number.EPSILON" property, which is supported from ES6, is additionally displayed.

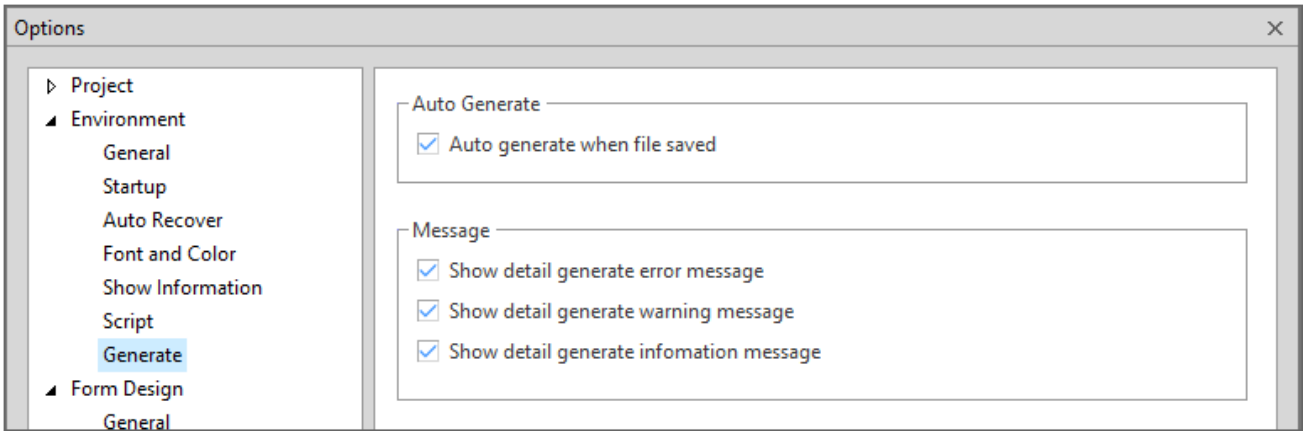


If you select "ECMAScript 5" as the Version option and use ES6 syntax in the script edit window, then it will be processed as a script error, and Generate will not be processed normally.



11.2.7 Generate

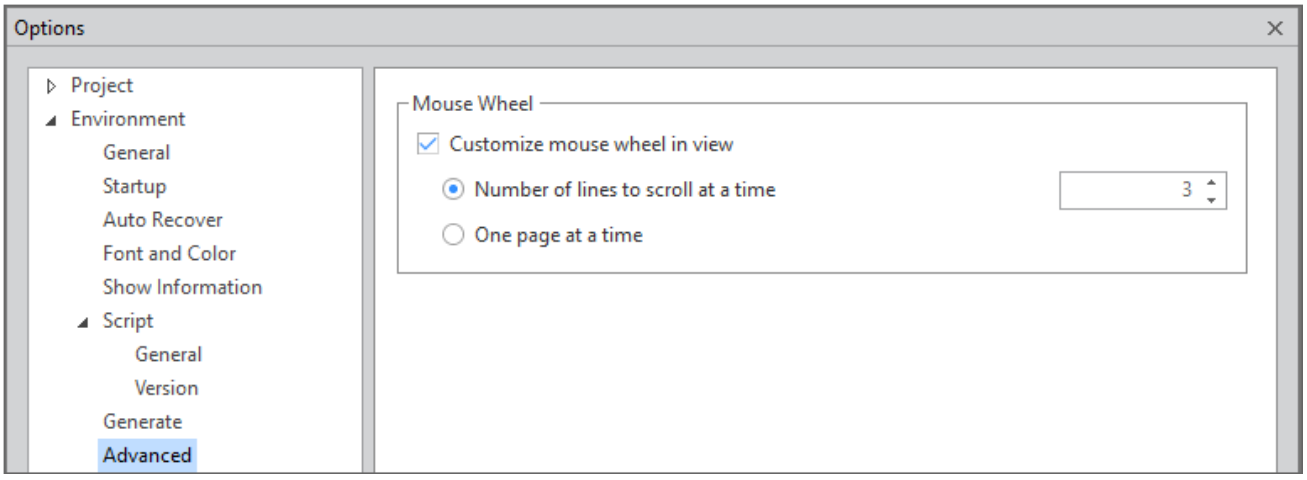
It sets the options related to running the Generate operation.



		Description
1	Auto Generate	Determines whether to conduct generation automatically when you save a file
2	Message	Selects the types of messages that will be printed in the generation process

11.2.8 Advanced

Sets the mouse wheel operation option.



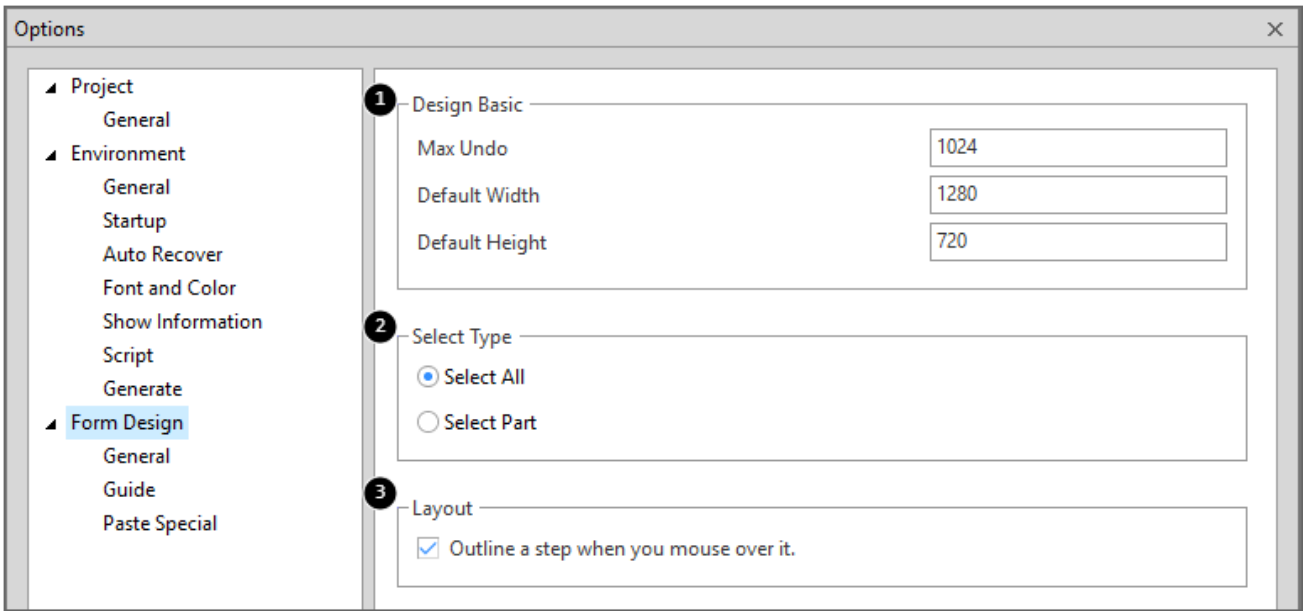
Property	Description
Customize mouse wheel in view	Checks whether to apply the mouse wheel operation option in the text editor. Checks whether to apply the mouse wheel motion option in the text editor. This option is applied only within Nexacro Studio apart from the Windows setting.
Number of lines to scroll at a time	Sets the number of lines to be scrolled at once for the mouse wheel scroll operation.
One page at a time	Moves as much as the number of currently displayed screen lines for the mouse wheel scroll operation. It may vary depending on the font size or window size.

11.3 Form Design

It sets the options related to Screen design.

11.3.1 General

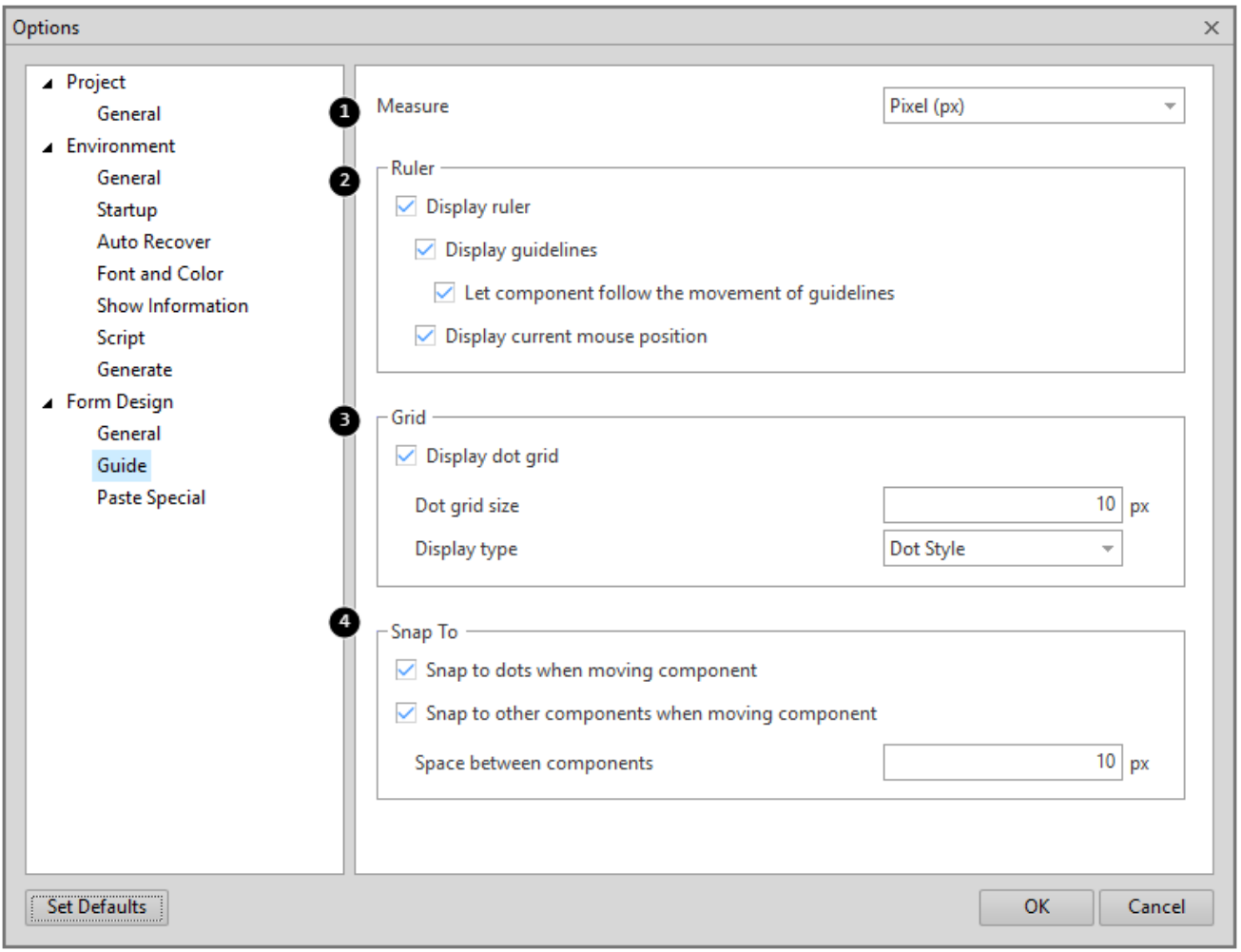
It sets the options related to Form Design.



	Option	Description
1	Max Undo	The maximum number of changes than can be undone
	Default Width	The default width when creating a new Form (0~12000)
	Default Height	The default height when creating a new Form (0~12000)
2	Select Type	The way of dragging a rectangle on the Form design panel to select components. Go to Selection Change for the details.
3	Layout	Displays the currently-edited step

11.3.2 Guide

It sets the options for the ruler, guideline, Grid, and Snap features of the Form design screen.



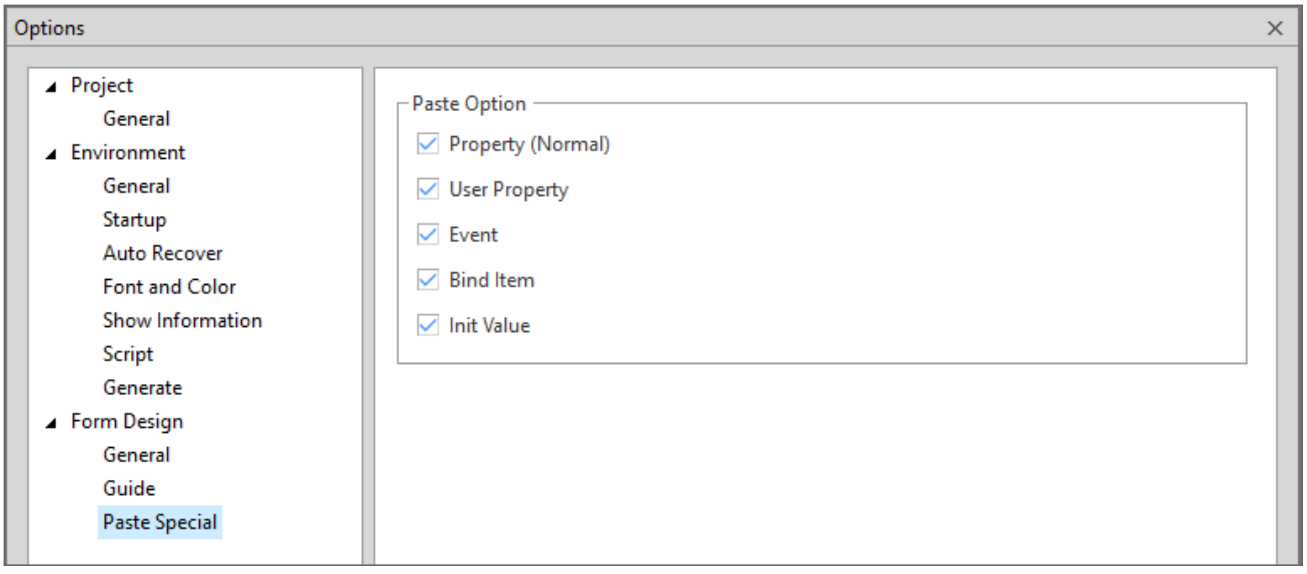
	Option	Description
1	Measure	Determines the types of unit for the position on the design window
2	Display ruler	When checked, the ruler will be displayed on the Design window canvas
	Display guidelines	When checked, guidelines will be displayed on the Design window canvas
	Let component follow the movement of guidelines	When checked, components will follow the movement of a guideline if they are set to align to the guideline
	Display current mouse position	When checked, indicators will appear on the horizontal and vertical rulers, showing the current coordinate of a mouse pointer
3	Display dot grid	When checked, the dot grid will be displayed on the canvas of the Design window
	Dot grid size	The dot grid size to be displayed on the canvas of the Design window (2~12000)
	Display type	Determines the types of dot displayed on the canvas of the design window
4	Snap to dots when moving Component	When checked, the magnetic function will be enabled and

	Option	Description
		will align controls with dots when moving them on the canvas
	Snap to other components when moving component	When checked, the magnetic function will be enabled and will align controls with other components when moving controls on the canvas
	Space between components	Determines the gap width between a control and components, applied when the magnetic function is implemented for the moving control

When you select a component and move the component using an arrow key, the movement will be made as far as the pixel number specified in the *Dot Grid Size* property. For example, if you set *Dot Grid Size* at 8, you will move a component 8 pixels every time you press an arrow button. You can make an 1-pixel movement if you hit an arrow key with the Ctrl key pressed.

11.3.3 Paste Special

It sets the option applied to the ‘Paste Special’ feature supported in Form Design.



Option	Description
Property (Normal)	Paste general properties
User Property	Paste user properties
Event	Paste event(s)
Bind Item	Paste bind item(s)
Init Value	Paste InitValue information

12.

Toolbar & Menu Bar

12.1 Toolbar Feature

Various features supported in nexacro Module Developer can be used directly from the toolbar.

12.1.1 Standard

It is a toolbar that supports the basic features such as opening, saving, copying, and pasting a project.



Menu		Feature
	Open Project	Opening Project
	Open	Opening a file in an editable format in nexacro Module Developer
	New	Generating a new project or adding a object
	Save	Saving the currently open document
	Save All	Saving all currently open documents
	Cut	Cutting the selected area and saving it to the clipboard
	Copy	Copying the selected area and saving it to the clipboard
	Paste	Pasting the saved content to the clipboard
	Options	Displaying option window
	About	Entering product information and license

12.1.2 Align

It is a toolbar that supports component alignment and placement features for screen development.



Menu Option		Description
	Align Lefts	Align to the left value of the component that was selected last
	Align Centers	Align to the horizontal center value of the component that was selected last
	Align Rights	Align to the right value of the component that was selected last
	Align Tops	Align to the top value of the component that was selected last
	Align Middles	Align to the vertical center value of the component that was selected last
	Align Bottoms	Align to the bottom value of the component that was selected last
	Same Width	Align to the width value of the component that was selected last
	Same Height	Align to the height value of the component that was selected last
	Same Size	Align to the size of the component that was selected last
	Distribute Horizontally	Distribute components with same horizontal space between them. Rearrange components with equal space.
	Distribute Vertically	Distribute components with same vertical space between them. Rearrange components with equal space.
	Distribute Horizontally by Specified Value	Distribute components with same horizontal space between them.
	Distribute Vertically by Specified Value	Distribute components with same vertical spaces in between.
	Position Center	Move selected components to the horizontal center of Form Canvas
	Position Middle	Move selected components to the vertical center of Form Canvas
	Position Left	Move selected components to the left side of Form Canvas
	Position Right	Move selected components to the right side of Form Canvas
	Position Top	Move selected components to the top of Form Canvas
	Position Bottom	Move selected components to the bottom of Form Canvas
	Fit to Contents	□□□ Component□□ □□□□ min, max □□□□ □□ □□□ □□□□□. Resize selected components so that they fit into contents, min- properties or max- properties
	Bring to Front	Bring selected component to the front
	Send to Back	Send selected component to the back
	Bring Forward	Bring selected component forward
	Send Backward	Send selected component back

Menu Option		Description
	Arrange to Tab Order	The order that components appear in the source codes is rearranged according to tab order

12.1.3 Bookmark

It is a toolbar that supports the Bookmark that allows user to easily move to a specific line while editing.



Menu Option		Description
	Toggle Bookmark	Sets or clears the bookmark at the current cursor location
	Previous Bookmark	Moves cursor to the previous bookmark location
	Next Bookmark	Moves cursor to the next bookmark location
	Delete All Bookmarks	Clears all bookmarks
	Previous Bookmark in Document	Moves cursor to the previous bookmark location in the current the current document
	Next Bookmark in Document	Moves cursor to the next bookmark location in the current the current document
	Delete All Bookmarks in Document	Clears all bookmarks in the current document
	Go to Bookmark	Moves the cursor to the line of a selected bookmark
	Enable/Disable Bookmark	Includes/excludes all the selected bookmarks when you are searching
	Enable/Disable All Bookmarks	Includes/excludes all the bookmarks when you are searching

12.1.4 Build

It is a toolbar that converts a project or file to a JavaScript file or supports the emulator operating feature.



Menu		Feature
	Generate Module	It converts the entire project to a JavaScript file. The converted file is saved in

Menu		Feature
		the folder specified in the options.
	Generate File	It converts the file being edited to a JavaScript file.
	Stop Generate	It stops the conversion operation in process.
	Emulate	It runs the selected object in the emulator.

12.1.5 Component

It is a toolbar that supports the feature of selecting an object registered in ModuleDefinition and placing or registering it on the screen.



The items displayed on the Component toolbar may look different depending on the components registered in the ModuleDefinition Objects list.

Selecting and moving components

Icon	Name	Description
	Select	Selects components by clicking and dragging with a mouse
	Move	Moves the position of the canvas area, which is present on the Form design tab

Basic components

Icon	Component Name	Description
	Button	Used to place a button control on a Form
	Calendar	Used to allow a user to enter a date by selecting from a calendar
	CheckBox	Used to place a checkbox (true/false) control on a Form
	Combo	Used to place a combo (dropdown) control on a Form
	Dataset	Saves data in tabular form
	Div	Used to include one or more Forms within another or when handling related components in one group
	Edit	Used to place an Edit (editable text box) control on a Form
	Grid	Used to display the contents of a Dataset in tabular form

Icon	Component Name	Description
	GroupBox	Groups related components
	ImageViewer	Used to display images on a Form
	ListBox	Used to place a listbox (list of items) control on a Form
	MaskEdit	Used to place a MaskEdit control on a Form. MaskEdit allows control of data formatting for items such as dates, times, phone numbers, and long numeric values
	Menu	Used to construct menu items
	Plugin	Used when using a plugin
	PopupDiv	Used when constructing several sections in one screen
	PopupMenu	Used to construct a popup menu
	ProgressBar	Used to place a graphic representing the status of a process on a Form
	Radio	Used to place a radio button (pick one item of many) control on a Form
	Spin	Used to allow users to enter data using a "spinner" control - a control that allows the user to increment or decrement the value in the fiedl
	Static	Used to place a static (fixed text) control on a Form
	Tab	Used to construct a page with multiple, tabbed sections
	TextArea	Used to place TextArea (multi-line text box) control on a Form

DeviceAPI

Icon	Name	Description
	Acceleration	Provides the information on the acceleration recorded by a mobile device.
	AudioPlayer	Plays an audio file using a mobile device
	AudioRecorder	Records audio data and save it as a file using a mobile device
	Camera	Takes a picture using the camera of a mobile device and then processes the image or image path
	Contact	Processes the contact data of a mobile device
	ContactAddress	Object of the address field of the contact data
	ContactField	Object of the note field of the contact data
	ContactOrganization	Object of the organization field of the contact data
	ContactPhoto	Object of the photo field of the contact data
	ContactSet	Combines all the fields of the contact data
	ExteralAPI	Operates an external device (otherwise, external module or app)
	FileDialog	Launches a file dialog for users to open a file or select a directory to save a file
	Geolocation	Provides the information on the location of a mobile device

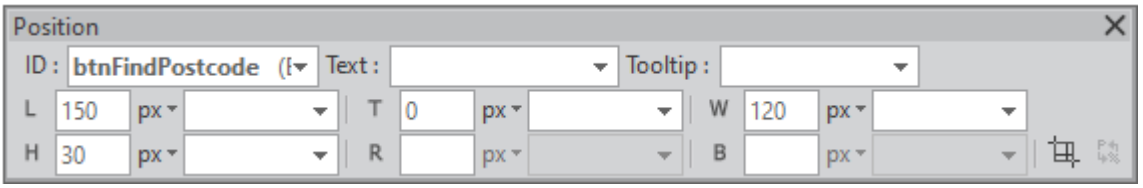
Icon	Name	Description
	ImagePicker	Connects to the photo album of a mobile device and processes an image or image path.
	LiteDBConnection	Processes the connection to a database supported by a mobile device
	LiteDBStatement	Object to pass a query
	Network	Provides the information on the network connection of a mobile device.
	Sms	Object for SMS service, thus sending and receiving SMSes
	Vibrator	Uses the vibration function of a mobile device

Other objects

Icon	Name	Description
	ExcelExportObject	Exports the contents of a Grid in the form of a spreadsheet file
	ExcelImportObject	Imports a spreadsheet file and converts it into a Dataset object
	FileDownload	Downloads files
	FileUpload	Uploads files
	GoogleMap	Prints a map where designated locations are marked or allows users to mark certain locations on a map
	VirtualFile	Virtual file object to read or use a file in a local system
	Sketch	Canvas where users can draw a simple figure, signature, etc.
	WebBrowser	Wrapper object of a web browser
	XPush	Object to process the messages from an X-PUSH server

12.1.6 Position

It is a toolbar that supports editing of the component Text, Tooltip, and Position property values.



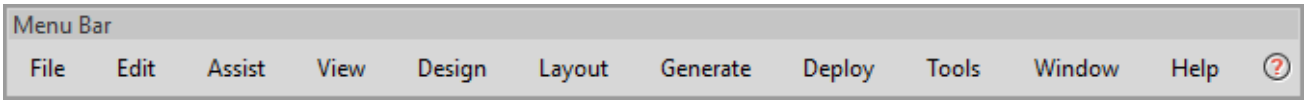
Menu	Description
ID	Select an ID of a component
Text	Enter a text of a component
Tooltip	Enter a tooltip text of a component
Left (L)	Enter a left-hand position value of a component

Menu	Description
Top (T)	Enter a top position value of a component
Width (W)	Enter a width value of a component
Height (H)	Enter a height value of a component
Right (R)	Enter a right-hand position value of a component
Bottom (B)	Enter a bottom position value of a component
Position Editor	Launch Position Editor for editing position properties of the selected component
Position Units	Change the default position unit used when no component is selected

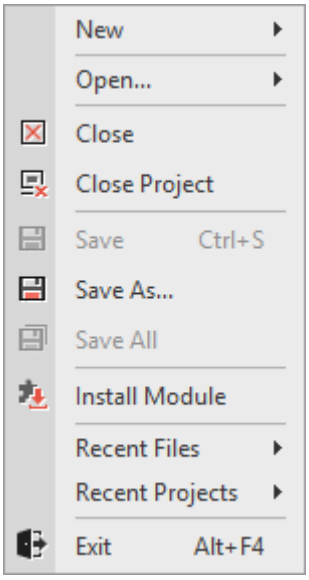
If a certain property is read-only, such as *text* of an Edit component, the relevant item on the toolbar will be disabled.

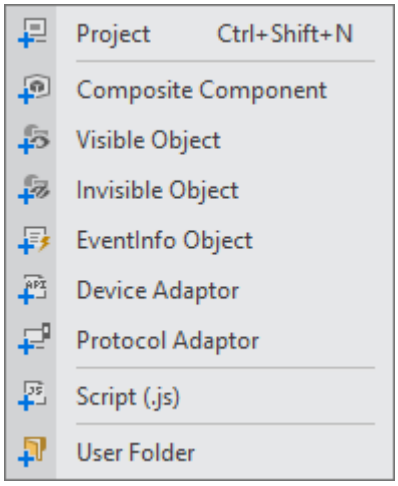
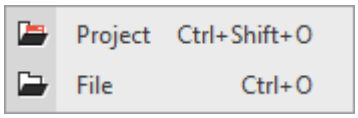
12.2 Menu Bar

You can select and run the basic features provided in nexacro Module Developer.

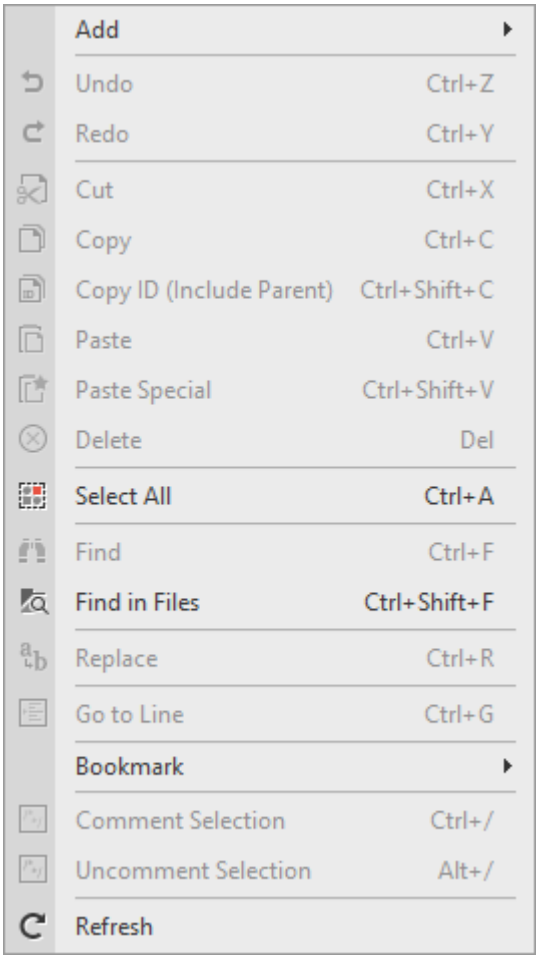


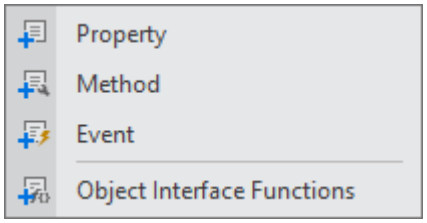
12.2.1 File



Menu Option	Description
New	 • Project - Generating a new project • Object creation – Refer to the Adding Object item • Script - Create a new script • User Folder - Generating a new folder
Open	 • Project - Opening a new module project • File - Open individual files
Close	Closing the currently open window
Close Project	Closing the currently open project
Save	Saving the currently active editing screen
Save As	Saving the currently active editing screen with a different name
Save All	Saving all currently open editing screens and all changes made in Project Explorer
Install Module	Installing the module file generated in the module project
Recent Files	Displaying a list of recently opened files
Recent Projects	Displaying a list of recently opened projects
Exit	Closing nexacro Module Developer

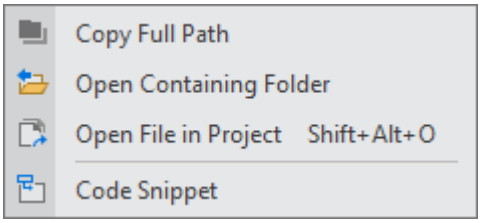
12.2.2 Edit



Menu Option	Description
Add	Select the object and add the property, method, and event items Object Interface Functions are enabled when the xcdl file is selected in the case of the Composite Component object, and other objects are enabled when the js file is selected. 
Undo	Returning to the previous state
Redo	Returning to the state before running Undo
Cut	Cutting the selected area and copying it to the clipboard
Copy	Copying the selected area to the clipboard
Copy ID	Copying the ID value of the selected area to the clipboard (It is activated only when selecting one component in design mode.)
Paste	Pasting content from the clipboard

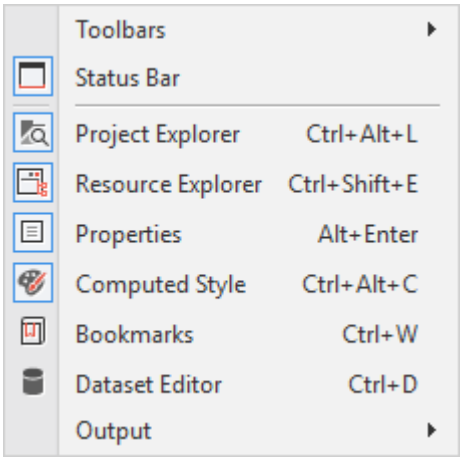
Menu Option	Description
Paste Special	The user selecting and pasting the component Property and Bind information in the clipboard
Delete	Deleting the text in the selected component or area
Select All	Selecting all text in the Script window
Find…	Finding a string in the Script window
Find in File…	Finding a file with specified words among files in the specified path
Replace	Replacing a specific string with another string in the Script window
Goto Line…	Moving the cursor to a specific line in the Script window
Bookmark	The feature provided in the Bookmark menu is the same as that of the Toolbar [Bookmark Bar].
Comment Selection	Commenting out the string in the selected area in the Script window
UnComment Selection	Uncommenting the string in the selected area in the Script window
Refresh	Reloading the file

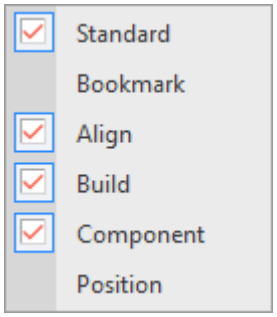
12.2.3 Assist

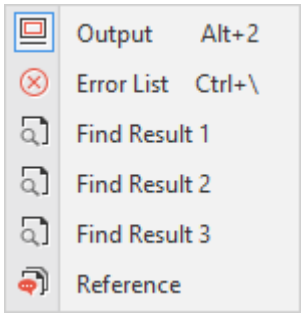


Menu Option	Description
Copy Full Path	Copies to clipboard the full path where selected files are saved
Open Containing Folder	Opens in Windows Explorer the folder containing the selected file
Open File in Project	Can search for files included in the project and execute them
Code Snippet	Opens the Code Snippet Editor

12.2.4 View

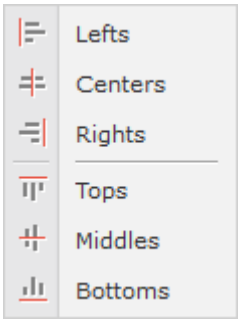
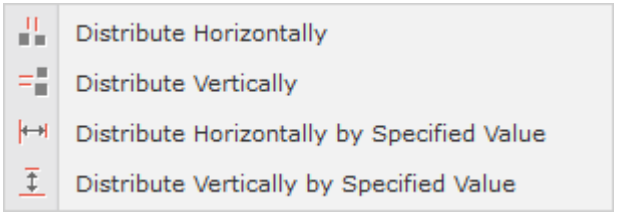


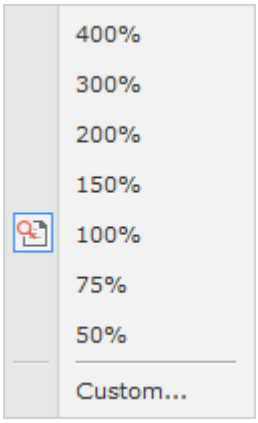
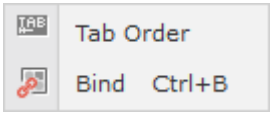
Menu	Feature
Toolbars	 • Standard - Showing/hiding Standard Bar • Bookmark - Showing/hiding Bookmark Bar • Align - Showing/hiding Align Bar • Build - Showing/hiding Build Bar • Component - Showing/hiding Component Bar • Position - Showing/hiding Position Bar
Statusbar	Showing/hiding Statusbar
Project Explorer	Displaying the Project Explorer window When the Project Explorer window is open, only the focus moves.
Resource Explorer	Displaying the resource items contained in the project
Properties	Displaying the property window When the property window is open, only the focus moves.
Computed Style	Displaying the Computed Style tab in the property window
Bookmarks	Displaying the Bookmarks window
Dataset Editor	Displaying the Dataset Editor window
Output	

Menu	Feature
	 • Output - Showing the Output window • Error List - Showing the Error List window • Find Result 1,2,3 - Showing the Find Result 1,2, and 3 windows • Reference - Showing the Reference window

12.2.5 Design

INCLUDE_ERROR(ELEMENT_REMOVED)

Menu Option	Function
Align	
Space	
Size	
Position	

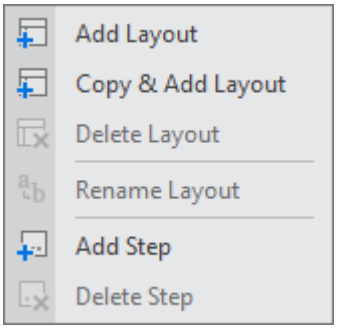
Menu Option	Function
	
Fit to Contents	Resizes selected components so that they fit into contents, min- properties or max-properties
Arrange	
Arrange to Tab Order	The order that components appear in the source codes is rearranged according to tab order
Lock Components	Locks (or unlocks) a component to prevent it from being moved by mouse
Zoom	 Enlarges or shrinks the current view
Hotkey Editor	Edits a list of components for which hotkeys are specified
Tab Order Editor (View Type)	Begins the tab order editing mode
Tab Order Editor (List Type)	Opens the Tab Order list window so they you can edit the order through it
State View	 Tab Order: Displays Tab order of component continuously on the Form design pane Bind: Displays bind status of component
Position Editor	Launches Position Editor for editing position properties of the selected component
Position Units	Changes the default position unit used when no component is selected
Show Invisible Object Area	Displays the Invisible Object pane when it is hidden
Show Binding Components	Displays the Binding Components List pane when it is hidden

Menu Option	Function
List	



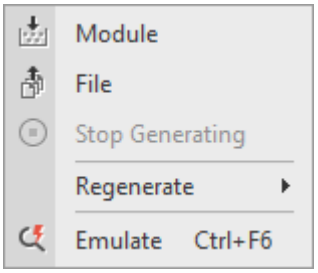
The features related to Align, Space, Size, Position and Arrange correspond to the features provided by their respective toolbars.
Please, refer to Align.

12.2.6 Layout



Name	Description
Add Layout	Adds a layout
Copy & Add Layout	Copies the selected layout and adds it as a new layout
Delete Layout	Deletes the selected layout
Rename Layout	Renames the selected layout
Add Step	Adds a step
Delete Step	Deletes a step

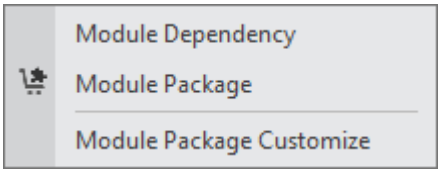
12.2.7 Generate



Menu	Feature

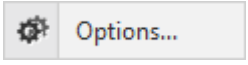
Menu	Feature
Module	It converts the entire project to a JavaScript file. The converted file is saved in the folder specified in the options.
File	It converts the file being edited to a JavaScript file.
Stop Generate	It stops the conversion operation in process.
Emulate	It runs the selected object in the emulator.

12.2.8 Deploy



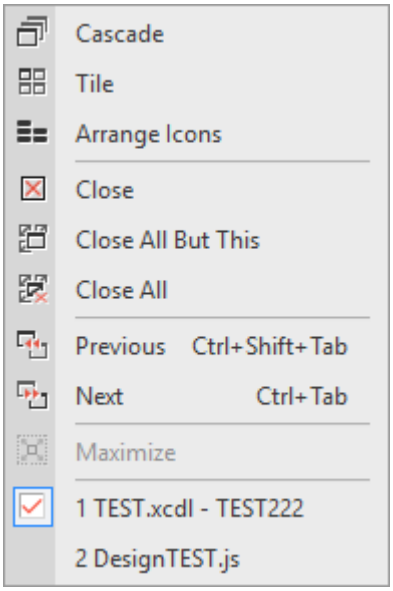
Menu	Feature
Module Dependency	Setting the order of adding objects in the deployment file when more than one object is added
Module Package	Generating the xmodule file by compressing the project in a format that can be deployed
Module Package Customzie	Changing the JSON configuration and generating the xmodule file when deploying the module It can be run even when the project is not open.

12.2.9 Tools



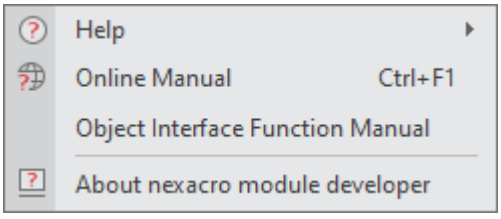
Menu	Feature
Options	Opening the Options setting window

12.2.10 Window



Menu	Feature
Cascade	Cascading the window being edited
Tile	Tiling the window being edited
Arrange Icons	Arranging the minimized icons
Close	Closing the selected edit window
Close All But This	Closing all screens except the currently open screen
Close All	Closing all edit window
Next	Moving to the next edit window
Previous	Moving to the previous edit window
Maximize	Maximizing the edit window

12.2.11 Help



Menu	Feature
Help	Displaying the help window.
Online Manual	Running the online manual in a web browser http://docs.tobesoft.com/nexacro_n_en
Object Interface Function Manual	Run the object interface function help provided in a separate JSDoc format.
About nexacro module developer	Displaying the nexacro Module Developer information window

Appendix A.

Composite Component Example

Here is an introduction to the composite component example created in Nexacro Module Developer. This is an example code, and its suitability has not been verified.

A.1 Postal Code Search

Here is an example of using the Kakao Postal Code Search API. Enter the searched results in the Edit component.

> Refer to the link below for details about the Postal Code Search API.
> https://postcode.map.kakao.com/guide

1. Create a new object using the Composite Component. Set the object id to "KakaoPostCode."
2. In the Design tab, structure the screen as follows.



	Component	id	Other Properties
1	Edit	editPostcode	displaynulltext: □□□□ readonly: true

	Component	id	Other Properties
2	Edit	editAddress	displaynulltext: □□□□□ readonly: true
3	Edit	editDetailAddress	displaynulltext: □□□□
4	Button	btnFindPostcode	text: □□□□ □□

3 In the Class Definition tab, modify the constructor code as follows.

Execution in the NRE has not been implemented to simplify the sample implementation. This ensures that the necessary API scripts are handled when the constructor is called.

```
if (!nexacro.KakaoPostCode)
{
    nexacro.KakaoPostCode = function (id, left, top, width, height, right,
bottom, minwidth, maxwidth, minheight, maxheight, parent)
    {
        nexacro._CompositeComponent.call(this, id, left, top, width,
height, right, bottom, minwidth, maxwidth, minheight, maxheight, parent);
        if (!system.navigatorname.startsWith("nexacro"))
        {
            nexacro.load_kakaopostcode();
        }
        else
        {
            trace("KakaoPostCode composite component supports only the web
browsers");
        }
    }
    nexacro.KakaoPostCode.prototype =
nexacro._createPrototype(nexacro._CompositeComponent,
nexacro.KakaoPostCode);
    nexacro.KakaoPostCode.prototype._type_name = "KakaoPostCode";
}
```

4 Create the load_kakaopostcode function that initially loads the API script above the constructor code in the Class Definition tab.

This approach dynamically inserts the script, and if multiple components are placed, the code should be inserted only once.

```
if (!nexacro.load_kakaopostcode)
{
    nexacro.kakaopostcode_loaded = false;
    nexacro.load_kakaopostcode = function()
    {
        if (nexacro.kakaopostcode_loaded)
        {
            return;
        }
        nexacro.kakaopostcode_loaded = true;
        var script = document.createElement("script");
        script.type = "text/javascript";
        script.src =
"//t1.kakaocdn.net/mapjsapi/bundle/postcode/prod/postcode.v2.js";
        document.getElementsByTagName("head")[0].appendChild(script);
    };
}
```

5 Add three properties, postcode, address, and detailAddress, to make it easy for users to check the search results.

Modify the code generated with the `postcode` property as follows. Modify the other properties in the same way.

```
nexacro.KakaoPostCode.prototype._p_postcode = "";
nexacro.KakaoPostCode.prototype.set_postcode = function (v)
{
    if(this.form.editPostcode)
    {
        this.form.editPostcode.value = v;
    }
    this._p_postcode = v;
};

Object.defineProperty(nexacro.KakaoPostCode.prototype, "postcode", {
        get: new Function("return this._p_postcode"),
        set: nexacro.KakaoPostCode.prototype["set_postcode"],
        enumerable : true,
        configurable : true});

nexacro.KakaoPostCode.prototype._p_address = "";
```

```
nexacro.KakaoPostCode.prototype.set_address = function (v)
{
    if(this.form.editAddress)
    {
        this.form.editAddress.value = v;
    }
    this._p_address = v;
};

Object.defineProperty(nexacro.KakaoPostCode.prototype, "address", {
        get: new Function("return this._p_address"),
        set: nexacro.KakaoPostCode.prototype["set_address"],
        enumerable : true,
        configurable : true});

nexacro.KakaoPostCode.prototype._p_detailAddress = "";
nexacro.KakaoPostCode.prototype.set_detailAddress = function (v)
{
    if(this.form.editDetailAddress)
    {
        this.form.editDetailAddress.value = v;
    }
    this._p_detailAddress = v;
};

Object.defineProperty(nexacro.KakaoPostCode.prototype, "detailAddress", {
        get: new Function("return this._p_detailAddress"),
        set: nexacro.KakaoPostCode.prototype["set_detailAddress"],
        enumerable : true,
        configurable : true});
```

6 Write the script to display the search window when the Postal Code Search Button component is clicked, as shown below.

The actual returned data contains various properties, but in the example, only the postal code and road name address values are checked.

```
var that = this;
this.btnFindPostcode_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
```

```
    if(kakao)
    {
        kakao.postcode.load(function(){
            new kakao.Postcode({
                oncomplete: function(data) {
                    that.parent.postcode = data.zonecode;
                    that.parent.address = data.roadAddress;
                }
            }).open();
        });
    }
};
```

7 Add an onchanged event function to the Edit component for entering the detailed address, so the value can be handled by the detailAddress property, as shown below.

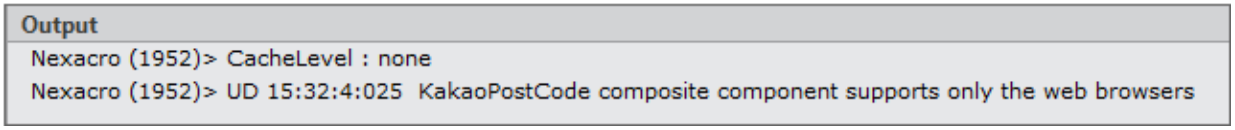
```
this.editDetailAddress_onchanged = function
(obj:nexacro.Edit,e:nexacro.ChangeEventInfo)
{
    this.parent.detailAddress = e.postvalue;
};
```

8 Deploy the module and install it in Nexacro Studio.

Execution in the NRE is not supported, so testing cannot be done in the emulator. Instead, verify functionality in a web browser after deployment.

9 Place the KakaoPostCode component on the screen and execute it in the NRE.

The component will be visible on the screen but will not function correctly. Messages are displayed in the Output window.

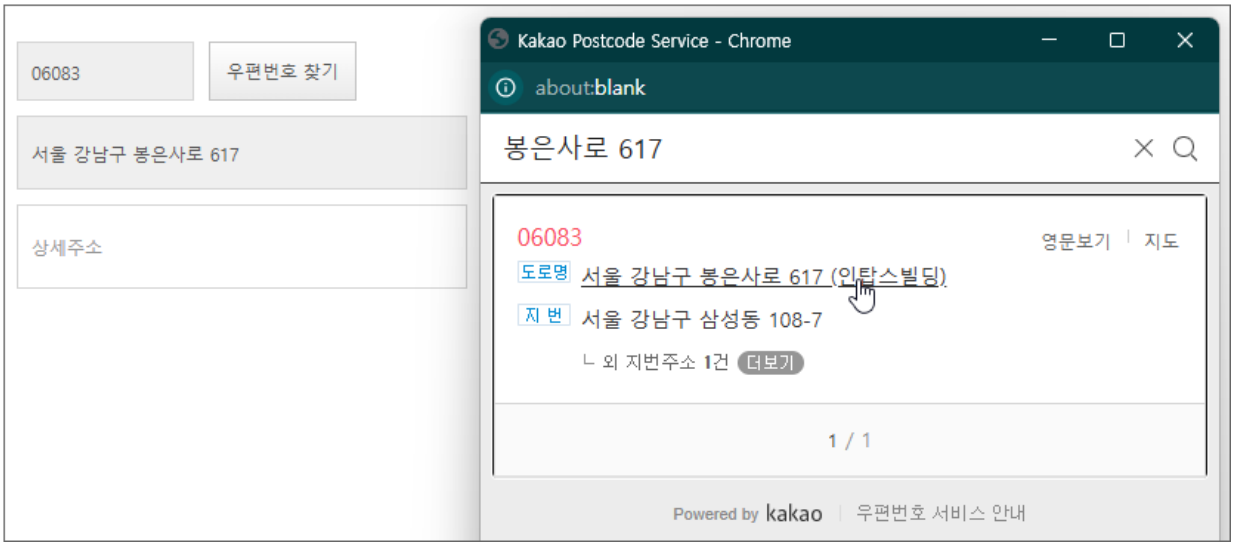


10 Add a Button component to the screen and write the onclick event as follows:

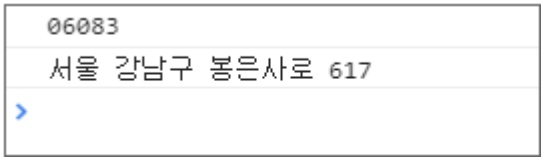
After searching for the postal code, verify whether the data has been processed in the corresponding properties.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    trace(this.KakaoPostCode00.postcode);
    trace(this.KakaoPostCode00.address);
};
```

11 After running in a web browser using QuickView (Ctrl + F6), clicking the “Postal Code Search" button opens a popup window where postal codes can be searched. Click the search result to insert the value into the Edit component.



12 Click the added button in the example to check the search result in the component's properties. Check the values output in the developer tools console tab.



A.2 Toggle Button

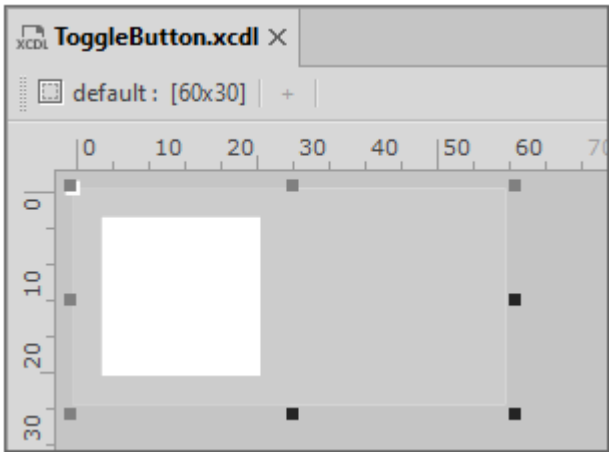
A toggle button is similar to a checkbox but is designed to represent the appearance of a physical switch. It is primarily used in mobile apps but is also commonly seen on the web.



The sample implemented in the example refers to the link below.
https://www.w3schools.com/howto/tryit.asp?filename=tryhow_css_switch

1. Create a new object using the Composite Component. Set the object id to ToggleButton.

2. In the Design tab, structure the screen as follows.



	Component	id	Other Properties
1	Static	staticToggleBox	left: 0 top: 0 width: 100% height: 100% background: #CCCCCC text: ""
2	Static	staticTogglebutton	left: 0 top: 0 width: 50% height: 100% background: #FFFFFF

	Component	id	Other Properties
			border: 4px solid #cccccc text: ""

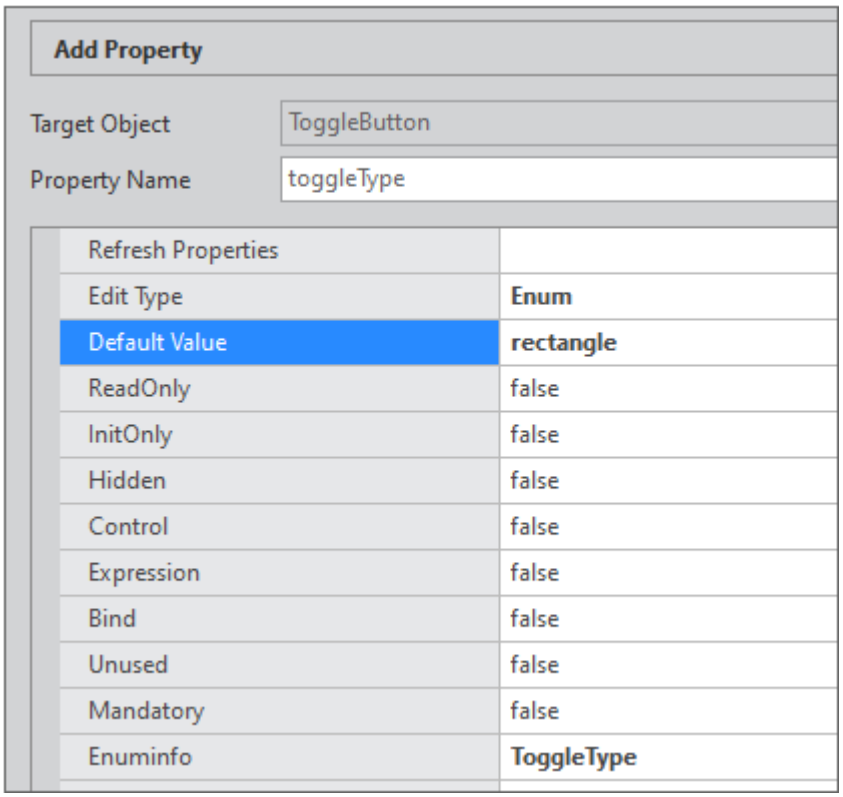
3 Add EnumInfo items to the toggleType property, which allows the appearance of the toggle button to be changed.

To use the two property values, "rectangle" and "rounded," in Enum format, first add the EnumInfo items as shown below.



4 Add the toggleType property.

Set the edittype item to “Enum” and the enuminfo item to the previously created “ToggleType”. Set the defaultvalue item to “rectangle”.



5 Modify the automatically generated script code related to the toggleType property as follows.

Adjust the borderRadius property of the Static component based on the selected property value to change its appearance.

```
nexacro.ToggleButton.prototype._p_toggleType = "rectangle";
nexacro.ToggleButton.prototype.set_toggleType = function (v)
{
    this._p_toggleType = v;
    var togglebackground = this.form.staticToggleBox;
    var togglebutton = this.form.staticTogglebutton;
    if(togglebackground) {
        if(v=="rounded")
        {
            togglebackground.borderRadius = this.form.height+"px";
            togglebutton.borderRadius = "50%";
        }
        else
        {
            togglebackground.borderRadius = undefined;
            togglebutton.borderRadius = undefined;
        }
    }
};
```

6 Add two properties, toggleOnColor and toggleOffColor, to specify colors, and modify the script code as shown below.

```
nexacro.ToggleButton.prototype._p_toggleOnColor = "#2196F3";
nexacro.ToggleButton.prototype.set_toggleOnColor = function (v)
{
    this._p_toggleOnColor  = v;
    this._changecolor(this.value);
};

nexacro.ToggleButton.prototype._p_toggleOffColor= "#CCCCCC";
nexacro.ToggleButton.prototype.set_toggleOffColor = function (v)
{
    this._p_toggleOffColor= v;
    this._changecolor(this.value);
};
```

7 Add the value property and modify the script code as shown below. Set the Edit Type of the value property to “Boolean”.

```
nexacro.ToggleButton.prototype._p_value= false;
nexacro.ToggleButton.prototype.set_value = function (v)
{
    v = this._isChecked(v);
    if (this._p_value != v) {
        if (this.applyto_bindSource("value", v)) {
            this._setValue(v);
            this._changebutton(this.value);
        }
    }
};

nexacro.ToggleButton.prototype._setValue = function (v) {
    this._p_value = v;
};
```

8 Add the code to handle the component initialization when it is created.

In the Project Explorer, select the ToggleButton.xcdl file, add the on_create_contents item in the properties window, and modify the code as follows. Initialize the Animation object to apply animation effects when toggling the on and off states of the button upon clicking the component.

The _changecolor function is responsible for changing colors based on the value property. Since the default value cannot be applied immediately when the component initializes in the app, add the code to reapply it in the on_create_contents function.

```
nexacro.ToggleButton.prototype.on_create_contents = function ()
{
    nexacro._CompositeComponent.prototype.on_create_contents.call(this);
    var aniObj = new nexacro.Animation("aniToggleBtn", this);
    aniObj.easing = "linear";
    aniObj.duration = 300;
    this.form.addChild("aniToggleBtn", aniObj);
    this.form.aniToggleBtn.addTarget("AniItem00", this
.form.staticTogglebutton, "left:0");
    this._changecolor(this.value);
};

nexacro.ToggleButton.prototype._changecolor = function (value) {
```

```
    var backgroundvalue = this.toggleOffColor;
    var bordervalue = "4px solid "+this.toggleOffColor;
    var togglebackground = this.form.staticToggleBox;
    var togglebutton = this.form.staticTogglebutton;

    if(value) {
        backgroundvalue = this.toggleOnColor;
        bordervalue = "4px solid "+this.toggleOnColor;
    }
    if(togglebackground) {
        togglebackground.background = backgroundvalue;
        togglebutton.border = bordervalue;
    }
}
```

9 In the Design tab, select the Static component and write the onclick event handler function as follows.

When the Static component within the composite component is clicked, the onclick event of the parent component is handled.

```
this.staticToggleBox_onclick = function
(obj:nexacro.Static,e:nexacro.ClickEventInfo)
{
    this.parent.on_fire_onclick(obj, e.pretext, e.prevalue, e.posttext,
e.postvalue);
};

this.staticTogglebutton_onclick = function
(obj:nexacro.Static,e:nexacro.ClickEventInfo)
{
    this.parent.on_fire_onclick(obj, e.pretext, e.prevalue, e.posttext,
e.postvalue);
};
```

10 Return to the Class Definition tab and add the necessary functions within the on_fire_onclick function.

```
nexacro.ToggleButton.prototype.readonly = false;

nexacro.ToggleButton.prototype._isChecked = function (value) {
    return nexacro._toBoolean(value);
};
```

```
nexacro.ToggleButton.prototype.on_fire_canchange = function (obj, prevalue,
postvalue) {
    if (this.canchange && this.canchange._has_handlers)
    {
        var evt = new nexacro.CheckBoxChangedEventInfo(this, "canchange",
prevalue, postvalue);
        return this.canchange._fireCheckEvent(this, evt);
    }
    return true;
};

_pToggleButton._changebutton = function (postvalue) {
    var leftvalue = 0;
    this._changecolor(postvalue);
    if(postvalue) {
        leftvalue = this.form.width/2;
    }

    if(this.form.aniToggleBtn)
    {
        this.form.aniToggleBtn.items[0].props = "left:"+leftvalue;
        this.form.aniToggleBtn.play();
    }
}

nexacro.ToggleButton.prototype.on_fire_onchanged = function (obj, prevalue,
postvalue) {
    if (this.onchanged && this.onchanged._has_handlers)
    {
        var evt = new nexacro.CheckBoxChangedEventInfo(this, "onchanged",
prevalue, postvalue);
        return this.onchanged.fireEvent(this, evt);
    }
};
```

11 Add and modify the on_fire_onclick object interface function as follows.

```
nexacro.ToggleButton.prototype.on_fire_onclick = function (button, alt_key,
ctrl_key, shift_key, screenX, screenY, canvasX, canvasY, clientX, clientY,
from_comp, from_refer_comp, meta_key)    {
```

```
    var ret = nexacro._CompositeComponent.prototype.on_fire_onclick.call(
this, button, alt_key, ctrl_key, shift_key, screenX, screenY, canvasX,
canvasY, clientX, clientY, from_comp, from_refer_comp, meta_key);
    if (!this.enable || this.readonly) {
        return false;
    }

    var pre_val = this.value;
    var post_val;
    if (this._isChecked(pre_val)) {
        post_val = false;
    } else {
        post_val = true;
    }

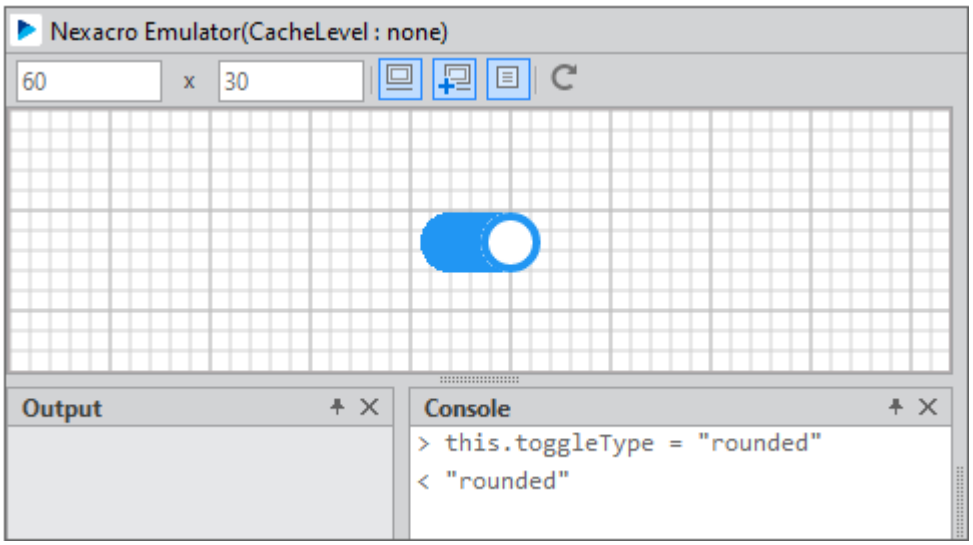
    var ret = this.on_fire_canchange(this, pre_val, post_val);
    if (ret) {
        if (this.applyto_bindSource("value", post_val)) {
            this._setValue(post_val);
        }

        if (pre_val !== post_val) {
            this._changebutton(post_val);
            this.on_fire_onchanged(this, pre_val, post_val);
        }
    }

    return ret;
};
```

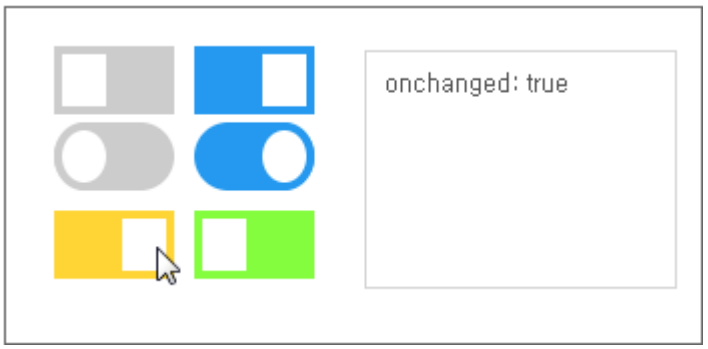
12 Check the functionality in the emulator.

- Whether the toggle animation plays when the component is clicked.
- Whether the button toggles and changes color when the component is clicked.
- Whether the property value is applied when the property is set through a script.



13 Deploy the module and install it in Nexacro Studio.

14 Place the ToggleButton component on the screen, run it in a web browser using QuickView (Ctrl + F6), and verify if the button functions and events are triggered.



Appendix B.

Example of the protocol adaptor

Here is an example of a protocol adapter developed in the Nexacro Module Developer. This code is written as an example, and the suitability of the code has not been verified.

B.1 JSON data processing

This is an example of receiving the JSON data and converting it to SSV format.

1. Create a new object with the Protocol Adapter. Set the object id to "JSONRecvAdp".

2. Edit the code as follows.

   Set a constant value in the initialize function, and convert the JSON data into SSV format in decrypt.

```
nexacro.JSONAdp.prototype.initialize = function()
{
    this._rs_ = String.fromCharCode(30); //Record Separator
    this._us_ = String.fromCharCode(31); //Unit Separator
};

nexacro.JSONAdp.prototype.decrypt = function(strUrl, strData)
{
    var oJsonData = JSON.parse(strData);
    // Stream Header
    var ssvData = "SSV:" + oJsonData.codepage + this._rs_;

    // Variables
    var arrObj = oJsonData.parameters;
    var arrKey = Object.keys(arrObj);
    if(arrKey)
```

```
{
    for(var i = 0; i < arrKey.length; i++)
    {
        if(i>0)
            ssvData +=this._rs_;
        ssvData += arrKey[i] + "=" + arrObj[arrKey[i]];
    }
    ssvData +=this._rs_;
}

// Datasets
for(var i = 0; i < oJsonData.datasets.length; i++)
{
    var dataset = oJsonData.datasets[i];
    // Dataset Header
    ssvData += "Dataset:" + dataset.ds_id + this._rs_;

    // Const Column Infos
    arrObj = dataset.ds_colinfo.constcolumn;
    arrKey = Object.keys(arrObj);
    if(arrKey)
    {
        ssvData += "_Const_"+this._us_;
        for(var j = 0; j < arrKey.length; j++)
        {
            if(j>0)
                ssvData +=this._us_;
            ssvData += arrKey[j] + "=" + arrObj[arrKey[j]];
        }
        ssvData += this._rs_;
    }

    // Column Infos
    var column = dataset.ds_colinfo.column;
    for(var j = 0; j < column.length; j++)
    {
        var cols = column[j];
        if(j == 0)
        {
            if(cols.id == "_RowType_")
            {
```

```
                    ssvData += cols.id;
                }
                else
                {
                    ssvData += "_RowType_";
                    ssvData += this._us_ + cols.id+ ":" + cols.type + "(" +
cols.size +")";
                }
            }
            else
            {
                ssvData += this._us_ + cols.id+ ":" + cols.type + "(" +
cols.size +")";
            }
        }
        ssvData += this._rs_;

        // Records
        var row = dataset.ds_rows.row;
        for(var j = 0; j < row.length; j++)
        {
            var recode = row[j];
            arrKey = Object.keys(recode);

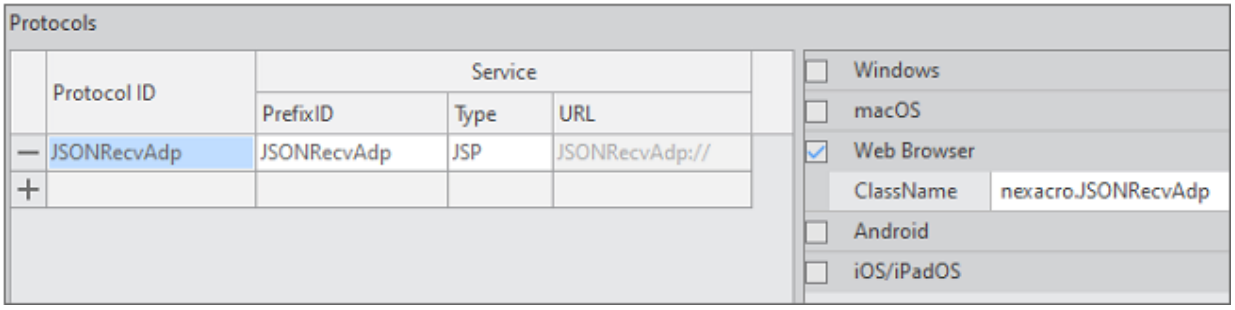
            if(arrKey[0] == "_RowType_")
            {
                ssvData += "N";
                var nCol = 1;
            }
            else
            {
                ssvData += "N";
                var nCol = 0;
            }
            for(; nCol < arrKey.length; nCol++)
            {
                if(recode[arrKey[nCol]] == null)
                {
                    ssvData += this._us_ + "";
                }
                else
```

```
                {
                    ssvData += this._us_ + recode[arrKey[nCol]];
                }
            }
            ssvData += this._rs_;
        }
        // Null Record
        ssvData += this._rs_;
    }
    trace(ssvData);
    return ssvData;
};
```

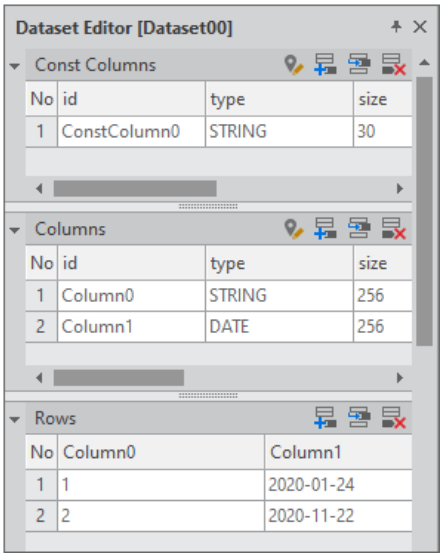
3. Select the menu [Deploy > Deploy Package] to run the Deploy Wizard.

4. Click the [Deploy] button to deploy the module installation file.

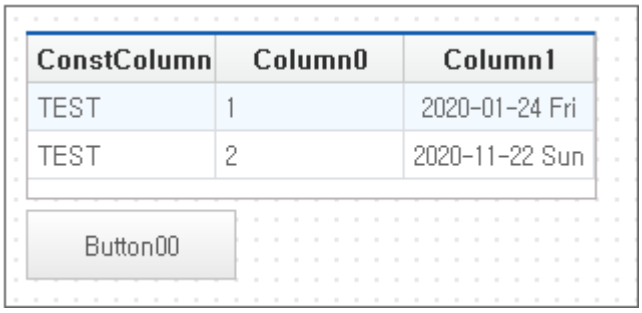
5. Install modules in Nexacro Studio and add protocol adapters.



6. Add a Dataset object and set it as follows.



7 Add the Grid and Button components to the screen and bind the Dataset object to the Grid component.



8 Write the onclick event handler function of the Button component as follows.

Set the nDataType parameter to 2 (SSV) because the JSON data will be converted to SSV format for processing.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.transaction("srv00", "JSONRecvAdp://127.0.0.1:4098/test.json", "",
"Dataset00=dsJson","","fnCallback", true, 2);
};

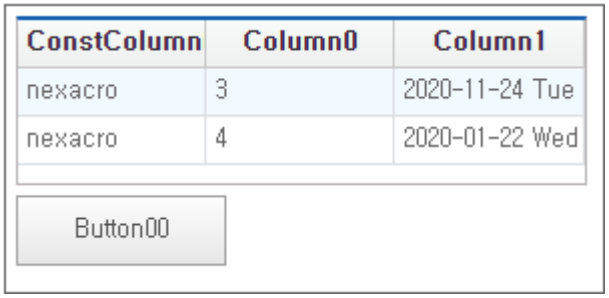
this.fnCallback = function(id, code, message)
{
    trace("Error["+code+"]:"+message);
}
```

9 Create a JSON file (test.json) in the Output folder as follows.

```
{
    "datatype": "JSON",
    "codepage": "utf-8",
    "parameters": {
        "ErrorCode": "0",
        "ErrorMsg": "OK"
    },
    "datasets": [
        {
            "ds_id": "dsJson",
            "ds_colinfo": {
                "constcolumn": {
```

```
                    "ConstColumn0": "nexacro"
                },
                "column": [
                    {
                        "id": "Column0",
                        "type": "STRING",
                        "size": "256"
                    },
                    {
                        "id": "Column1",
                        "type": "DATE",
                        "size": "256"
                    }
                ]
            },
            "ds_rows": {
                "row": [
                    {
                        "Column0": "3",
                        "Column1": "20201124"
                    },
                    {
                        "Column0": "4",
                        "Column1": "20200122"
                    }
                ]
            }
        }
    ]
}
```

10 After running it in a web browser with QuickView (Ctrl + F6), check if the data set in the JSON file is reflected when the button is clicked.



You can see the string converted to SSV data and a success message in the web browser console window.



B.2 Symmetric key encryption and decryption processing (crypto-js)

This is an example using an open source crypto-js. Encryption and decryption are applied when a protocol adapter is applied.

> Refer to the following URL for the crypto-js.
> https://code.google.com/archive/p/crypto-js/
> The CryptoJS v3.1.2 was used for the sample below.
> A license notice must be included when distributing modules including crypto-js.
> https://code.google.com/archive/p/crypto-js/wikis/License.wiki

1. Create a new object with the Protocol Adapter. Set the object id to "cryptojsAdp".

2. Edit the code as follows.

   Set the symmetric key ("nexacro platform") and the initialization vector values in the initialize function. Encryption and decryption are processed in the encrypt and decrpt, and the original data and converted data are displayed using the trace method.

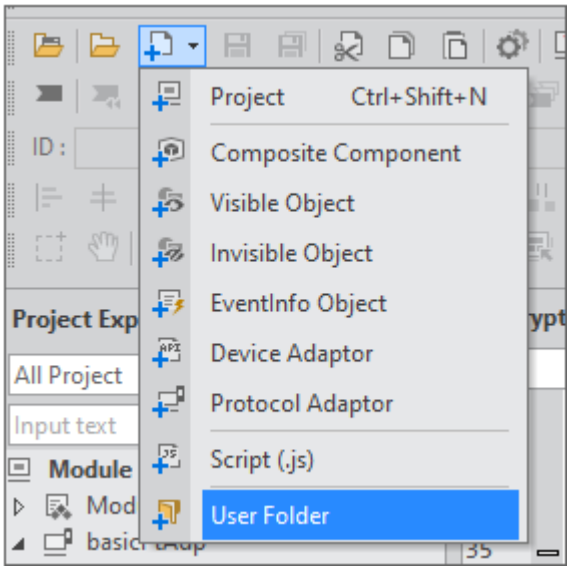
```
nexacro.cryptojsAdp.prototype.initialize = function()
{
    this._key = CryptoJS.enc.Utf8.parse("nexacro platform");
    this._iv = CryptoJS.enc.Utf8.parse(1234567812345678);
};

nexacro.cryptojsAdp.prototype.encrypt = function(strUrl, strData)
{
```

```
    var encrypted = CryptoJS.AES.encrypt(strData, this._key, {
        iv: this._iv,
        mode:CryptoJS.mode.CBC,
        padding:CryptoJS.pad.ZeroPadding
    });
    trace("strData: "+strData);
    trace("encrypted.toString(): "+encrypted.toString());
    return encrypted.toString();
};

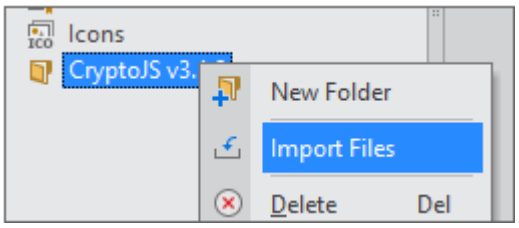
nexacro.cryptojsAdp.prototype.decrypt = function(strUrl, strData)
{
    var decrypted = CryptoJS.AES.decrypt(strData, this._key, {
        iv:this._iv,
        padding:CryptoJS.pad.ZeroPadding
    });
    trace("strData: "+strData);
    trace("decrypted: "+decrypted.toString(CryptoJS.enc.Utf8));
    return decrypted.toString(CryptoJS.enc.Utf8);
};
```

3 Add User Folder.



4 Change the name of User Folder to “CryptoJS v3.1.2”.

5 Select User Folder and select [Import Files] from the context menu.



6 Select the two files below in Explorer and add them to User Folder.

```
CryptoJS v3.1.2
+ rollups > aes.js
+ components > pad-zeropadding-min.js
```

7 Select the menu [Deploy > Deploy Package] to run the Deploy Wizard.

8 Click [Next] and move to the JSON editing screen.

9 Add the js file added to the User Folder to the scripts item.

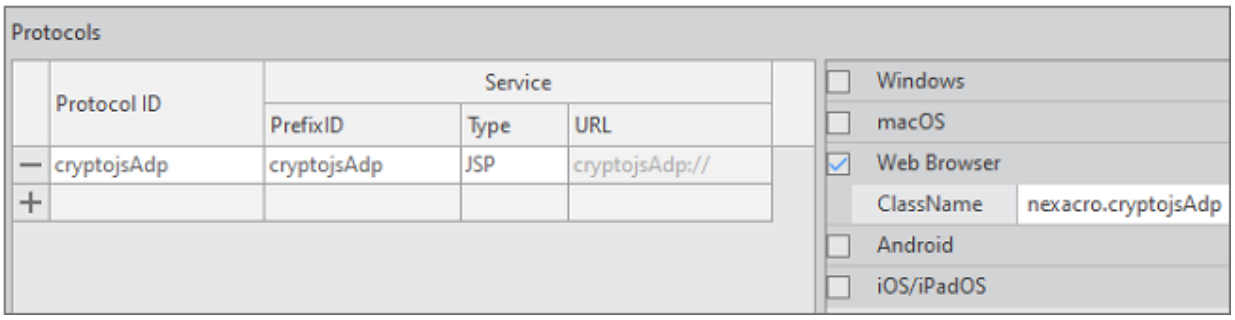
```
{
    "name": "cryptojsAdp",
    "moduletype": "protocoladaptor",
    "objInfo": [
        "cryptojsAdp/_metainfo_/cryptojsAdp.info"
    ],
    "scripts": [
        "cryptojsAdp/CryptoJS v3.1.2/aes.js",
        "cryptojsAdp/CryptoJS v3.1.2/pad-zeropadding-min.js",
        "cryptojsAdp/cryptojsAdp.js"
    ],
    "userfolder": [
        "cryptojsAdp/CryptoJS v3.1.2"
    ]
}
```



The edited JSON scripts are not saved. They need to be manually edited when distributing.

10 Click the [Deploy] button to deploy the module installation file.

11 Install modules in Nexacro Studio and add protocol adapters.



12 Enter the transaction script in the screen as follows.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.transaction("srv00", "cryptojsAdp://127.0.0.1:4098/test.xml", "",
"dsOut=Dataset00","value=a","fnCallback");
};

this.fnCallback = function(id, code, message)
{
    trace("Error["+code+"]:"+message);
}
```

13 After running it in a web browser with QuickView (Ctrl + F6), check if the data is encrypted and decrypted when the button is clicked.



The test.xml file will return a string that encrypts the request value for the functional test. In this example, the request value is decrypted and returned.

```
strData: <?xml version="1.0" encoding="UTF-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset">
    <Parameters>
        <Parameter id="value">a</Parameter>
    </Parameters>
</Root>

encrypted.toString(): 3TlyqNLXxHG8CN1MMIb3vAO/KGkU1OSAFl1wuMvAtEfq6jS6Vp95
xvFLhCH/KKrSr5svsoRf1zi8lYmuQ0QMecYIPS+gz9KjeVCdwZ1F9/n/BQIrhN2ahuVDlQevU0
+1fwirvXakwR6whvDPPY4ecgDgv7c3kCRVpxjYQk3AKJnZcKTwscCmfolfx9JzHmhq8R6SQKS9
C8YJ/ju8j+VI2ZstQCoLU9JaJ0ATZbwGCmE=
```

```
strData: 3TlyqNLXxHG8CN1MMIb3vAO/KGkU1OSAFl1wuMvAtEfq6jS6Vp95xvFLhCH/KKrSr
5svsoRf1zi8lYmuQ0QMecYIPS+gz9KjeVCdwZ1F9/n/BQIrhN2ahuVDlQevU0+1fwirvXakwR6
whvDPPY4ecgDgv7c3kCRVpxjYQk3AKJnZcKTwscCmfolfx9JzHmhq8R6SQKS9C8YJ/ju8j+VI2
ZstQCoLU9JaJ0ATZbwGCmE=


decrypted.toString(CryptoJS.enc.Utf8): <?xml version="1.0"
encoding="UTF-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset">
    <Parameters>
        <Parameter id="value">a</Parameter>
    </Parameters>
</Root>
```